



Valentijn Jacobs

500905079

+31 6 14 98 44 48

valentijnjacobs@me.com

Hogeschool van Amsterdam, HBO-ICT

Game development

Stagedocent:

Reggie Schildmeijer

Februari '26 – Juni '26 Leerjaar 4

Voorwoord

Dit onderzoek is opgesteld in het kader van mijn afstudeerstage bij Domits. Tijdens deze stage stond analyse, ontwerp en realiseren van een template gebaseerde Direct Booking Website centraal. Het doel hiervan was om hosts op een laagdrempelige manier een eigen directe boeking mogelijkheid te bieden, zonder dat zij zelf een website hoeven te ontwikkelen of technisch hoeven te beheren. Daarbij moest deze feature niet alleen visueel bruikbaar zijn, maar ook correct aansluiten op bestaande data uit Domits haar Property Management System, waaronder beschikbaarheid, actuele prijzen en boekingen.

Tijdens dit traject kwamen onderzoek en praktijk samen. Aan de ene kant was het nodig om architectuur keuzes, hostingstrategieën, multi-tenant inrichting, security en domeinbeheer te onderbouwen. En aan de andere kant moest de onderbouwing leiden tot een werkend product binnen de bestaande Domits structuur. De combinatie van analyseren, afwegen, bouwen en itereren maakte dit onderzoek waardevoller. Ik wil Domits bedanken voor de ruimte om aan een product te werken dat direct relevant is voor het platform, en mijn stagedocent en betrokken begeleiders voor hun feedback en begeleiding gedurende dit proces.

Samenvatting

In dit onderzoek staat de vraag centraal hoe Domits een template-gebaseerde, one-click Direct Booking Website kan ontwerpen en realiseren die schaalbaar, veilig en kostenefficiënt is, en tegelijkertijd correct blijft aansluiten met het bestaande Property Management Systeem. De aanleiding voor dit onderzoek ligt in de behoefte om hosts een eigen Direct Booking platform te bieden, zodat zij minder afhankelijk zijn van externe platformen, geen tarief hoeven in te ruilen en meer directe omzet kunnen genereren.

Om deze vraag te beantwoorden is een ontwerpgerichte onderzoeksaanpak gehanteerd. Eerst zijn de functionele en technische eisen uitgelijnd, waarna meerdere mogelijkheden zijn vergeleken op het gebied van template architectuur, hosting, domein beheer, veiligheid, multi tenant en PMS integratie. Hierdoor is een onderbouwd ontwerp opgesteld en vervolgens is deze vertaald naar een gerealiseerd product binnen het bestaande platform.

Het resultaat van dit onderzoek is een werkende eerste versie van de Direct Booking Website feature, waarmee hosts een listing kunnen selecteren, een layout kunnen kiezen, een website kunnen bouwen en wijzigingen als draft kunnen opslaan en in een voorvertoning omgeving kunnen bekijken. De feature is bewust opgebouwd rond een gedeeld template- en datamodel, zodat diverse websites schaalbaar beheerd kunnen worden zonder per template een los project te moeten onderhouden. Daarnaast is de editor-pagina gerealiseerd waarmee content en visuele elementen per website aangepast kunnen worden, terwijl technische structuur beheersbaar blijft.

Inhoudsopgave

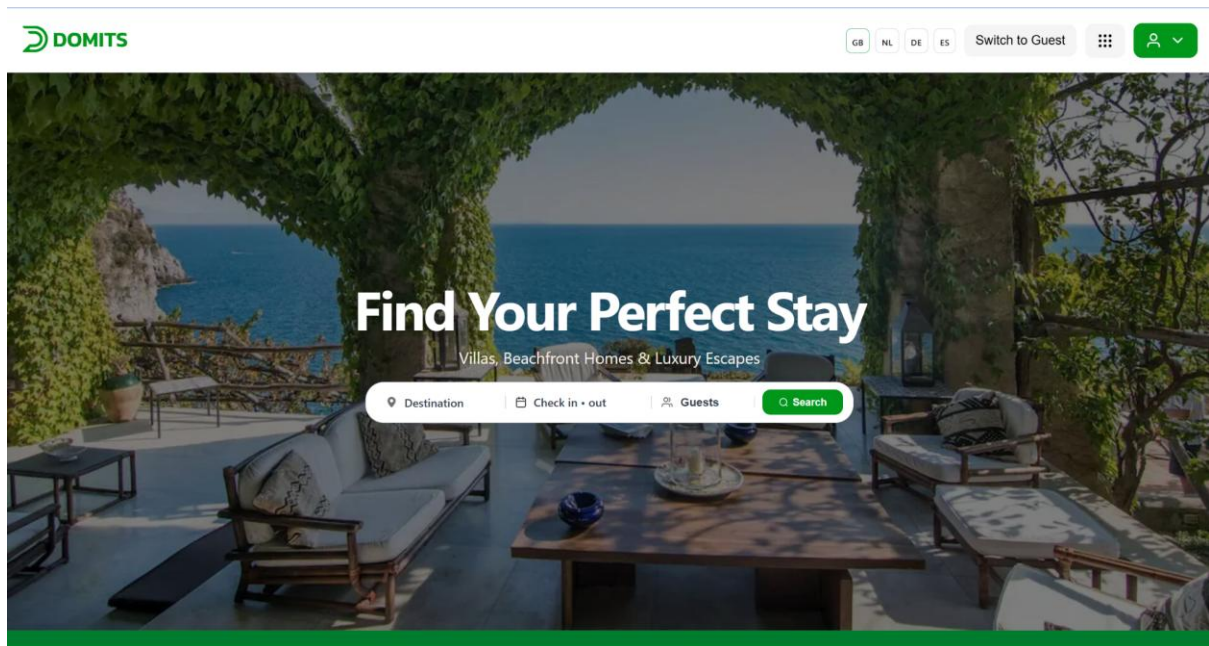
1. Inleiding	6
1.1 Aanleiding	6
1.2 Introductie opdracht	7
1.3 Opdrachtgever	7
1.4 Omschrijving organisatie	7
1.5 Doelstellingen	8
1.6 Onderzoeksvragen	8
1.7 Onderzoeksopzet en leeswijzer	9
2. Begrippenlijst	10
2.1 Inleiding	10
2.2 Begrippenlijst	10
3. Probleemanalyse	14
3.1 Domits en afbakening van de feature	14
3.2 Doel en resultaat van het onderzoek	15
3.3 Aanpak van het onderzoek	15
3.4 Planning onderzoek	16
3.5 Uitwerking probleemanalyse (6W-formule, Nel Verhoeven)	16
3.6 Formuleren van centrale vraagstelling	18
3.7 Formuleren van deelvragen	18
3.8 Formuleren van doelstelling	19
3.9 Validiteit en betrouwbaarheid	19
4. Onderzoeksaanpak	22
4.1 Onderzoeksmethode	22
4.2 Vergelijkingskader	23
4.3 Validatie en Meetplan	24
4.4 Inzet van AI tijdens de realisatie	26
5. Analyse	28
5.1 Minimale v1 functionaliteit	28
5.2 Template architectuur	31
5.3 Hosting strategieën	37
5.4 Multi-tenant architectuur	40
5.5 Routing en domeinbeheer	41

5.6 Data integratie	44
5.7 Security	48
5.8 KPI's	51
6. Ontwerp	55
6.1 Architectuur overzicht en datamodel	55
6.2 Template- en contentmodel	58
6.3 Website lifecycle en publicatieflow	60
6.4 Routing, preview links en domeinbeheer	62
6.5 PMS integratie: bezettingsgraad, prijs en boeking	64
6.6 Tenant isolatie en security	66
6.7 Deployment en rollout	67
7. Feature implementatie	69
8. Validatie & Evaluatie	74
8.1 KPI validatie	74
8.2 Validatie van scenario's en interpretatie	78
8.3 Conclusie van de validatie	79
9. Conclusie	80
Bronnenlijst (IEEE)	82
Bronnen	82
Bijlagen	87
Figuren	87
Tabellen	97

1. Inleiding

1.1 Aanleiding

Domits is een platform binnen de hospitality sector dat verhuurders ondersteunt bij het beheren van accommodaties, reserveringen, communicatie, beschikbaarheid en prijsinformatie. Binnen het platform vervult Domits hiermee een rol die aansluit op een Property Management System (PMS). Naast intern beheer wil Domits verhuurders ook beter ondersteunen in het aantrekken van directe boekingen via een eigen kanaal.



Figuur 1: Domits product

Vanuit die product richting is de wens ontstaan om een template gebaseerde Direct Booking Website te ontwikkelen. Met deze functionaliteit moet een verhuurder met beperkt technisch inzicht een professionele website kunnen publiceren, zonder zelf beheer of programmering uit te hoeven voeren. Tegelijkertijd moet deze website correct blijven aansluiten op bestaande data afkomstig van Domits, zoals beschikbaarheid, prijzen en boeking gerelateerde data.

De aanleiding voor dit onderzoek ligt bij twee behoeften. Aan de ene kant wil Domits de waarde van het platform vergroten door verhuurders een extra direct boekingskanaal te bieden en deze websites zelf te kunnen aanbieden aan klanten. Aan de andere kant brengt zo'n functionaliteit technische vraagstukken met zich mee op het gebied van schaalbaarheid, hosting, template architectuur, multi-tenancy, beveiliging en integratie met PMS data. Dit onderzoek richt zich op het onderbouwen en realiseren van een geschikte aanpak hiervoor.

1.2 Introductie opdracht

De opdracht richt zich op het ontwerpen en realiseren van een Direct Booking Website functionaliteit binnen Domits. Deze functionaliteit moet verhuurders de mogelijkheid geven om op basis van een bestaande listing en een gekozen layout snel een eigen website samen te stellen. De website functioneert als een directe boeking mogelijkheid en moet daarom niet alleen visueel bruikbaar zijn, maar ook correct aansluiten op data afkomstig uit het bestaande PMS.

Binnen deze opdracht staat een eerste werkbare versie centraal. Dat houdt in dat niet alle gewenste functionaliteiten direct hoeven te worden gerealiseerd, maar dat de eerste versie wel voldoende waarde moet bieden voor verhuurders en daarbij ook technisch beheersbaar moet blijven voor Domits. De opdracht vraagt om onderzoek naar architectuur- en ontwerpkeuzes zowel als een vertaling naar een werkende feature.

1.3 Opdrachtgever

De opdrachtgever van dit onderzoek is Domits. Binnen de opdracht zijn vooral het product- en development team relevante stakeholders, omdat zij betrokken zijn bij de richting, de technische haalbaarheid en de verdere ontwikkeling van de feature. Daarnaast vormen toekomstige hosts op het platform een belangrijk groep, omdat zij uiteindelijk degene zijn die met deze Direct Booking Website moeten kunnen werken.

1.4 Omschrijving organisatie

Domits is een kleinschalig en groeiend hospitality platform dat zich richt op het ondersteunen van verhuurders bij het beheren van accommodaties en de bijbehorende processen. Binnen het platform komen verschillende onderdelen samen, zoals listing beheer, reserveringen, communicatie, prijsinformatie en beschikbaarheid. Hierdoor ontwikkelt Domits zich verder in de richting van een breder PMS systeem voor de short-term huurmarkt.

Doordat het bedrijf zich nog in een ontwikkelingsfase bevindt, worden nieuwe functionaliteiten in korte tijd ontwikkeld en gevalideerd. Dit vraagt om oplossingen die niet alleen aantrekkelijk zijn, maar ook schaalbaar, onderhoudbaar en kostenefficiënt blijven. De Direct Booking Website feature past binnen deze context, omdat de functionaliteit zowel interessant als technisch goed moet blijven aansluiten op de bestaande platform structuur.

1.5 Doelstellingen

Met dit onderzoek worden de volgende doelstellingen nagestreefd:

- Verhuurders moeten met minimale inspanning een eigen Direct Booking Website kunnen publiceren
- Een template aanpak moet ontworpen worden die customization ondersteunt zonder dat elk template een los project wordt
- Hosting en deployment moeten worden ingericht, zodat de aanpak en gekozen oplossing schaalbaar en kostenefficiënt blijft
- Correcte integratie moet worden gewaarborgd met betrekking tot beschikbaarheid, prijzen en boekingen uit het PMS
- Security moet worden meegenomen binnen een multi-tenant omgeving

1.6 Onderzoeksvragen

Om te bepalen hoe een schaalbare en veilige Direct Booking Website binnen Domits gerealiseerd kan worden, is een centrale onderzoeksvraag opgesteld. Deze wordt onderverdeeld in meerdere deelvragen, die elk een specifiek onderdeel van het probleem aanspreken, zoals functionaliteit, architectuur, website hosting, integratie, achterwaartse compatibiliteit en beveiliging.

De centrale hoofdvraag luidt als volgt:

Hoofdvraag

Hoe kunnen we een template-gebaseerde, one-click standalone booking website binnen Domits ontwerpen zodat deze schaalbaar, veilig en kostenefficiënt te hosten is, terwijl hij correct integreert met beschikbaarheid en boekingen uit het Property Management System?

Deelvragen

1. *Welke minimale functionaliteit moet de standalone website bevatten om als v1 waardevol en haalbaar te zijn binnen Domits?*
2. *Welke template-architectuur is het meest geschikt om herbruikbare websites met beperkte maar bruikbare customization mogelijk te maken zonder dat elke template een apart project wordt?*
3. *Welke hostingstrategie is het meest geschikt voor Domits, gegeven de eisen rondom kosten, performance, schaalbaarheid en beheerbaarheid?*

- 4. Welke multi-tenant architectuur is voor de Direct Booking Website het meest geschikt om schaalbaarheid, kostenefficiëntie en tenant-isolatie te waarborgen?*
- 5. Hoe kan multi-tenant routing en domeinbeheer worden ingericht zodat gegenereerde subdomains en latere custom domains betrouwbaar en schaalbaar functioneren?*
- 6. Hoe kan de standalone website correct blijven aansluiten op availability- en pricingdata uit het PMS, ook bij wijzigingen, caching en foutscenario's?*
- 7. Welke securitymaatregelen zijn nodig om een multi-tenant standalone websiteplatform veilig te realiseren binnen Domits?*
- 8. Welke KPI's zijn geschikt om de technische en productmatige kwaliteit van de oplossing te evalueren?*

1.7 Onderzoekopzet en leeswijzer

Dit onderzoek volgt een ontwerpgerichte aanpak waarin analyse, afweging, ontwerp, realisatie en validatie samenkomen. Eerst worden de functionele en technische eisen van de feature in kaart gebracht, zodat duidelijk wordt welke onderdelen noodzakelijk zijn voor een eerste waardevolle versie en welke onderdelen beter doorgeschoven kunnen worden naar een latere implementatiefase.

Vervolgens worden meerdere oplossingsrichtingen onderzocht en met elkaar vergeleken. Daarbij wordt gekeken naar onderwerpen, zoals template architectuur, hosting strategie, domeinbeheer, multi-tenancy, PMS integratie en beveiliging. De diverse mogelijkheden worden beoordeeld op criteria, zoals schaalbaarheid, kosten, onderhoud, betrouwbaarheid en haalbaarheid binnen een v1 scope.

Op basis van deze analyse wordt daarna een onderbouwd ontwerp vastgesteld. Dit ontwerp vormt de basis voor de realisatie van een eerste werkende versie binnen de bestaande structuur van Domits. Daarbij staat niet alleen de technische kant centraal, maar ook de vraag of de gekozen aanpak toepasbaar is binnen het huidige platform.

Tot slot wordt de gerealiseerde oplossing gevalideerd aan de hand van KPI's. Hiermee wordt beoordeeld in hoeverre de huidige feature voldoet aan de vastgestelde eisen op het gebied van gebruik, performance, betrouwbaarheid, kosten en correcte PMS connectie.

Na de probleemanalyse volgt de analyse per deelvraag om te onderzoeken welke oplossingsmogelijkheden het meest geschikt zijn. Op basis daarvan worden de meest geschikte keuzes uitgewerkt binnen het ontwerp hoofdstuk. Vervolgens wordt de gerealiseerde feature beschreven, waarna de oplossing wordt gevalideerd en geëvalueerd.

2. Begrippenlijst

2.1 Inleiding

In deze begrippenlijst worden de belangrijkste termen uit het onderzoek kort toegelicht. De lijst ondersteunt de leesbaarheid van het onderzoek.

2.2 Begrippenlijst

Begrip	Betekenis
PMS (Property Management System)	Systeem voor het beheren van accommodaties, reserveringen, prijzen, beschikbaarheid en bijbehorende host processen.
Direct Booking Website	Nieuwe Domits feature waarmee een host een eigen direct boekingskanaal krijgt, die losstaat van externe platformen
Standalone website	Een losstaande publieke website die buiten het host dashboard zichtbaar is, maar wel gekoppeld blijft aan Domits data.
Direct booking	Een boeking die rechtstreeks via het eigen kanaal van een verhuurder plaatsvindt, zonder dat een extern platform hier tussenkomt.
OTA (Online Travel Agency)	Een extern boekingsplatform, zoals Airbnb of Booking.com, waarop accommodaties aangeboden en geboekt kunnen worden.
Template gebaseerde website	Website waarvan de structuur en layout vooraf vastgesteld zijn, terwijl inhoud en stijl binnen bepaalde grenzen aangepast kan worden.
Template architectuur	De manier waarop templates technisch zijn opgebouwd en herbruikbaar worden gemaakt binnen hetzelfde systeem.
Gedeeld datamodel	Datastructuur waarin listing data eerst wordt omgezet, voordat deze door verschillende templates wordt gebruikt.
Override-laag	Laag boven op een vast template structuur, waarmee inhoud of stijl aangepast kunnen worden, zonder de onderliggende pagina architectuur te wijzigen.

One-click publicatie	Het geautomatiseerd opzetten of publiceren van een website, zonder handmatige infrastructuur stappen te ondernemen per host.
Multi-tenancy	Een platform dat meerdere gebruikers of hosts ondersteunt, waarbij data en configuratie van elkaar gescheiden blijven.
Tenant-isolatie	Het technisch scheiden van data en toegangsrechten tussen verschillende hosts binnen hetzelfde platform.
Shared runtime	Gedeelde infrastructuur, waarin meerdere websites of tenant op dezelfde technische basis draaien.
Statische hosting	Hosting manier, waarbij content vooraf wordt opgebouwd en vervolgens als vaste website bestand wordt aangeboden.
Server-side rendering (SSR)	Rendering strategie, waarbij pagina inhoud per aanvraag op de server wordt opgebouwd.
Hybride hosting	Hosting manier, waarbij statische content efficiënt geleverd wordt, terwijl dynamische gegevens, zoals prijs en beschikbaarheid, server-side up to date blijven.
Generated subdomain	Automatisch gegenereerd subdomein waarmee een Direct booking Website bereikbaar wordt gemaakt voor bezoekers.
Fallback subdomain	Standaard terugval route voor de website, zodat deze ook zonder custom domain publiek bereikbaar blijft.
Custom domain	Eigen domeinnaam van een host die aan de Direct Booking Website gekoppeld kan worden.
Draft	Niet-gepubliceerde versie van een website, waarin wijzigingen opgeslagen kunnen worden.
Preview	Voorvertoning van een website, voordat deze publiek gepubliceerd gepubliceerd wordt.
Public preview	Publieke weergave van een website die gebruikt wordt om de versie buiten het host dashboard te bekijken.
Live website	De gepubliceerde versie van de Direct Booking Website.
Publicatie flow	het proces waarmee een draft versie wordt omgezet

	naar een bereikbare live website.
Live synchronisatie	Het proces waarbij wijzigingen uit de draft of editor worden doorgezet naar de gepubliceerde website.
Editor	De beheeromgeving, waarin een host de inhoud en visuele elementen van de website kan aanpassen.
Builderflow	Het proces waarin een host van een bestaande listing en template keuze een eerste website opbouwt.
Availability	Beschikbaarheid informatie van een accommodatie, zoals boekingen of geblokkeerde dagen.
Pricing	Prijs informatie van een accommodatie, zoals basisprijs of dynamische prijs.
Booking engine	De flow die beschikbaarheid toont, prijzen berekent en uiteindelijk boekingen ondersteunt.
iCal synchronisatie	Koppeling waarbij kalender gerelateerde data uit externe bronnen wordt uitgelezen en verwerkt.
Caching	Tijdelijk opslaan van data of content om prestatie te verbeteren en onnodige herhaalde verzoeken te beperken.
Server-side validatie	Controle van data, rechten en toegang via de server, in plaats van alleen via de frontend.
Rate limiting	Techniek om het aantal verzoeken te beperken, zodat misbruik en overbelasting voorkomen wordt.
CSP (Content Security Policy)	Browser beleid waarmee risico's, zoals ongewenste scripts beperkt worden.
KPI (Key Performance Indicator)	Meetpunt waarmee de kwaliteit, het gebruik of de prestatie van de feature beoordeeld wordt.
LCP (Largest Contentful Paint)	Prestatie meting die aangeeft hoe snel het grootste zichtbare element op de pagina geladen wordt.
Core Web Vitals	Set van indicatoren van Google die inzicht geven in de gebruikerservaring van websites.
p95 / p75	Percentielen die aangeven binnen welke grens 95% of 75% van de metingen valt.
Abandonment rate	Het percentage gestarte builder- of procesflows die niet worden afgerond.
Time to first preview	De tijd tussen het starten van het website

	bouwproces en het moment waarop de preview beschikbaar is voor de host.
Time to publish	De tijd tussen verzoek van publicatie en het moment waarop de live website en link zijn bijgewerkt.
Fallback subdomein availability	KPI die aangeeft in hoeverre gepubliceerde websites bereikbaar blijven via het terugval subdomein.
Quote-to-charge mismatch rate	KPI die inzicht geeft over afwijkingen tussen prijsinformatie of betaalafhandelingen.
V1	De eerste afgebakende en werkbare versie van de feature, gericht op de minimale waardevolle scope.
V2	Toekomstige uitbreiding fase, waarin extra functionaliteiten buiten de eerste scope toegevoegd kunnen worden.
MVP (Minimal Viable Product)	Minimale versie van een product dat voldoende functionaliteit bevat om waarde te leveren.

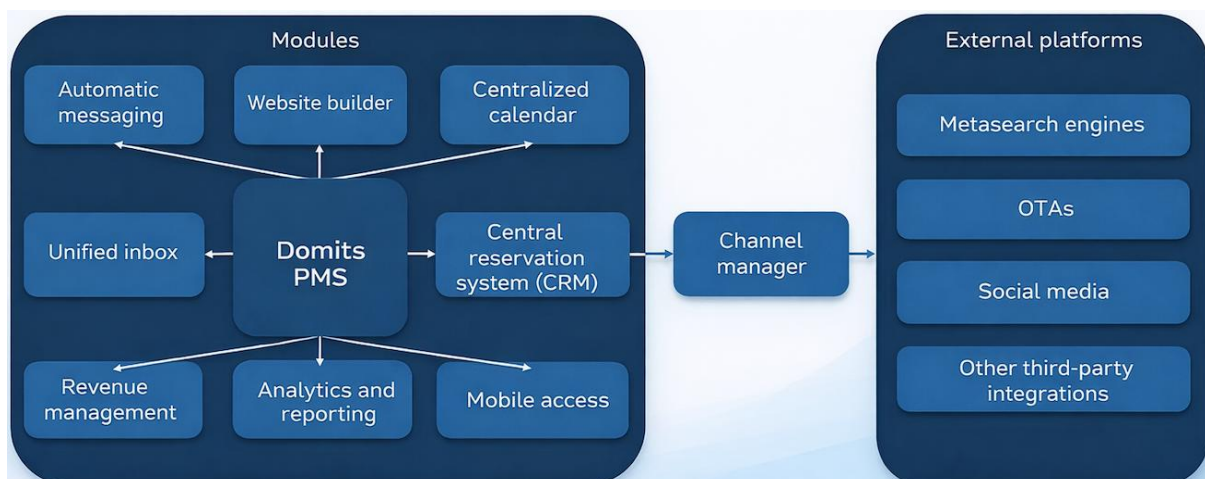
3. Probleemanalyse

3.1 Domits en afbakening van de feature

Domits is een hospitality platform dat verhuurders ondersteunt bij het beheren van accommodaties, reserveringen, beschikbaarheid, informatie omtrent prijs en communicatie. Binnen het platform komen verschillende processen samen die passen binnen de werking van het Property Management System (PMS). Daardoor vormt Domits niet alleen een omgeving voor het tonen van accommodatie, maar ook een centrale plek waar gegevens, zoals pricing, availability en bookings, worden beheerd en bijgewerkt.

Binnen deze bestaande structuur wordt in dit onderzoek gekeken naar een nieuwe feature: de Direct Booking Website. Deze functionaliteit moet verhuurders in staat stellen om een bestaande listing snel om te toveren in een website, zonder zich druk te hoeven maken over beheer of implementatie. De feature staat daarmee niet los van Domits, maar bouwt voort op al aanwezige data en processen in het platform.

Het aandeel van dit onderzoek ligt bij het ontwerpen en realiseren van deze Direct Booking Website functionaliteit. Daarbij gaat het niet om het opnieuw ontwerpen van het volledige PMS, maar om het toevoegen van een direct boekingskanaal dat correct moet aansluiten op al bestaande listing data, beschikbaarheid, prijzen en publicatie processen. Omdat deze feature met meerdere onderdelen van Domits in contact komt, waaronder frontend, backend, routing, domeinbeheer en PMS integratie, is een duidelijke probleemanalyse noodzakelijk.



Figuur 2: Positie van de Website Builder binnen Domits, als extra direct boekingskanaal bovenop bestaande PMS data.

3.2 Doel en resultaat van het onderzoek

Het doel van dit onderzoek is om te bepalen hoe Domits een template gebaseerde Direct Booking Website kan realiseren die technisch haalbaar, schaalbaar, veilig en kostenefficiënt is, en die correct integreert met data uit het huidige Property Management System binnen Domits. Om deze feature te valideren wordt onderzocht welke functionele en technische keuzes het meest geschikt zijn voor een eerste versie van het product.

Het onderzoek moet leiden tot een onderbouwde oplossing voor de nieuwe website feature binnen Domits. Het resultaat bestaat uit een gerealiseerde en bruikbare eerste versie van de feature, ondersteund door keuzes binnen architectuur, afwegingen binnen ontwerp en evaluatie door middel van Key Performance Indicators. Op basis hiervan kan worden beoordeeld in hoeverre de implementatie voldoet aan de eisen op het gebied van schaalbaarheid, veiligheid, kostenbeheersing en correcte PMS integratie.

3.3 Aanpak van het onderzoek

Het onderzoek ondervindt als allereerst een ontwerpgerichte aanpak, waarbij eerst het probleem en de voorwaarden/vereisten worden geanalyseerd en de verschillende oplossingen met elkaar worden vergeleken en afgewogen. Op basis van deze informatie wordt een onderbouwde ontwerpkeuze gemaakt, die daarna wordt gerealiseerd en gevalideerd.

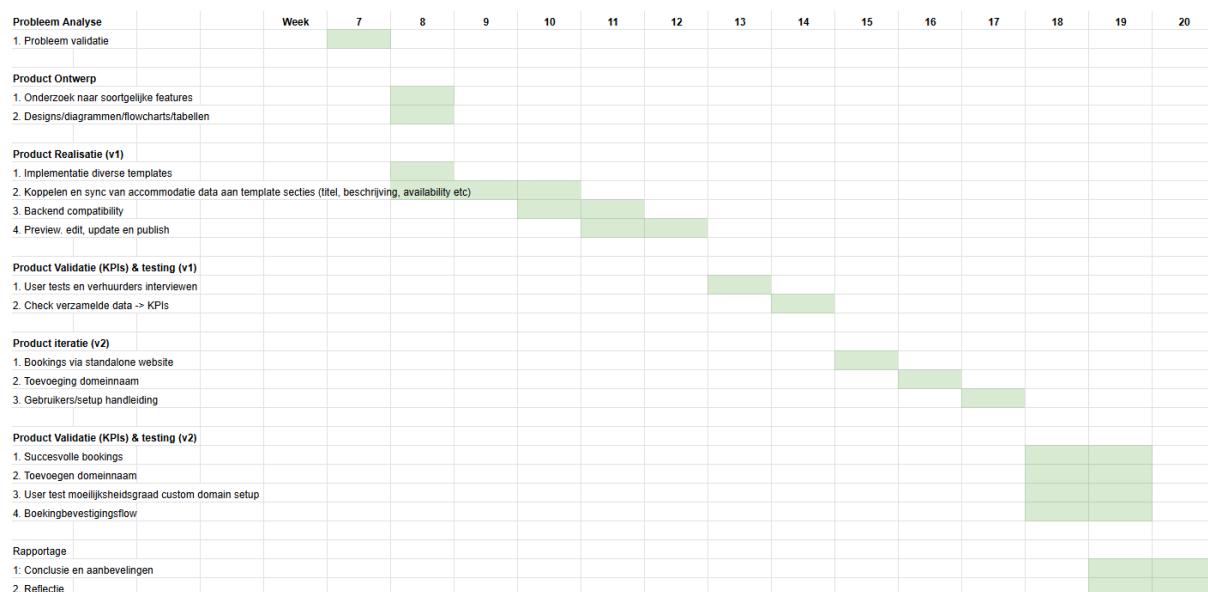
Deze aanpak bestaat uit de volgende stappen:

- Vereistenanalyse: inventarisering van functionele en technische eisen. Bij deze analyse wordt gekeken naar de wensen van stakeholders, potentiële weerhoudingen en de afbakening van een eerste versie (v1 scope).
- Oplossingsrichtingen: een vergelijking van mogelijkheden binnen template architectuur, hostingstrategie, domeinbeheer, PMS integratie en beveiliging.
- Kritische bepaling: afweging van alternatieven op basis van criteria zoals kosten, schaalbaarheid, performance, onderhoudbaarheid, complexiteit en risico's.
- Ontwerp en realisatie: uitwerking van de gekozen richtingen in een concreet ontwerp en een gerealiseerde feature of uitgebreid prototype.

- Validatie: toetsen van het ontwerp en de realisatie door middel van KPI's/metrics, user tests, stakeholders hun inbreng, en fout scenario's, zoals publicering, gast ervaring en beschikbaarheid/prijs toning correctheid.

3.4 Planning onderzoek

Om de beschikbare tijd voor dit onderzoek goed te waarborgen en af te bakenen, is er een planning opgesteld waarbij de benodigde punten uitgelijnd en wekelijks ingedeeld zijn. Deze planning gaat hand in hand met het streven naar een goed werkend product dat hieruit voortvloeit.



Figuur 3: Opgestelde planning

3.5 Uitwerking probleemanalyse (6W-formule, Nel Verhoeven)

Om het probleem rondom Direct Booking Websites binnen Domits gestructureerd en volledig te analyseren, wordt gebruikgemaakt van de 6W-formule van Nel Verhoeven [49]. De beantwoording van deze 6W's is niet alleen gebaseerd op eigen observatie van het platform, maar ook op afstemming met mijn bedrijfsbegeleider Stefan Hopman. Tijdens gesprekken is besproken wat het probleem precies is, voor wie het relevant is, waarom het bedrijf deze functionaliteit nastreeft en welke risico's daarbij een rol spelen. De methode helpt om het probleem vanuit verschillende perspectieven te bekijken, zoals betrokken stakeholders, oorzaken, context en impact.

Wat is het probleem?

Verhuurders missen een simpele manier om snel een professionele directe booking website te publiceren die gekoppeld en betrouwbaar is, met betrekking tot beschikbaarheid, dagprijs en boekingen binnen het PMS. Daarbij hanteren hospitalityplatformen, zoals Domits en Airbnb een verhuurtarief (meestal 10-20% per booking). Graag zou een verhuurder alle verdiensten in eigen zak willen houden.

Wie heeft het probleem?

Verhuurders en of vastgoedbeheerders zijn momenteel afhankelijk van hospitalityplatformen hun website gebruik m.b.t. boekingsaanvragen. Zo willen verhuurders direct boekingen kunnen ontvangen zonder dat een platform hier tussen zit.

Wanneer is het probleem ontstaan?

Bij de ambitie om directe boekingen te ondersteunen naast bestaande functionaliteiten op boekingsplatformen, zonder dat hosts zelf websites hoeven te bouwen of te beheren.

Waarom is het een probleem?

Zonder goede oplossing blijft de verhuurder afhankelijk van andere Online Travel Agencies (OTAs), stijgen operationele kosten en ontstaat risico op foutieve beschikbaarheid en/of prijsweergave, wat leidt tot gemiste omzet of reputatieschade.

Waar doet het probleem zich voor?

Op Domits haar platform: het publiceren/aanbieden van websites en de integratie met huidige PMS-data (beschikbaarheid, prijzen, boekingen), inclusief domein beheer en beveiliging.

Wat is de aanleiding?

Strategische behoefte om directe boekingen te stimuleren en hosts een snelle, professionele digitale aanwezigheid te bieden met een minimale technische drempel. Daarbij biedt Domits deze functionaliteit om haar positie als Property Management System nog aantrekkelijker te maken voor meer verhuurders.

3.6 Formuleren van centrale vraagstelling

Op basis van de probleemanalyse wordt de volgende centrale onderzoeksvraag geformuleerd:

“Hoe kan Domits een template-gebaseerde, one-click direct booking website ontwerpen die schaalbaar, veilig en kostenefficiënt is, en correct integreert met availability- en pricingdata uit het Property Management System?”

Deze vraag richt zich op de zowel technische als de functionele aspecten van de oplossing en vormt de basis voor het verdere onderzoek.

3.7 Formuleren van deelvragen

Om de centrale onderzoeksvraag gestructureerd te kunnen beantwoorden, is deze opgedeeld in meerdere deelvragen. Elke deelvraag bedraagt een afzonderlijk onderdeel van het probleem, zodat niet alleen duidelijk wordt welke oplossing mogelijk is, maar ook waarom de oplossing voor Domits het meest geschikt is.

De eerste deelvraag richt zich op de minimale functionaliteiten van een v1. Hiermee wordt bepaald welke onderdelen nodig zijn om waarde te leveren, en welke onderdelen beter naar een latere implementatiefase kunnen worden doorgeschoven. De tweede en derde deelvraag richten zich op template architectuur en hosting strategie. Hiermee wordt onderzocht hoe de websites technisch opgebouwd kunnen worden zonder dat kosten, beheer en uitbreidbaarheid uit de hand lopen.

De vierde en vijfde deelvraag gaan over multi-tenancy, routing en domeinbeheer. Deze vragen zijn relevant, omdat Domits meerdere websites voor verschillende verhuurders wil ondersteunen binnen dezelfde platform structuur, zonder dat data door elkaar loopt of websites aparte infrastructuur vereisen.

De zesde deelvraag richt zich op de aansluiting met beschikbaarheid en prijs data uit het PMS, omdat deze koppeling essentieel is voor de betrouwbaarheid van de feature.

De zevende deelvraag behandelt veiligheid, omdat een directe boeking website alleen bruikbaar is mits toegang, dataweergave en tenant-isolatie goed zijn afgebakend.

Tot slot richt de achtste deelvraag zich op KPI's, zodat de gerealiseerde feature niet alleen gebouwd wordt, maar ook op een meetbare manier geëvalueerd kan worden. Deze deelvragen vormen gezamenlijk het fundament voor zowel het ontwerp als de validatie van de feature.

3.8 Formuleren van doelstelling

Het doel van dit onderzoek is het ontwikkelen van een oplossing voor een template-gebaseerde standalone websitefunctionaliteit binnen Domits. Hierbij wordt gestreefd naar een ontwerp- en implementatie-aanpak die schaalbaar, veilig en kostenefficiënt is, en bovendien correct integreert met beschikbaarheidsdata en de actuele prijs uit het Property Management System.

Daarnaast wordt een reflectiekader opgesteld met Key Performance Indicators (KPIs) en testscenario's, waarmee herhaaldelijk vastgesteld kan worden in hoeverre de oplossing voldoet aan de gestelde eisen en hoe goed de feature huidige functioneert.

3.9 Validiteit en betrouwbaarheid

Naast de realisatie van de feature is het belangrijk om de werking ervan ook op een gestructureerde manier te kunnen beoordelen. Daarom maken we gebruik van Key Performance Indicators (KPI's) die direct aansluiten op de werking van de Direct Booking Website. De KPI's helpen om inzicht te krijgen in punten, zoals snelheid van publicatie, prestatie van de live website, operationele kosten, betrouwbaarheid van routing en de mate waarin prijzen correct blijven aansluiten op PMS data.

Voor extra validatie wordt in dit onderzoek gekozen om meetpunten direct te koppelen aan de eisen en risico's van de feature. Hierbij wordt niet alleen gekeken naar ideale scenario's, maar ook naar fout scenario's, zoals problemen in prijs, beschikbaarheid, publicatie of bereikbaarheid van domein. Betrouwbaarheid wordt vergroot door dezelfde scenario's en meetpunten toe te passen, zodat beoordeeld kan worden of de gekozen aanpak consistent functioneert.

Bij meerdere KPI's is gekozen voor percentielen, zoals p95 en p75, in plaats van alleen gemiddelden. De reden hiervoor is dat gemiddelden een gladgestreken beeld kunnen weergeven wanneer uitschieters geregistreerd worden [50]. Een p95-meting laat zien binnen welke tijd 95% van de gevallen valt en is daardoor geschikt voor processen, zoals publicatie en bouw van preview. p75 is gekozen voor website performance, omdat dit aansluit op de meetmethode van google core web vitals [51]. Hierdoor ontstaat een realistischer beeld van zowel betrouwbaarheid als gebruikservaring.

Criterium	KPI / meting	Betekenis
Schaalbaarheid	time_to_publish_p95	Meet hoe snel een website in 95% van de gevallen gepubliceerd kan worden.
Kosten	cost_per_active_site_per_month	Geeft een indicatie van de gemiddelde operationele kosten per actieve website per maand.
Performance	site_lcp_p75 (mobile, tablet & desktop)	Meet hoe snel de belangrijkste zichtbare content geladen wordt.
Betrouwbaarheid	fallback_subdomain_availability	Laat zien in hoeverre terugval domeinen beschikbaar en bereikbaar blijven.
Correctheid	quote_to_charge_mismatch_rate	Geeft aan hoe vaak er verschil ontstaat tussen verwachte prijsinformatie en uiteindelijke prijsafhandeling.
Toekomstige betrouwbaarheid (v2)	booking_api_error_rate	Meet hoe vaak fouten optreden tijdens boekingen via api, zodra boeken een mogelijkheid is
Gebruikerservaring (v2)	booking_funnel_completion_rate	Laat zien welk deel van gebruikers de boeking stroom afrondt
Domeinbeheer (v2)	custom_domain_setup_succes_rate	Meet hoe vaak het koppelen van eigen domein succesvol verloopt.
Praktische validatie	end-to-end testing	Test of de functionaliteit van bouwen, publiceren en uitlezen correct werkt in praktijk scenario's.

Tabel 1: KPI's en betekenis binnen validatie van de Direct Booking Website

In de huidige fase zijn niet alle KPI's al meetbaar. Met name KPI's rondom de volledige booking flow behoren tot een volgende implementatiefase, maar zijn nu al opgenomen, zodat duidelijk is hoe validatie plaatsvindt in een later stadium.

3.10 Conclusie

Uit de probleemanalyse blijkt dat verhuurders momenteel afhankelijk zijn van externe platformen voor boekingsaanvragen, wat leidt tot commissie en beperkte controle over hun eigen listing. Tegelijkertijd ontbreekt binnen Domits een simpele en schaalbare feature om Direct Booking Websites te publiceren die gekoppeld zijn aan verhuurders hun beschikbaarheids- en prijsdata uit het Property Management System.

Het probleem ligt niet alleen in het ontbreken van een aantrekkelijke functionaliteit, maar ook in de complexiteit van het combineren van eisen, zoals multi-tenant-architectuur, correcte data-integratie, beveiliging en kostenefficiënte hosting. Deze factoren maken het ontwikkelen van een standalone website een technisch vraagstuk.

Op basis van deze analyse is een centrale onderzoeksvraag geformuleerd, ondersteund door deelvragen die zich richten op onder andere functionaliteit, architectuur, hosting, integratie en beveiliging.

In het volgende hoofdstuk wordt beschreven hoe deze vragen worden onderzocht en welke manier van onderzoek gehanteerd wordt om tot een onderbouwd resultaat te komen.

4. Onderzoeksaanpak

In dit hoofdstuk wordt beschreven op welke manier de onderzoeksvragen worden beantwoord. De aanpak richt zich op het analyseren van verschillende oplossingsrichtingen, het onderbouwen van ontwerpkeuzes en het valideren van de gekozen aanpak binnen de context van deze nieuwe Domits-feature.

De onderzoeksstrategie bestaat uit een combinatie van bureau- en literatuuronderzoek, technische analyse en praktische validatie.

Tijdens het onderzoeksproces worden bestaande/soortgelijke features en architectuur patronen onderzocht, zoals diverse website builders, multi-tenant platformen, template creatie en waarborging, en hosting strategieën. Aan de hand van deze vergaarde informatie en kennis wordt een afgebakende v1 opgesteld die aansluit bij de technische en productmatige eisen van deze functionaliteit.

4.1 Onderzoeksmethode

Het onderzoek volgt een ontwerpgerichte en implementatie gerichte aanpak, in lijn lopend met de opgestelde hoofdvraag en deelvragen, waarvoor eerst literatuuronderzoek wordt geanalyseerd en verschillende mogelijkheden vergeleken, hieruit volgt de ontwerpfase en daarna de implementatiefase van een v1. Om dit concreet en onderbouwd aan te pakken, worden functionele en technische eisen van de directe website eerst vastgesteld en centraal opgesteld.

Vanuit dit punt worden meerdere mogelijkheden en oplossingsrichtingen onderzocht voor onderwerpen zoals template-architectuur, aanbod van professionele websites, hosting, domein beheer, PMS-integratie en security.

Deze volgorde is gekozen, omdat de feature meerdere technische en onderdelen van het product raakt. Zonder analyse bestaat het risico dat gebouwd wordt op aannames die later onverdedigbaar blijken. Door eerst de verschillende mogelijkheden te vergelijken en pas daarna te realiseren, ontstaat een beter onderbouwde basis voor de uiteindelijke implementatie.

Kortom doorlopen we de volgende stappen:

Methode	Uitleg
Vereisten analyse	In kaart brengen van functionele- en technische eisen, incl stakeholder behoeften en v1-scope afbakening

Oplossingsrichtingen	Afweging van architectuur/strategie: template-architecturen, hostingmodellen, domeinbeheer
Vergelijking m.b.t. basiscriteria	Beoordelen van alternatieven op criteria zoals kosten, schaalbaarheid, performance, onderhoudbaarheid, complexiteit en risico's
Ontwerp en realisatie	Uitwerking van keuzes in een concreet ontwerp en het ontwikkelen van een v1 MVP
Validatie	Testen/validatie van keuze d.m.v. KPI's, testscenario's en gebruikssituaties, waaronder site publicatie en correcte weergave van beschikbaarheid en prijs

Tabel 2: Onderzoeksstappen

Aan de hand van bevindingen, geanalyseerde KPI data, praktijkgericht onderzoek binnen Domits, in samenhang met de functionaliteitseisen, wordt deze nieuwe feature iteratief opgebouwd, zodat er gewerkt wordt naar een onderbouwd en daadwerkelijk bruikbaar eindproduct.

4.2 Vergelijkingskader

Om oplossingsrichtingen op een consistente manier te beoordelen, wordt gebruikgemaakt van een vergelijkingskader. De gekozen criteria zijn niet random opgesteld, maar staan parallel met de eisen van Domits, de kern van de feature en de belangrijkste risico's uit de probleemanalyse. Omdat het gaat om een nieuwe functionaliteit binnen een al bestaand platform, zijn vooral schaalbaarheid, kosten, onderhoudbaarheid, veiligheid en correcte PMS integratie bepalend voor het succes van de oplossing.

Daarnaast is de criteria gebaseerd op de centrale onderzoeksvraag en de daaruit voortkomende deelvragen. Zo sluiten schaalbaarheid en kosten aan op de vraag hoe de oplossing betaalbaar en uitbreidbaar kan blijven, terwijl correctheid en veiligheid gekoppeld is aan de betrouwbaarheid van data rondom beschikbaarheid en prijzen en aan tenant-isolatie. Complexiteit en haalbaarheid van de v1 scope zijn toegevoegd, omdat de oplossing niet alleen inhoudelijk goed moet zijn, maar ook binnen bepaalde tijd gerealiseerd moet kunnen worden.

Door deze eisen vooraf vast te stellen, ontstaat er een onderbouwde basis om verschillende implementatiemogelijkheden met elkaar te vergelijken en een verantwoorde ontwerpkeuze te kunnen maken.

4.3 Validatie en Meetplan

De gekozen richting/aanpak wordt gevalideerd door deze systematisch te toetsen aan de hand van de hierboven genoemde criteria uit het vergelijkingskader. Hierbij leggen we de nadruk op operationele haalbaarheid en meetbare evaluatie.

Daarbij valideren en meten we aan de hand van de volgende drie punten:

1. Scenario validatie

De werking van de gekozen oplossing wordt getest aan de hand van gebruiksscenario's.

- Voorvertoning van de gemaakte website
- Publicatie van de website
- Uitvoering van een beschikbaarheid- en informatieaanvraag

Daarnaast worden fout scenario's opgevangen en getest.

- Onbeschikbare datas
- Verlopen informatie
- Falende domeinen
- Niet beschikbare PMS services
- Foutive rendering

Deze scenario's maken de gekozen aanpak inzichtelijk en constateren of het product robuust functioneert onder zowel normale omstandigheden als bij valkuilen.

2. KPI metingen

Om de kwaliteit van de oplossing te beoordelen wordt gebruikgemaakt van een meetplan op basis van Key Performance Indicators (KPI's). Waarbij deze KPI's zich richten op onder andere prestatie, kosten, betrouwbaarheid en correctheid van data. Verder zijn deze informatie/statistieken direct gekoppeld aan de criteria uit het vergelijkingskader.

Criteria	KPI / Meting
Schaalbaarheid	time_to_publish_p95 & cost_per_active_site_per_month
Kosten	cost_per_active_site_per_month
Performance	site_lcp_mobile_p75
Betrouwbaarheid	booking_api_error_rate & fallback_subdomain_availability
Correctness	quote_to_charge_mismatch_rate & booking_api_error_rate
Gebruikservaring	time_to_publish_p95, booking_funnel_completion_rate & custom_domain_setup_success_rate

Door deze metingen wordt duidelijk in hoeverre de keuzes daadwerkelijk voldoen aan de vooraf gestelde eisen. Daarbij is dit een handig inzichtelijk overzicht voor Domits zelf om te blijven constateren of de feature blijft presteren zoals hij uiteindelijk opgeleverd wordt.

3. Risico's en vangnetten

Naast functionele en technische validatie worden de keuzes beoordeeld op risico's en terugval gedrag.

Hierbij wordt gebruikgemaakt van gedefinieerde risico's en foute werkwijzen. Voor een gemakkelijk overzicht van verschillende uitgangspunten, verwachtingen en oplossingen wordt hiervoor een risico matrix opgesteld.

Momentele punten van waarborging:

- Gedrag bij uitval van afhankelijkheden (PMS, prijs, beschikbaarheid)

- Terugval gedrag (zoals subdomain fallback)
- Bescherming tegen inconsistent getoonde data
- Isolatie tussen verhuurders

Deze manier van meting toont aan of de oplossing niet alleen werkt onder ideale omstandigheden, maar ook veilig en voorspelbaar blijft bij optredende fouten.

4.4 Inzet van AI tijdens de realisatie

Tijdens de realisatie van de Direct Booking Website feature is AI gebruikt als ondersteunend hulpmiddel bij de ontwikkeling en interne documentatie. Waar de voorgaande onderdelen van dit hoofdstuk inzicht geven over hoe het onderzoek is opgezet, richt deze paragraaf zich op de ondersteuning tijdens uitvoering van de feature. ChatGPT is hierbij ingezet voor aanscherping in verband met formuleringen en het ordenen van informatie, terwijl Codex tijdens de ontwikkeling gebruikt is als programmeerondersteuning bij implementatie en iteratie.

Daarnaast is gebruikgemaakt van Obsidian als tweede kennis laag of “second brain”. Binnen deze omgeving werden business context, technische keuzes, voortgang, werkwijzen en gemaakte beslissingen gestructureerd toegevoegd. Dit tweede brein maakte het mogelijk om relevante context te behouden, zonder dat deze telkens opnieuw volledig in een nieuwe sessie hoefde te worden benoemd. Hierdoor was het mogelijk om gericht met context te werken en werd onnodig gebruik van tokens beperkt.

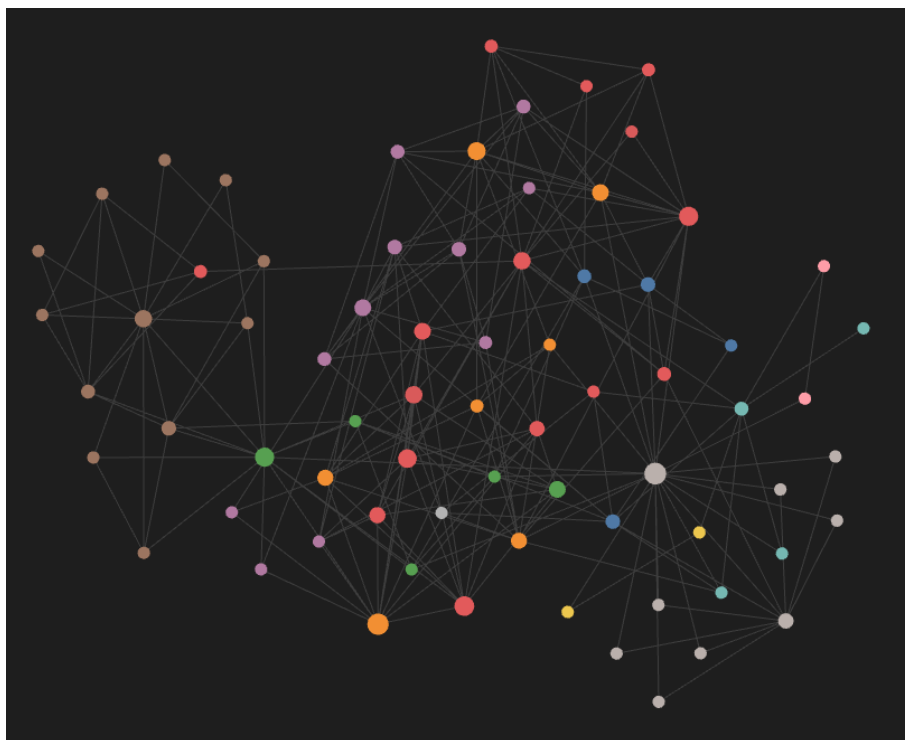
Figuur 5 hieronder toont een voorbeeld van deze gestructureerde context binnen Obsidian, waarin zichtbaar is hoe notities over business context, technische afspraken, voortgang en werkwijzen met elkaar in verbinding staan.



Figuur 5: Visualisatie van context en verbintenis van notities binnen Obsidian.

Tijdens dit proces bleek dat AI pas echt bruikbaar en betrouwbaarder wordt wanneer eerst voldoende context wordt opgebouwd en gestructureerd gecontroleerd wordt, of wanneer model en mens over dezelfde criteria beschikken. Om die reden werd vooraf afgestemd op doel, werkwijze en systeemcontext, voordat daadwerkelijke samenstelling werd gestart. Nieuwe inzichten, beslissingen en context werden daarna opnieuw toegevoegd in Obsidian, zodat een iteratief groeiende en verbeterende kennisstructuur ontstond. AI output is hierbij niet blindelings overgenomen, maar steeds gecontroleerd aan de hand van code, gedrag en beoordeling.

Figuur 6 hieronder toont een voorbeeld van steeds meer toegevoegde context binnen Obsidian, waarin notitie samenhang zichtbaar is.



Figuur 6: Weergeeft de verbetering van meer verzamelde notities en context binnen het Obsidian tweede brein.

5. Analyse

In dit hoofdstuk worden per deelonderwerp meerdere oplossingsrichtingen onderzocht en afgewogen. De analyse richt zich op de vraag welke keuzes het best aansluiten bij de vereisten van Domits en behoren tot een v1 scope. Per onderdeel worden onderzochte alternatieven vergeleken en wordt er afgesloten met een bewuste keuze die in hoofdstuk 6 verder wordt gerealiseerd.

5.1 Minimale v1 functionaliteit

In deze deelvraag staat centraal welke minimale functionaliteit de Direct Booking Website moet bevatten om als v1 waardevol en haalbaar te zijn binnen Domits.

De bedoeling van deze toekomstige feature is het verkrijgen van boekingen op een losstaande website, die simpel op te zetten is en verzorgd is met data over de accommodatie. Binnen deze feature zijn er veel aspecten die getoond moeten worden om voldoening te halen uit deze feature. Hierbij is het goed om een v1 af te bakenen en om overige wensen op de roadmap te zetten voor een toekomstige implementatiefase. Hierdoor leiden we niet direct onder een te brede of risicovolle scope. Daarbij moet niet alleen gekeken worden naar wat technisch mogelijk is, maar vooral naar wat in een v1 voldoende waarde levert voor hosts en haalbaar blijft binnen tijdgebondenheid, onderhoud en implementatie risico's.

Volgens Nielsen Norman Group moet een MVP (Minimal Viable Product) van begin tot eind bruikbaar zijn, maar ook beperkt blijven tot wat gebruikers nodig hebben [1]. Dat uitgangspunt is hier relevant, omdat de Direct Booking Website geen volledig los boekingsplatform hoeft te worden in de eerste versie, zolang kernwaarden voor gast en host aanwezig zijn.

Afbekening van een v1

Om deze afbakening te maken is het nodig om de diverse functionaliteiten op te delen in drie lagen: presentatie, beheer en transactie. De presentatielaag duidt op alles wat nodig is om een listing behapbaar en aantrekkelijk te tonen, zoals titel, beschrijving, foto's, voorzieningen, huisregels en contactinformatie. De beheerlaag duidt op alles wat de host nodig heeft om de website samen te stellen en te beheren, zoals keuze in layout, opslaan van wijzigingen, preview, publicatie en beperking tot content aanpassingen. De transactielaag betreft uitlezing van prijs, bezettingsgraad, checkout en het aanmaken van boekingen.

Voor Domits is deze opdeling logisch binnen onderzoek naar hospitality context en laat zien dat de kwaliteit van website, vertrouwen en waarde direct samenhangt met online boeking intentie [2], [3], [4]. Dit betekent dat een v1 in ieder geval voldoende informatie en kwaliteit moet bieden voor het opbouwen van vertrouwen, ook als latere transactie stappen nog niet volledig zijn uitgewerkt.

Prestatielaag

De prestatie laag duidt op overtuigingskracht bij potentiële gasten. Zonder duidelijke informatie over de accommodatie en vertrouwen heeft de website nauwelijks kans op boekingen. Onderzoek naar hotelwebsites laat zien dat de kwaliteit van de website belangrijk is voor vertrouwen en intentie tot boekingen [2], terwijl ook waarde en website trust samenhangt met intentie tot online boeken [3]. Verder blijft dat websitekwaliteit, sociale aanwezigheid en e-trust invloed hebben op reizigers om een accommodatie te reserveren [4].

Dit betekent voor Domits dat een eerste versie minimaal de volgende elementen moet bevatten:

- naam / titel van de accommodatie
- beschrijving
- afbeeldingen
- voorzieningen
- locatie
- contactinformatie
- verblijfvoorwaarde

Deze onderdelen vormen gezamenlijk de minimale informatiebasis waarmee een gast kan begrijpen wat aangeboden wordt en in hoeverre de accommodatie aansluit naar wens. Zonder deze context ontstaat onzekerheid, wat het vertrouwen schaadt en het effect van deze feature afzwakt.

Beheerlaag

Naast prestatie is de beheerlaag ook relevant. Deze oplossing is juist bedoeld als een laagdrempelige website bouwer voor hosts. Dat betekent dat een host minimaal een bestaande listing moet kunnen selecteren, een layout moet kunnen kiezen, wijzigingen moet kunnen aanbrengen, gewijzigde onderdelen moet kunnen opslaan, voorvertoning van de website moet kunnen bekijken, de website moet kunnen publiceren en verwijderen. Dit sluit aan op dat een eerste MVP beperkt mag zijn, zolang de kernstroom van begin tot eind werkt [1].

Transactielaag

Bij de transactielaag is de beschikbaarheid belangrijk. Dit komt doordat de gast anders niet kan inschatten of een verblijf relevant is voor de gewenste periode. Daarbij brengen prijsberekening, check out, boeking aanmaak en bevestiging extra fout scenario's met zich mee. Een v1 hoeft hierdoor niet te bestaan uit een volledige end-to-end boeking funnel om waardevol te zijn. Voor een MVP is een informatief

kalender overzicht en publicatie flow voldoende, terwijl check out en boeking gerelateerde implementaties doorgeschoven wordt naar een v2.

Keuze

Belangrijkste overwegingen voor de v1 keuze:

- De website moet voor hosts snel en laagdrempelig op te zetten zijn.
- De eerste versie moet bruikbaar zijn, zonder gelijk een volledige end-to-end boeking functionaliteit te vereisen.
- De informatie moet voor de gast voldoende vertrouwen geven in de accommodatie.
- De implementatie moet beheersbaar blijven binnen tijd, onderhoud en risico's op technisch gebied.
- Functionaliteiten zoals checkout, boeking registratie en eigen domeinen voegen waarde toe, maar vergroten de complexiteit te sterk voor een eerste versie.

Daarom bestaat de minimale waardevolle v1 uit:

- Selectie van bestaande listing
- Keuze uit website layyout
- Automatische data import van het Property Management System
- Presentatie van titel, beschrijving, afbeeldingen, voorzieningen, locatie en huisregels
- Contactinformatie
- Overzicht van bezettingsgraad
- Een aparte omgeving waarin de website kan worden aangepast
- Voorvertoning van website
- Publicatie stroom met statusbeheer en link

Andere functionaliteiten zoals check out, boeking registratie, custom domein en volledige vrijheid in layout vallen daarmee buiten de v1 scope. De elementen binnen de v1 leveren voldoende waarde voor hosts, terwijl de oplossing haalbaar, sneller te valideren en minder risicovol blijft. Daarnaast streven we voor een professionele uitstraling. Echter is data connectie voor nu de eerste prioriteit en kan er later gekeken worden naar een waardige professionele inrichting per aangeboden layout.

5.2 Template architectuur

Deze paragraaf onderzoekt welke template architectuur het meest geschikt is om herbruikbare websites met beperkte maar bruikbare customization mogelijk te maken, zonder dat elke template een apart project wordt.

Binnen Domits worden verhuurders voorzien van een nieuwe functionaliteit waarmee zij gemakkelijk, zonder technisch inzicht, een eigen standalone website kunnen generen. Om deze feature goed op te bouwen, moet eerst onderzocht worden welke vormen van template architectuur bestaan en welke structuur het meest geschikt is gegeven Domits. Daarbij is het erg belangrijk dat de gekozen aanpak niet alleen werkt voor een eerste versie, maar ook naar wens blijft functioneren in de toekomst wanneer nieuwe templates, uitbreidingen en varianten worden toegevoegd.

Het doel van dit hoofdstuk is daarom niet om te bepalen welke websites er visueel het meest aantrekkelijk zijn, maar om doordacht te analyseren welke architectuur technisch houdbaar, uitbreidbaar en, om niet te vergeten, haalbaar is binnen een v1 binnen de context van Domits.

Op basis van bestaande platformen, met soortgelijke website bouw features en hoe deze functioneren, maken we een 3-delige analytische indeling. Deze indeling bestaat uit vastgestelde templates, block-based templates & builder aanpak om te achterhalen wat deze mogelijkheden van constructie inhoudelijk betekenen.

1) **Vastgestelde/fixed templates**

Bestaande platformen zoals Ghost, tonen dat een template grotendeels vastligt, terwijl eigen configuratie aanwezig is met betrekking tot kleuren/thema, afbeeldingen, tekst en logo [5].

2) **Block-based templates**

Platformen waaronder Shopify tonen een tussenlaag waarin een pagina vaste, maar ook verplaats- en aanpasbare componenten heeft of blocks [6], [7].

Deze structuur biedt de gebruiker niet volledige vrijheid, maar ook niet volledige vastheid binnen de configuratie van een website. Deze manier van template realisatie zorgt dus voor een gestructureerde flexibiliteit tussen block/elementen van de website.

3) **Vrijuit bouwen**

Diverse platformen waaronder Wordpress Site Editor, Webflow en Wix bieden gebruikers de mogelijkheid tot aanzienlijk meer vrijheid in vorm van pagina opbouw en element & component structuur. Zo krijgt de gebruiker de mogelijkheid zelf deze aspecten te wijzigen en te plaatsen op welk soort plek dan ook. [5]-[11].

Fixed templates

Bij een vaste template aanpak liggen aanpassingen van de website template structuur grotendeels vast. Bij deze vorm kiest de host een template waar inhoud en stijl via configuratie vooraf ingevuld en vastgesteld zijn. Denk hierbij aan titel, beschrijving, foto's, contactinformatie, logo, kleuren en het in- of uitschakelen van vooraf gedefinieerde secties. De layout zelf blijft bij deze vorm in grove lijnen onveranderd.

Deze manier van implementatie komt terug in andere bedrijven, zoals Ghost. Ghost biedt ondersteuning voor keuze uit eigen thema, waarbij de ontwikkelaar vooraf bepaald welke instellingen beschikbaar zijn. Deze instellingen zijn beperkt en bestaan uit vastgestelde types, zoals select, boolean, color, image en tekst [5]. Het bedrijf beschrijft deze mogelijkheid van eigen voorkeur voor website-eigenaren als een manier om stilistische keuzes te laten maken, zonder dat zij theme files hoeven aan te passen [5]. Tegelijkertijd heeft Ghost grenzen ingebouwd, waaronder de beperktheid van maximale aanpassingen van 20 voor eigen voorkeur [5], Ghost dwingt hiermee ook voorzichtig versiebeheer af: wanneer een setting key wordt aangepast, gaan eerder opgeslagen waarden voor de bestaande gebruikers verloren [5]. Dit toont aan dat een vastgestelde template-architectuur vooral sterk is op technisch gebied, doordat de vrijheid van configuratie bewust begrensd blijft.

Deze implementatie mogelijkheid biedt duidelijke voordelen voor Domits. Zo is een vastgesteld template snel te constructueren, heeft een laag technisch risico en is goed te valideren door middel van tests. Daarnaast blijft achterwaartse compatibiliteit beheersbaar, omdat een site kan worden vastgezet op een *template_key* en *template_version*. Nieuwe templates kunnen dan worden toegevoegd zonder dat bestaande websites hoeven te breken of aanpassingen nodig hebben. Het nadeel is de beperkte gebruikersvrijheid. Verhuurders kunnen templates wel configureren maar fundamentele elementen niet herschikken.

Block-based templates

Een blok-gebaseerde aanpak biedt meer flexibiliteit. In plaats van een vastgestelde, niet aanpasbare layout, stelt de block-based aanpak de gebruikers in staat de website elementen naar eigen voorkeur in te delen. Dit komt doordat de structuur opgebouwd is vanuit verschillende vooraf gedefinieerde elementen, secties en/of blokken die in bepaalde maten aanpasbaar zijn. Opties die hierbij komen kijken zijn bijvoorbeeld: secties die aan/uit gezet kunnen worden, bepaling van volgorde, of dat een template meerdere varianten van dezelfde sectie ondersteunt.

Shopify is hierbij een goed voorbeeld. Het platform maakt gebruik van JSON templates waarin een lijst van secties en bijbehorende instellingen worden opgeslagen. Gebruikers kunnen bepaalde secties vanuit de editor toevoegen,

verwijderen en reorganiseren [7]. Shopify geeft aan dat blocks kleinere, herbruikbare onderdelen van secties zijn, ieder met eigen instellingen, en dat ze binnen een sectie toegevoegd, verwijderd en geordend kunnen worden [8]. Deze mogelijkheid biedt aanzienlijk meer flexibiliteit ten opzichte van een vaste layout, maar introduceert ook extra beperkingen en validaties. Een maximum wordt bijvoorbeeld gehanteerd bij het aantal secties en blocks per gekozen template [7].

Dat toont aan dat meer vrijheid ook meer beheer-complexiteit en regels vereist.

Een block-gebaseerde aanpak is voor Domits interessant als tussenweg. Hierdoor stellen we een host bijvoorbeeld in staat secties zoals, hero, galerij, voorzieningen, contactinformatie of accommodatiebeleid te kunnen tonen of verbergen, zonder dat een volledig vrije website builder nodig is. Door deze vrijheid maakt dit de rendering, validatie en testen tegelijkertijd complexer. Hoe meer vrijheid ontstaat in volgorde, variatie of combinatie van secties, hoe groter de kans op inconsistente layouts en fouten. Binnen een v1 scope brengt dat meer risico met zich mee dan vastgestelde templates.

Vrijuit bouwen

De derde mogelijkheid is volledig vrij te kunnen bouwen. Hierbij krijgt de gebruiker maximale vrijheid om de pagina zelf in elkaar te zetten, onderdelen te verplaatsen en wijzigingen aan te brengen binnen de structuur. Hoewel deze mogelijkheid van aanpak de meeste vrijheid biedt, is dit ook verreweg het zwaarst / intensiefs om te implementeren en beheren binnen Domits.

WordPress laat goed zien hoeveel vrijheid gekoppeld is aan block themes. Het platform beschrijft block themes als themes die block gebruiken voor alle onderdelen van een site, inclusief navigatie, header, content en footer [10]. In de Site Editor kan zowel pagina inhoud als het template worden aangepast [10]. Ook Webflow vermeldt dat een builder aanpak snel samenhangt met props, slots en varianten per component, zodat verschillende elementen/aspecten afzonderlijk geconfigureerd kunnen worden [11]. Wix toont bovendien aan dat kunstmatige intelligentie (AI) meestal niet de architectuur vervangt, maar extra secties genereert binnen een bestaande structuur. Wix benoemt dat AI-gegenereerde secties volledig aanpasbaar blijven qua ontwerp en inhoud [12]. Ook Shopify plaatst AI-gegenereerde blocks nog steeds binnen de bestaande block architectuur [8].

Voor Domits is deze vorm van vrijuit bouwen heel interessant en een belangrijk besprekingspunt. Het hebben van een AI-builder klinkt aantrekkelijk, maar maakt deze feature implementatie aanzienlijk lastiger en implementatie snelheid trager. Bovendien vereist een AI-builder een geavanceerde editor omgeving, meer support, uitgebreidere validatie, complexere migratie, en kans op meer risico's, scheuren en/of breuken tussen oude en nieuwe templates. Voor een eerste versie van een

standalone website feature is dit niet passend en aantrekkelijk. Wel is dit een mogelijkheid waar in de toekomst naartoe gewerkt kan worden.

In de meest ideale situatie zouden we de mogelijkheid willen bieden tot een geheel eigen indeling, aanpassing, voorkeur en configuratie voor verhuurders binnen deze feature aan de hand van een volledig vrije website builder. Helaas moet er realistisch gekeken worden naar beschikbare tijd, ruimte, kosten en voldoening voor een eerste versie. Om deze mogelijkheid niet volledig van gebruikers af te nemen en uit te sluiten is er na naderhand inspectie gekozen om een mix te implementeren tussen vastgestelde en block-based templates. Wij vermoeden dat de mogelijkheid tot eigen voorkeur op deze manier breed genoeg is voor een v1, en waarbij verhuurders alsnog voldoende kunnen profiteren van deze template keuze.

Afweging

Wanneer de drie bovenstaande opties met elkaar worden vergeleken op gebied van onderhoudbaarheid, complexiteit, functionaliteit en haalbaarheid binnen een v1, ontstaat een duidelijk beeld.

Om de verschillen tussen de onderzochte opties overzichtelijk te maken, zijn de belangrijkste eigenschappen samengevat in Tabel 3.

Aspect	Vastgestelde templates	Block-based templates	Vrije builder
Vrijheid voor host	Laag	Gemiddeld	Hoog
Technische complexiteit	Laag	Gemiddeld	Hoog
Beheersbaarheid	Hoog	Gemiddeld	Laag
Testbaarheid	Hoog	Gemiddeld	Laag
Achterwaartse compatibiliteit	Goed beheersbaar	Complexer	Moeilijk
Implementatie snelheid	hoog	Gemiddeld	Laag
geschiktheid voor v1	Hoog	Middelmatig	laag
Toekomstige uitbreidbaarheid	Goed, mits gedeeld contract	Goed	Hoog, maar risicovoller

Tabel 3: Vergelijking van template architecturen

Een vastgestelde template aanpak scoort het beste op beheersbaarheid. De technische structuur is simpel, de werking valt goed te meten en uitbreiding met nieuwe templates is mogelijk zolang deze onder een gedeeld contract blijven bestaan en functioneren. Dit maakt toekomstige template toevoeging makkelijk zonder bestaande functionaliteiten te slopen.

Een block-gebaseerde aanpak biedt meer vrijheid, maar bevat een complexere implementatie. Hierbij moet er meer gevalideerd worden, meer combinaties moeten worden ondersteund en de kans op inconsistente data stijgt. Als groeirichting is dit zeker bruikbaar, maar het verhoogt de implementatie tijd/kracht/kennis en onderhoudslast. Dit samen lijkt al gauw te veel voor een eerste versie.

Een vrije builder aanpak biedt de gebruiker maximale vrijheid tijdens de ontwikkeling van zijn/haar website, maar voor Domits haar v1 is dit te zwaar en de lat te hoog gelegd. De (ontwikkelkosten) validatie, support overlast en complexiteit van achterwaartse compatibiliteit wegen in deze fase nog niet op tegen de extra vrijheid die deze aanpak oplevert.

Vergelijking met bestaande website builders

Om de keuze voor een geschikte template architectuur goed te onderbouwen, is het van belang deze af te wegen tegen bestaande website builders binnen dezelfde markt. Platformen, zoals Lodgify, Guesty, Hostaway en Smoobu bieden soortgelijke functionaliteiten waardoor verhuurders zonder hoog technisch inzicht een eigen boeking gerichte website kunnen samenstellen [13]-[17]. Alhoewel tussen deze platformen verschil zit, volgt de productaanpak in grove lijnen dezelfde ontwerprichting. Namelijk dat een host geen volledig vrije omgeving krijgt, maar een beheersbare configuratie laag bovenop een vastgestelde site structuur / layout [13]-[17].

Wat hierbij opvalt is dat de waarde niet gehaald wordt uit vrijheid binnen ontwerp, maar uit snelheid van publicatie, professionele presentatie, koppeling met bestaande accommodatie gegevens en het direct kunnen reserveren [13], [15]-[17]. De gebruiker kiest een template of website layout, waarna de toepassing van visuele en inhoudelijke onderdelen gelimiteerd aangepast kunnen worden. Denk hierbij aan afbeeldingen, tekst, kleuren, contactinformatie en beschikbaarheid. Dit laat zien dat de nadruk dus niet ligt bij een volledige vrije websitebouwer, maar op een standaard systeem waardoor hosts spoedig een daadwerkelijk bruikbare website kunnen publiceren zonder dat zij zelf de layout en/of technische structuur hoeven te beheren [13], [15]-[17]. Ook toont dit aan dat een template gebaseerde structuur niet zwak, maar juist een veelvoorkomend model is. Binnen de context van directe reserveringen is onbeperkte vrijheid niet de belangrijkste factor. Veel belangrijker is consistentie, betrouwbaarheid, beheersbaarheid en de mogelijkheid tot snelle

publicatie omtrent een professionele website [13]-[17].

Keuze

Voor Domits v1 is een vastgestelde template aanpak met beperkte configuratie via sections en design tokens het meest geschikt. Hierdoor blijft de technische complexiteit beheersbaar, kan de kwaliteit van de output beter bewaakt worden en wordt het mogelijk om meerdere templates binnen één codebase aan te bieden. Zo biedt Domits verhuurders voldoende aanpasbaarheid door middel van branding (logo/favicon), inhoud, contactinformatie, beperkte styling, zonder dat het bedrijf direct een page builder functionaliteit hoeft te maken.

Deze bewuste keuze sluit aan op het doel om de feature zonder technische ervaring tijdig een bruikbare, maar toch professionele, website te realiseren en publiceren. Daarvoor is geen maximale vrijheid nodig, maar liever een beheerbare en uitbreidbare structuur. Tijd en energie kunnen beter gestoken worden in beheersbaarheid indien nieuwe template varianten toegevoegd worden. Nieuwe templates kunnen later worden toegevoegd als varianten binnen hetzelfde template-systeem, terwijl bestaande sites op hun eigen template-versie blijven werken. Deze aanpak is hierdoor zowel haalbaar voor een v1 als uitbreidbaar voor toekomstige iteraties.

5.3 Hosting strategieën

In deze deelvraag wordt onderzocht welke hosting strategie het meest geschikt is voor Domits, gegeven de eisen rondom kosten, performance, schaalbaarheid en beheersbaarheid.

Een belangrijk punt binnen deze nieuwe feature is de bepaling van een hosting strategie, omdat dit namelijk direct invloed heeft op kosten, prestatie, schaalbaarheid en beheersbaarheid. Voor websites hosten zijn er in de praktijk drie uitgangspunten van elkaar te onderscheiden: statische hosting, volledige server-side rendering en hybride aanpak.

Statische hosting

De naam van deze hosting strategie verklapt al de manier van hosting, namelijk dat deze manier statisch is. Het voordeel van statisch hosten is dat pagina's snel geladen worden. Dat komt doordat bij deze manier van hosting pagina's als statische bestanden gepubliceerd worden en via een Content Delivery Network (CDN) verspreid worden. CDN kun je zien als een netwerk van servers die websites doen versnellen door essentiële bestanden tijdelijk op te slaan (caching) en beschikbaar te maken voor gebruikers over de hele wereld. AWS vermeldt dat statische websites op Amazon S3 gehost kunnen worden en adviseert Amplify Hosting voor gebruik. Vervolgens wordt de content/inhoud wereldwijd gedistribueerd via CloudFront [29]. Daarnaast benoemt AWS statische hosting niet alleen als een goedkope manier, maar blijkt deze aanpak ook betrouwbaar en tegelijkertijd schaalbaar, waarbij het operationeel gebied amper beheerd hoeft te worden [30].

Wanneer er gekeken wordt naar Domits zien we dat deze aanpak duidelijk voordelen heeft. Statische inhoud is snel, goedkoop en makkelijk schaalbaar. Voor de toekomstige feature klinkt deze aanpak zeer aantrekkelijk, vanwege de website's content die niet per aanvraag steeds opnieuw opgebouwd hoeft te worden, wat lang kan duren met accommodatie informatie, branding en meerdere foto's. Zodra boeking integratie, live quotes en prijs data geïmplementeerd worden, voorzien we bij deze hosting manier nadelen. Het is namelijk verstandig om deze punten dynamisch af te lezen van Domits i.p.v. een statische website. Dit kan anders leiden tot ongecontroleerde boekingen, valse data en kan meer werk opleveren. Complexiteit zit hem dan in aanvullende API's.

Server-side rendering

Naast statisch hosten bestaat server-side rendering (SSR), waarbij de pagina (of een belangrijk deel) per pagina aanvraag gegenereerd wordt in plaats van dat dit vooraf statisch gegenereerd is, volgens Next.js [26]. Indien data actueel moet zijn en niet vooraf betrouwbaar vastgesteld kan worden, is deze server-side manier van hosting zeer geschikt.

Een volledige SSR aanpak biedt voordelen op gebied van dynamische data en gecontroleerde data, bovendien zijn fouten gemakkelijk te detecteren. Wel gaat dit hand in hand met afhankelijkheid van runtime en veroorzaakt hoogstwaarschijnlijk meer latency, wat hogere kosten benadrukt ten opzichte van statische hosting. Elke website refresh is ten slotte een data aanvraag/verwerking.

Hybride hosting

Als laatste hebben we hybride hosting. Dit is een gecombineerde aanpak van statische content met, waar nodig, dynamische data uitlezing door middel van API (Application Programming Interface) integratie. Volgens Next.js kunnen hybride applicaties bestaan uit zowel statisch gegenereerde content als server-side gegenereerde pagina's [26]. Daarbij beschrijft Cloudflare een vergelijkbaar model waarin statische pagina's gecombineerd kunnen worden met server-side informatie via dynamische functies, zonder een toegewijde server te beheren [27], [28].

Een gecombineerde hosting aanpak voor Domits sluit het best aan op de toekomstige feature functionaliteit. Bepaalde onderdelen van de website hoeven niet op elke refresh live uitgelezen te worden vanuit het PMS, zoals beschrijvende context, en kunnen vooraf vastgesteld worden. Bovendien bespaart dit ook op uitgaven omtrent de Direct Booking Website. De beschikbaarheid, actuele prijzen en boekingen moeten daarentegen wel uitgelezen worden. Bij een hybride aanpak kan er bewust worden gekozen welke onderdelen vastgesteld en dynamisch worden getoond. Hoewel deze manier van data synchronisering meer kosten met zich meebrengt dan statische hosting, voorziet dit de gebruiker van de meest actuele data en beschermt het tegelijkertijd de verhuurder tegen pogingen tot fraude.

Keuze

Wanneer de drie hosting strategieën met elkaar vergeleken worden op basis van kosten, schaalbaarheid en beheersbaarheid, ontstaat er een duidelijk beeld. Om de verschillen tussen de drie strategieën te verduidelijken, is in Figuur 7 een visuele representatie opgenomen.



Figuur 7: Vergelijking tussen statische hosting, server-side rendering en hybride hosting

Statische hosting scoort hoog op kosten, snelheid en eenvoud, maar is minder betrouwbaar op gebieden waar het tonen van live data noodzakelijk is. Volledige server-side rendering scoort goed op het tonen van actuele data, maar is zwakker als het aankomt op kosten en performance. Dit komt door het verwerken van elk apart verzoek. Gegeven Domits is een hybride hosting strategie het meest geschikt.

De keuze bestaat uit een samenvoeging van statisch en server-side hosten en stelt websites in staat om accommodatie beschrijvende elementen vast te zetten, terwijl schommelende data, waaronder beschikbaarheid, prijs en boekingen dynamisch kunnen worden uitgelezen vanuit Domits haar PMS. Op deze manier wordt niet ingeleverd op correcte data en wordt er niet overbodig betaald voor aanvullende accommodatie informatie, zoals beschrijvingen en foto's.

5.4 Multi-tenant architectuur

In deze deelvraag staat centraal welke multi-tenant architectuur het meest geschikt is om schaalbaarheid, kostenefficiëntie en tenant-isolatie voor de Direct Booking Website binnen Domits te waarborgen.

Om data tussen gebruikers gescheiden te houden moet bepaald worden hoe de losstaande website functionaliteit multi-tenant ingericht wordt. Hierbij is het van belang hoe de infrastructuur gedeeld wordt tussen gebruikers en hoe data isolatie gewaarborgd wordt. De bedoeling hiervan is om de ene gebruiker geen toegang te verstrekken tot een ander zijn inzichten. Er bestaan drie modellen die het meest voor de hand liggend zijn: shared runtime, geïsoleerde tenant-omgevingen en een gemengde vorm.

Shared runtime

Met een shared runtime bedoelen we een gedeelde infrastructuur tussen meerdere gebruikers. Deze vorm maakt gebruik van een pooled model, waarbij gebruikers middelen met elkaar delen om eenvoudig beheer, snelle innovatie en om kostenefficiënt te zijn, volgens AWS [29]. Ook benoemt AWS nadelen rondom deze aanpak, beschreven als noisy-neighbor-risico; hoe meer data er gedeeld wordt, des te groter de kans dat de data van een gebruiker effect heeft op dat van de ander, en/of grotere impact bij optredende fouten [29].

Gegeven Domits is dit model aantrekkelijk, omdat het goed aansluit op schaalbaarheid en bovendien kostenefficiënt is. Een centrale distributie voor site logica, rendering en uitleesopdragingen door middel van een shared runtime maken beheer eenvoudig. Echter betekent deze vorm van infrastructuur dat user isolatie niet vanzelf ontstaat, maar afgedwongen moet worden op technisch gebied.

Geïsoleerde omgevingen

Naast een infrastructuur die gedeeld wordt bestaat er ook een model dat gebruikers compleet van elkaar isoleert. Op deze manier krijgen gebruikers volledig hun eigen toegewezen infrastructuur en stack. Deze full stack isolatie vorm wordt beschreven door AWS als een model dat pas relevant is wanneer naleving, profielen, of isolatie eisen dit noodzakelijk maken [30]. Deze vorm biedt significante isolatie ten opzichte van shared runtime, maar gaat gepaard met kosten die oplopen, beheer complexiteit en functionerende lasten.

Gegeven Domits biedt dit model maximale isolatie, maar haakt averechts in op het doel om snel en kostenefficiënt meerdere zelfstandige websites te kunnen hosten. Daardoor is deze manier van infrastructuur aanpak minder interessant voor het

platform.

Gemengde of tier-based isolatie

Een bestaande optie, naast sharing en een geïsoleerde omgeving, is een manier die tussen deze twee hangt. Namelijk een tier gebaseerde isolatie, waar sommige gebruikers in een gedeelde omgeving draaien, terwijl andere verschillende isolatieniveaus per gebruikersgroep of product tier worden aangeboden. AWS beschrijft dit als tier-based isolation [31]. Door deze methode ontstaat flexibiliteit, maar tegelijkertijd meer belasting op beheer niveau, vanwege ondersteuning van meerdere isolatie modellen.

Gegeven Domits is deze methode relevant als toekomstige groeirichting. Denk bijvoorbeeld aan wanneer grotere klanten of specifieke compliance-eisen optreden. Voor de eerste versie voegt dit potentieel interessant gemengd model momenteel helaas onnodige complexiteit toe.

Keuze

Met oog voor de vastgestelde eisen, is een runtime met strikte gebruiker isolatie het meest geschikt voor Domits. De gekozen aanpak past momenteel het best bij de voor nu gewenste schaalbaarheid en kostenefficiëntie van de feature.

Het is noodzakelijk bij deze aanpak om gebruikers hun data consistent te synchroniseren met de server en deze te valideren. Zo beschrijft AWS gebruiker isolatie als het scheiden van toegang tussen gebruikers, zodat data niet toegankelijk is tussen verschillende accounts [32].

Omdat we niet blindelings uit willen gaan van de cliënt, betekent dit dat gebruiker resolutie plaatsvindt via host of domein. Zo worden property- of website-identiteit vertrouwd vanuit de server-side APIs, die ook tenant-scoped zijn ingericht. Hierdoor ontstaat een multi-gebruiker model dat niet alleen goed te onderhouden valt, maar ook betaalbaar blijft.

5.5 Routing en domeinbeheer

Deze paragraaf onderzoekt hoe multi-tenant routing en domeinbeheer ingericht kunnen worden, zodat subdomains en custom domains betrouwbaar en schaalbaar functioneren.

Naast template keuze, hosting en data integratie, moet ook bepaald worden hoe Direct Booking Websites beschikbaar zijn voor externen. Bij deze kwestie gaat het niet alleen om koppelen van domeinnaam, maar ook om routing logica, zoals hoe we een aanvraag naar de juiste website leiden, hoe gegenereerde subdomains en later custom domains tegelijk bestaan, en hoe mislukte scenario's worden opgevangen.

Belangrijk voor Domits is dat websites per verhuurder vertrouwd bereikbaar moeten zijn, zonder een handmatige infrastructuur per site te hoeven inrichten. Tegelijkertijd moet het model schaalbaar blijven wanneer er later meer websites en eigen domeinen bijkomen.

Generated Subdomains

Voor een eerste versie is de eenvoudigste aanpak het gebruik van een gegenereerde subdomain, in de vorm van accommodatienaam.direct.domits.com. Zo beschrijft AWS Route 53 dat een hosted zone gebruikt wordt om websiteverkeer te routeren voor een domein en subdomein [33]. Ook wordt door AWS beschreven dat subdomeinen apart aangemaakt en beheerd kunnen worden [34].

Dit is aantrekkelijk gegeven Domits, omdat deze aanpak snel op te zetten is, weinig acties van de verhuurder vereist en makkelijk te automatiseren valt binnen een one-click publicatie stroming. Volledige controle over DNS, certificaten en bereikbaarheid blijft binnen Domits gezien deze v1. Gezien deze informatie maakt dit subdomeinen het meest logisch qua standaard.

Het nadeel is dat een gegenereerd subdomain minder merkgericht kan aanvoelen dan een eigen domein van de verhuurder. Voor een v1 weegt dit nadeel echter niet op tegen de gemakkelijk- en betrouwbaarheid van deze implementatie aanpak.

Custom Domains

Een tweede mogelijkheid is ondersteuning voor eigen domein namen, waarbij de verhuurder een eigen domein koppelt aan de standalone website. CloudFront beschrijft zulke domeinen als alternate domain names (CNAME's), waarvoor een geldig Transport Layer Security (TLS) certificaat vereist is en waarbij een domein niet tegelijkertijd gekoppeld mag zijn aan meerdere distributies gekoppeld [35]. Ook benoemt AWS dat wildcard domeinen ondersteund worden, mits ze door dit certificaat worden gedekt [36].

Hoewel custom domains commercieel aantrekkelijk zijn, brengen zij meer complexiteit met zich mee. De gebruiker moet DNS records correct instellen, een domein moet gevalideerd worden, certificaten moeten aanwezig zijn en de configuratie moet foutbestendig zijn. Hierdoor is custom domein support niet de meest logische start voor een v1, maar wel een duidelijke implementatie richting voor de toekomst.

Fallback en routing

Naast een keuze te moeten maken tussen gegenereerde subdomeinen en een eigen custom domein, moet het routing model duidelijk zijn. Zo moet een aanvraag binnen een multi-tenant platform eerst gecontroleerd worden via hostnaam en gebruiker- en website context resolveren, voordat de inhoud gerenderd wordt. Dit betekent dat een accommodatie-id niet vanuit de potentiële gast leidend is, maar vanuit de host of het domeinnaam waar de bezoeker data voor opvraagt.

Bij deze aanpak is een belangrijk onderdeel het terugval gedrag. Indien eigen domeinnamen mislukken of tijdelijk incorrect werken, mag de Direct Booking Website niet hieronder leiden. Hiervoor is de oplossing om het gegenereerd sub-domein te gebruiken als publieke terugval, zodat er altijd een website versie actief is. Verder voorkomt dit downtime, waardoor verhuurders geen potentiële omzet mislopen.

Gezien DNS-routing is het volgens AWS Route 53 belangrijk om ondersteuning via alias records te hebben, zodat verkeer op apex-niveau ook naar AWS bronnen, zoals CloudFront, gestuurd kan worden [36]. Met apex domain bedoelen we de eenvoudigste vorm van een domeinnaam. Dus zonder voorvoegsel of sub-domein, zoals voorbeeld.com, in tegenstelling tot www.voorbeeld.com. Dit is vooral relevant op technisch gebied, omdat apex-domeinen niet via een normale CNAME kunnen worden gekoppeld [36].

Keuze

Gegeven de v1 voor Domits is een gegenereerd subdomein als standaardroute, in combinatie met host gebaseerde gebruiker resolutie en een blijvende terugval domein, de meest geschikte keuze. Dit komt doordat deze aanpak niet alleen schaalbaar is, maar ook beheerbaar is op technisch niveau. Verder is deze stroming goed te automatiseren binnen een one-click publicatieproces.

De mogelijkheid om een eigen domeinnaam te kunnen koppelen is erg aantrekkelijk, maar voegt voor een v1 te veel complexiteit toe op gebied van validatie, certificaten & beheer, ondersteuning en om geïmplementeerde functionaliteiten te testen. Het is hierdoor verstandiger om gegenereerde subdomeinen als publieke standaard te gebruiken en bij een toekomstige implementatiefase te kijken naar het kunnen koppelen van een eigen domeinnaam.

5.6 Data integratie

In deze deelvraag wordt onderzocht hoe de Direct Booking Website correct kan blijven aansluiten op beschikbaarheidsdata en actuele prijzen uit het PMS.

Tijdens de implementatie van de losstaande website is het van belang data uit het bestaande Property Management Systeem goed aan te laten sluiten op de toekomstige functionaliteiten. Het gaat hier niet alleen om het tonen van informatie omtrent de accommodatie, maar ook om correcte bezettingsgraad, prijs en boekingen actueel te tonen en verwerken. Er moet onderzocht worden welke data live uitgelezen kan worden vanuit het PMS, welke data veilig kan worden opgeslagen als gepubliceerde snapshot, en of data server-side of client-side ingedeeld wordt.

Data integratie is juist belangrijk omdat foutieve/verouderde data direct invloed kan hebben op betrouwbaarheid, gebruikerservaring en potentiële omzet. Een onjuiste aanpak kan vertrouwen schaden en bovendien leiden tot foutieve boekingen. Op technisch niveau moet deze aanpak niet alleen inhoudelijk kloppen, maar ook goed aansluiten op de toekomstige implementaties en feature uitbreidingen.

Realtime uitlezen

Om actuele data ten alle tijden toonbaar te maken op de site, is er een mogelijkheid om data uit te lezen zodra de pagina bezocht wordt. Het voordeel van deze aanpak is dat de website consistent de meest actuele informatie toont. Wijzigingen binnen de bezettingsgraad of prijs zijn direct zichtbaar zonder extra synchronisatie.

Bij deze aanpak zijn er helaas ook nadelen. Nadelen zoals afhankelijkheid op gebied van bezettingsgraad en prijs uit het PMS. Het probleem is dat niet alle inhoud van de website bij elk bezoek opnieuw opgehaald hoeft te worden. Zo maken we ook gebruik van data die statisch geleverd kunnen worden, zoals titel, beschrijving en foto's. Hierdoor zou het inefficiënt zijn om alle gegevens opnieuw te laden bij een websitebezoek. Daarbij is de Direct Booking Website bij deze vorm onnodig afhankelijk van het PMS en neemt de kans op vertragingen of storingen hierdoor toe.

Live uitlezen sluit hierdoor minder goed aan op de gewenste snel- en robuustheid van Domits.

Caching en snapshots

Naast live data te allen tijde uit te lezen bestaat er ook een mogelijkheid om data te importeren vanuit het PMS en op te slaan via een gepubliceerde snapshot. Kortom leest de site dan geen actuele directe databron, maar weergeeft een opgeslagen gepubliceerde versie.

Synchronisatie gebeurt dan alleen bij publicatie en refresh. Volgens Next.js is het mogelijk om gecachte data te vernieuwen, zodat data niet bij elk request opnieuw opgeroepen hoeft te worden [38]. Dit kan via time-based of op aanvraag gesynchroniseerd worden. Daarnaast kan de inhoud tijdelijk opgeslagen worden, vanwege ondersteuning van stale-while-revalidate en stale-if-error door Cloudfront [39]. Op de achtergrond wordt tegelijkertijd een nieuwe versie opgehaald. Deze vorm is nuttig bij tijdelijke onbeschikbaarheid en behoud kostenefficiëntie.

Gegeven Domits met betrekking tot getoonde informatie op de pagina, zoals titel, beschrijving, foto's, voorzieningen/amenities, locatie, policies/terms of service en kalender is deze aanpak geschikt.

Tegelijkertijd wordt de site rendering sneller, levert dit consistente stabiliteit, en minder afhankelijkheid van live PMS aanvragen, indien we gegevens opslag met behulp van snapshots. De website blijft, in andere woorden, hierdoor bruikbaar functioneren ongeacht PMS vertragingen of uitval.

Om duidelijkheid te krijgen over welke informatie waar vandaan komt, is er een tabel opgesteld die deze structuur visueel in kaart brengt.

Dit onderstaande Tabel toont dat data gescheiden is. Een deel is namelijk afkomstig van het PMS, terwijl andere informatie zich binnen de Direct Booking Website Builder bevindt. Hiertussen is er een bewuste scheiding gemaakt om importeerbare inhoud en live data uiteen te houden.

Content Mapping Tabel			
Bronveld	Eigenaar	Standalone-sectie	Notities
property.title	PMS-bron -> standalone gepubliceerde snapshot	hero	geïmporteerd bij publish/refresh
property.subtitle	PMS-bron -> standalone gepubliceerde snapshot	hero	geïmporteerd bij publish/refresh
property.description	PMS-bron -> standalone gepubliceerde snapshot	hero / intro_highlights	geen niet-bestaande about-sectie gebruiken
images[]	PMS-bron -> standalone gepubliceerde snapshot	gallery	geïmporteerd bij publish/refresh
amenities[]	PMS-bron -> standalone gepubliceerde snapshot	amenities	geïmporteerd bij publish/refresh
location.city / location.country	PMS-bron -> standalone gepubliceerde snapshot	location	volledig adres alleen tonen als beleid dit toestaat
rules[]	PMS-bron -> standalone gepubliceerde snapshot	house_rules	geïmporteerd bij publish/refresh
availability.window	PMS-bron -> standalone weergavesnapshot	availability_quote	alleen voor weergave; quote API valideert live
property.timezone	PMS-bron -> standalone quote-context	availability_quote	opgeslagen op standalone_site; leidend voor quote-gedrag
site_name	Standalone	globaal / hero	door host beheerde sitenaam
logo_asset_key	Standalone	globaal / hero	S3-brand asset
favicon_asset_key	Standalone	globaal	S3-browser asset
primary_locale	Standalone	globaal	één sitetaal in v1 (Engels)
contact_name	Standalone	contact	publieke override
contact_email	Standalone	contact	publieke override
contact_phone	Standalone	contact	publieke override
policy_content	Standalone	policy	publieke override
content_overrides.json	Standalone	meerdere secties	override-laag boven geïmporteerde snapshot waar toegestaan
theme_tokens.json	Standalone	alle secties	runtime theme tokens

Tabel 4: Content Mapping Tabel

Evenement gedreven synchronisatie

Een andere mogelijkheid om website informatie te wijzigen is via evenement gedreven synchronisatie. AWS beschrijft dit als een manier om systemen van elkaar los te koppelen en asynchroon te laten reageren [40]. EventBridge wordt gebruikt om events hiervoor te routeren naar verschillende consumenten zonder een expliciete koppeling tussen verhuurder en gast vast te stellen. [41].

Deze mogelijkheid is vooral voor latere optimalisatie relevant. Wanneer wijzigingen van data verricht worden in het PMS, kan een toekomstige trigger een refresh van de Direct Booking Website snapshot triggeren. Hierdoor blijft content automatisch up to date zonder manuele actie.

Gezien een v1 is deze aanpak niet noodzakelijk en is een refresh model meer voor de hand liggend, beter controleerbaar en voldoende voor de eerste versie, mits prijs data en bezettingsgraad actueel blijven.

Koppeling van PMS data aan de template structuur

Na de data vorm vastgesteld te hebben is het volgende aandachtspunt hoe bestaande PMS data gekoppeld wordt aan de website template structuur. Dit betreft niet alleen data uitlezen, maar een directe één op één aansluiting en koppeling van informatie aan geïmplementeerde template elementen en secties. Op deze manier is PMS data herbruikbaar en biedt het gebruikers een snellere ervaring.

Gezien ondersteuning van diverse templates, zonder dat elke template een apart datamodel of een geheel eigen implementatie nodig heeft, is deze aanpak voor Domits een belangrijke optie.

De belangrijkste oplossing is een gedeelde mappinglaag die de één op één aansluiting van PMS data mogelijk maakt, onderverdeeld in vaste secties, zoals hero, gallery, amenities, policies en photos. Aan deze vaste secties wordt dan bestaande listing data gekoppeld. Denk aan: titel, beschrijving en foto's. Bij deze manier van data aansluiting wordt de mappinglaag zo generiek mogelijk geïmplementeerd. Uitbreiding met nieuwe templates is hierdoor beter beheersbaar.

Foutscenario's en consistentie

Correcte aansluiting op PMS data betekent ook dat de website voorspelbaar moet reageren bij fouten. Denk aan tijdelijke uitval van het PMS, vertraging van prijsinformatie of cache data die net vernieuwd kan worden. Voor Domits is het daarom belangrijk dat niet-essentiële data beschikbaar blijft via snapshots, terwijl essentiële data altijd via de server gevalideerd moet worden voordat interactie hierop gebaseerd wordt. Dit onderscheid zorgt voor een website die gedeeltelijk bruikbaar blijft bij storingen. Tegelijkertijd voorkomt dit dat bezoekers onterecht vertrouwen op

data omtrent boekingen.

Keuze

Gegeven de eisen van Domits is een hybride data integratieaanpak het meest geschikt. Hierdoor wordt diverse informatie uit het PMS geïmporteerd in een gepubliceerde snapshot, terwijl data waaronder prijs en bezettingsgraad actueel getoond worden door server-side connectie uitlezing vanuit Domits haar PMS. Deze aanpak zorgt voor optimalisatie in performance en versiebeheer. Wel is evenement gedreven synchronisatie logisch bij toekomstige uitbreiding, maar niet noodzakelijk voor een v1. Op deze manier bieden we balans tussen performance, stabiliteit, schaalbaarheid en correctheid van data.

5.7 Security

Deze paragraaf richt zich op de vraag welke securitymaatregelen nodig zijn om een multi-tenant losstaand websiteplatform veilig te realiseren binnen Domits.

Nadat de vorm van data integratie onderzocht is, is de beveiliging van deze data ook zeer relevant. Domits wil meerdere verhuurders op een gedeelde infrastructuur bedienen en beheren, terwijl elke host enkel toegang mag hebben tot zijn of haar eigen websites, drafts, configuratie van domein en gekoppelde accommodatie data. Tegelijkertijd moet de publiekelijke website veilig kunnen functioneren, zodat potentiële gasten data niet kunnen manipuleren, toegang kunnen krijgen tot data van andere verhuurders of foutieve prijs- en bezettingsgraad informatie te zien krijgen.

Risico Matrix				
Risico	Oorzaak	Invloed	Voorzorg	Detectie
Verouderde content op live site	PMS-content verandert na publish of refresh gebeurt te laat	Verkeerde titel, foto's, beschrijving of voorzieningen zichtbaar	Snapshot refresh + republish flow met zichtbare versiebeheer	Monitoring op snapshot-leeftijd + refresh audit logs
Prijs of beschikbaarheid klopt niet meer	Data verandert na genereren van quote	Gast ziet verkeerde prijs	Korte expiry + hervalidatie vóór boeken	Quote conflict ratio + stale-quote metrics
Site niet bereikbaar of verkeerde site geladen	Foutie DNS / mapping	Geen site of verkeerde tenant zichtbaar	Unieke domain index + validatie + fallback domein	Resolve logs + synthetic host checks
Data van ene tenant zichtbaar bij andere	Host resolution bug / geen tenant validatie	Datalek (groot incident)	Resolve op host + valideer site + noot client properlyd vertrouwen	Cross-tenant mismatch alarmen + request tracing
Verkeerde database schema wordt gebruikt	Queries raken public i.p.v. main	Foutie data / missende records	Expliciet main.table_name + verificatie SQL + tests	Informatie checks + rollout verificatie
Verkeerde site wordt geserveerd (v2)	Cache key niet gebaseerd op host	Cross-tenant content lek	Host-aware cache keys + geen path-only caching	Synthetische oplossende checks + cache anomaly monitoring
Fake host request manipuleert routing	Onverwachte / vervalse host headers	Tenant kaping / verkeerde content	Host toelatinglijst + strenge validatie	vreemde of verdachte host logs + mismatch alerts
Manipulatie van prijs via client (v2)	Client past quote aan	Verkeerde prijs / inconsistentie	Alleen Server-side prijsbepaling	Quote vs booking mismatch checks + logs
Overbelasting (v2)	Massa aanvragen	Prestatievermindering	Rate limiting + throttling + WAF	Meldingen over aanvraagpercentages + dashboards voor misbruik
Oude quote opnieuw gebruikt (v2)	Hergebruik token	Ongeldige boeking	Vervalling + revalidatie	Invalid token metrics + replay detectie
Injectie of script exploit (v2)	Onveilige content of escaping	Sessie kaping / beschadiging	Strikte CSP + escaping + vertrouwde assets	Browser reports + frontend error monitoring

Tabel 5: Risico Matrix

Tenant-isolatie en identiteit

Om ervoor te zorgen dat gebruiker A niet bij de data van gebruiker B kan komen, is een sterke fundamentele tenant-isolatiestructuur bij een multi-tenant platform, zoals Domits, vereist. Tenant-isolatie zorgt er volgens AWS expliciet voor dat gebruikers slechts bij hun eigen resources kunnen, en deze niet toegankelijk zijn voor andere accounts, zelfs onder een gedeelde infrastructuur [42]. Hierbij maakt AWS onderscheid tussen authenticatie en daadwerkelijke tenant-isolatie. Zo kan een gebruiker geauthenticeerd en geautoriseerd zijn, en alsnog onrechte toegang krijgen tot data van een ander als isolatie niet goed is opgelegd [42], [43].

Dit betekent voor Domits dat de server leidend moet zijn met betrekking tot identiteit. Bij een zwakke architectuur vertrouwt de backend geheel data die wordt meegestuurd vanuit de frontend. OWASP (Open Worldwide Application Security Project) benoemt juist dat API's fragiel zijn wanneer object identifiers vanuit de client worden gebruikt zonder dat object-level autorisatie checks aanwezig zijn [44]. Daarom moet bij host functionaliteiten, zoals welke accommodatie bij welke host hoort, server-side gevalideerd worden. Daarbij worden verzoeken bepaald door hostnaam, domein en identiteit. Hierdoor verzekeren we validatie via de server in plaats van via een website verzoek.

Token- en toegangsbeheer

Token- en toegangsbeheer komen kijken bij het opslaan, publiceren en de-publiceren van de website feature. Hier is het belangrijk dat deze host acties alleen mogelijk zijn mits toegang geautoriseerd is via Domits. Bij paden, zoals preview- of live gebruik, moet juist voorkomen worden dat rechten in de browser terechtkomen. AWS Prescriptive Guidance adviseert hierbij om API toegang en logica in een multi-tenant SaaS systeem consistent af te dwingen, bijvoorbeeld via policies en enforcements points die centraal staan op API niveau [43]. Dit is relevant wanneer meerdere routes binnen een gedeeld platform verschillende rechten omvatten.

Gegeven Domits betekent dit dat toegang doelgericht moet blijven. Voor bijvoorbeeld preview of andere functionaliteiten, is toegang gedrag nodig. Dit is voor de handliggend door met korte duur, gebonden tokens of gecontroleerde routes te werken, los van brede systeemrechten. Hierdoor kan de frontend wel tonen wat nodig is, maar nooit toegang verleend aan gegevens of andere bevoegdheden.

Misbruik preventie en rate limiting

Naast identiteit en tokens is bescherming tegen misbruik noodzakelijk. Over het algemeen kunnen publieke endpoints aantrekkelijk zijn voor site scraping, spamming of overbelasting. OAWSP benoemt resource consumptie een vaak voorkomend API risico en adviseert onder andere limieten op interactie, snelheidsbeperking/throttling en requestparameters [45]. Voor Domits is deze aanbeveling relevant, omdat deze aanpak gebruikmaakt van een gedeelde runtime en een gedeelde API infrastructuur bij live websites. Zonder een begrenzing kunnen misbruik scenario's invloed hebben op andere websites binnen dezelfde omgeving.

Browserbeveiliging en Content Security Policy

De Direct Booking Website toont dynamische inhoud, waaronder beschrijving, afbeeldingen, logo en later nog meer host aanpasbare velden. Hierdoor ontstaat risico op kwetsbaarheden, zoals cross-site scripting, indien onvoldoende validatie plaatsvindt. OWASP adviseert hiervoor een samenstelling van framework bescherming, output encoding en HTML sanitization [46]. Daarbij wordt CSP niet gezien als vervanging, maar ingezet als een extra bescherming [47].

Gezien Domits is dit relevant, omdat hosts aanpassingen kunnen verrichten binnen een gedeelde omgeving. Een strak Content Security Policy biedt beperking op welke scripts, stijl, afbeeldingen en externe data geladen mag worden. Zo verkleint dit de impact van foutieve inhoud [48]. Binnen Domits ontstaat hierdoor een sterke beveiligingslaag voor de Direct Booking Website door middel van server validatie,

veilige rendering en data ontsmetting.

Privacy en dataminimalisatie

Privacy speelt ook een grote rol binnen deze analyse. Niet alle PMS data hoort automatisch zichtbaar of opgeslagen te kunnen worden omtrent de website. Volgens AVG (Algemene Verordening Gegevensbescherming) mogen alleen persoonsgegevens worden verwerkt die relevant zijn en niet verder gaan dan noodzakelijk is voor het verwerkingsdoel. Daarnaast moeten gegevens doelgebonden verwerkt worden en worden beveiligd.

Voor Domits betekent dit dat alleen beschrijvende en publieke content in de website terecht mag komen, terwijl gevoelige gegevens bij de server moeten blijven. Ditzelfde geldt voor analytics door middel van KPI's en gebruikersregistratie. Deze data is waardevol, maar mag alleen gegevens bevatten die noodzakelijk zijn voor technische evaluatie. Zo blijft deze oplossing niet alleen veilig, maar ook beter op basis van privacy en compliance.

Keuze

Gegeven Domits is de meest geschikte keuze een gedeelde runtime met strikte server-ide tenant-isolatie, beperkte toegang gevalideerde tokens waar nodig, rate limiting op routes en een CSP gestuurde beveiliging. Een volledig geïsoleerde tenant infrastructuur zou de sterkste scheduling bieden, maar is naar verhouding tot de gewenste schaalbaarheid en begroting niet gewenst voor een v1 [43], [44]. De gedeelde runtime blijft voldoende, zolang isolatie en validatie niet overgelaten worden aan de frontend, maar via backend, routing en publicatie flow worden afgedwongen [43].

5.8 KPI's

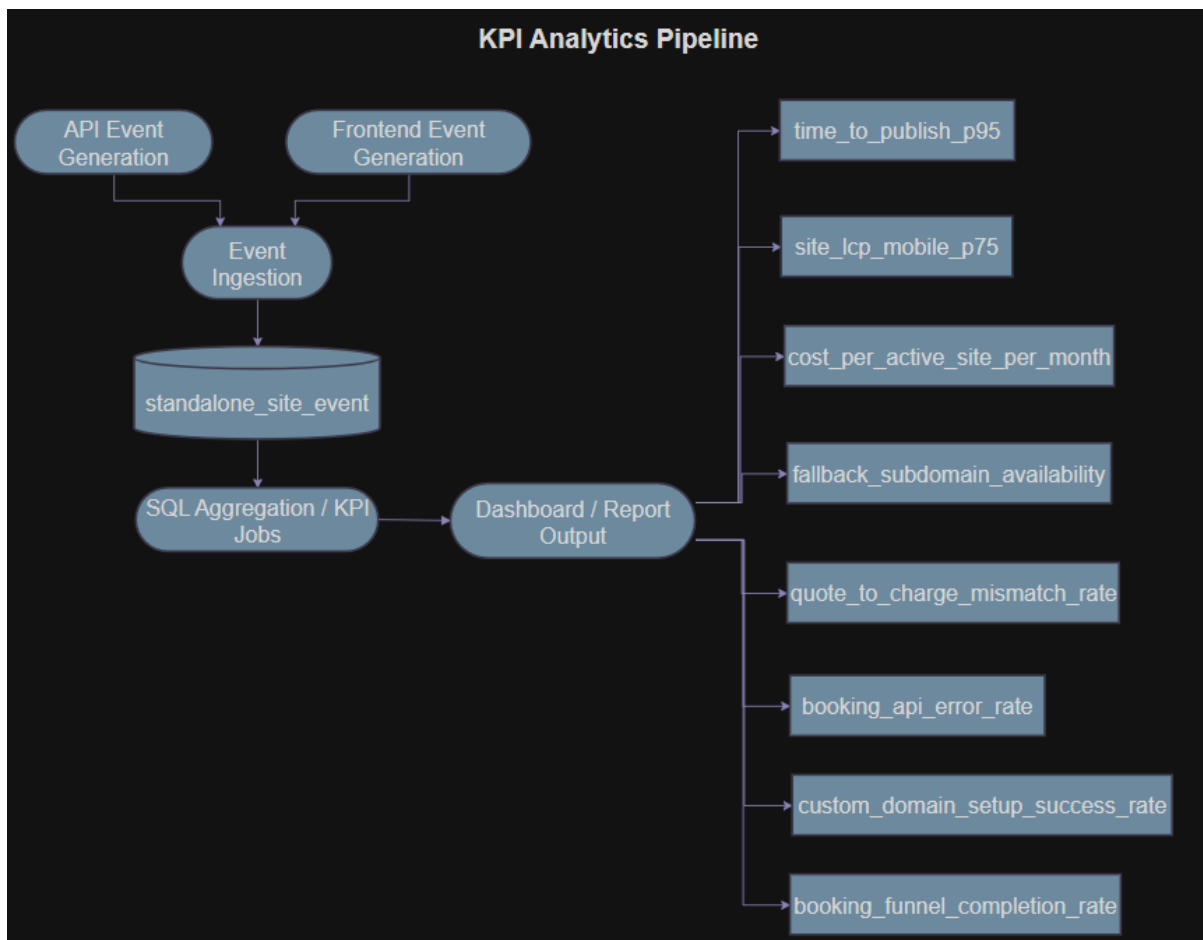
In deze deelvraag wordt onderzocht welke KPI's geschikt zijn om de technische en productmatige kwaliteit van de oplossing op een meetbare manier te evalueren.

Binnen dit onderzoek wordt gestreefd naar een productie-ready implementatie. Maar om daadwerkelijk te achterhalen of dit product wel valide is, moeten functionaliteiten getest worden. Dit komt doordat een gerealiseerde feature niet automatisch betekent dat het ook operationeel succesvol is. Naast het afnemen van interviews, verbaal te communiceren en nauwkeurig te letten op lichaamstaal, kunnen we de informatie tijdens het testen ook digitaal afnemen. Om dit te kunnen doen zijn er meetbare indicatoren nodig. KPI's maken het mogelijk om niet alleen informatie te vergaren door handmatig te testen, maar ook extra inzicht te krijgen in hoe de feature zich gedraagt bij gebruik.

Rol van KPI's binnen dit onderzoek

Binnen dit onderzoek zijn KPI's relevant omdat deze aanpak meerdere criteria tegelijkertijd kan beoordelen. In Validatie en meetplan is al vastgesteld dat schaalbaarheid, kosten, performance, betrouwbaarheid, correctheid en gebruikerservaring behoren tot de beoordelingscriteria. Hierdoor zijn de KPI's niet alleen aanvullende statistieken, maar eerder concrete meetpunten die extra inzicht geven in hoeverre de Direct Booking Website feature voldoet aan de vastgestelde eisen.

De werking van dataverzameling is in Figuur 8 hieronder visueel toegelicht:



Figuur 8: KPI Analytics Pipeline

Geschikte KPI's voor v1

Niet elke KPI's zijn relevant voor een eerste versie. Voor een v1 moeten vooral KPI's gekozen worden die aansluiten op de gerealiseerde functionaliteiten en de belangrijkste risico's. Die Key Performance Indicators bestaan uit:

time_to_publish_p95

Deze KPI meet hoe snel een host van configuratie naar een live website gaat. Deze metric is relevant, omdat de standalone site bedoeld is als snelle, laagdrempelige publicatie flow. p95 voorkomt dat alleen de gemiddelden worden bekeken en laat ook tragere, maar realistische gevallen zien. Kortom willen we dat data niet is gladgestreken, maar in ruwe vorm getoond wordt.

site_lcp_p75 (mobile, tablet, desktop)

Deze KPI meet hoe snel de grootste zichtbare inhoud op diverse apparaten geladen wordt. Dit is relevant omdat mensen via verschillende apparaten de website bezoeken en performance direct invloed heeft op de gebruikerservaring en boeking conversie. De keuze voor p75 geeft een realistischer beeld van de ervaring van de meeste gebruikers dan alleen gladgestreken gemiddelden.

cost_per_active_site_per_month

Deze KPI meet wat het gemiddeld kost om een actieve Direct Booking Website per maand te draaien. Deze KPI is belangrijk om te beoordelen of de gekozen hosting- en infrastructuur aanpak schaalbaar en kostenefficiënt blijft nadat het aantal sites toeneemt. Een te hoog bedrag maakt het model op lange termijn minder lucratief voor Domits.

fallback_subdomain_availability

Deze KPI meet hoe betrouwbaar de publieke website standaardroute is. In de eerste versie vormt het subdomein de voornaamste publieke ingang van de website. Hierdoor is de beschikbaarheid van deze site direct gekoppeld aan bereikbaarheid, vertrouwen en potentiële omzet.

KPI's voor correctheid en latere implementatiefase

Deze tweede KPI categorie richt zich op correctheid en toekomstige feature uitbreidingen. Deze KPI's zijn zeer relevant, maar niet volledig meetbaar binnen een v1. Deze Key Performance Indicators bestaan uit:

quote_to_charge_mismatch_rate

Deze KPI meet hoe vaak een uitgelezen informatie afwijkt van het uiteindelijke resultaat. Deze KPI is belangrijk op gebied van controle omtrent consistente uitlezing van prijzen en bezettingsgraad van het PMS. Een (hoge) afwijking kan vertrouwen schaden en benadrukt fouten van prijs logica, caching of herstel. De KPI is meer van belang zodra boeking implementatie voltooid is.

custom_domain_setup_success_rate

Deze KPI meet hoe vaak het instellen van een custom domain succesvol verliep. Omdat custom domains niet behoren tot de implementatie van een v1, is deze KPI vooral bedoeld als voorbereidende metric voor een latere fase. Een laag succesratio duidt op technische complexiteit, slechte onboarding of fouten binnen het DNS proces.

booking_api_error_rate

Deze KPI meet hoe vaak boeking gerelateerde API verzoeken mislukken. Dit is relevant om de betrouwbaarheid en stabiliteit van de backend te waarborgen, vooral bij latere uitbreiding van de Direct Booking Website in een v2, waarbij checkouts en boekingen mogelijk worden. Een hoge foutscore duidt op technische instabiliteit en/of onvoldoende foutieve checks.

booking_funnel_completion_rate

Deze KPI meet hoeveel gebruikers de boeking flow geheel doorlopen. Deze data is relevant om inzicht te krijgen in gebruiksgedrag, uitvalmomenten en de effectiviteit van de site flow. Deze KPI is bedoeld voor de tweede fase, waarbij nut zit in toekomstige succesmeting en uitbreiding.

Koppeling aan beoordelingscriteria

De gekozen KPI's sluiten aan op de criteria uit het vergelijkingskader.

time_to_publish_p95 zegt iets over gebruikerservaring en haalbaarheid.

site_lcp_mobile_p75 meet performance. cost_per_active_site_per_month sluit aan op kosten en schaalbaarheid. fallback_subdomein_availability sluit aan op betrouwbaarheid en bereikbaarheid. quote_to_charge_mismatch_rate en booking_api_error_rate richten zich op correctheid en stabiliteit van de koppeling met PMS- en booking logica.

6. Ontwerp

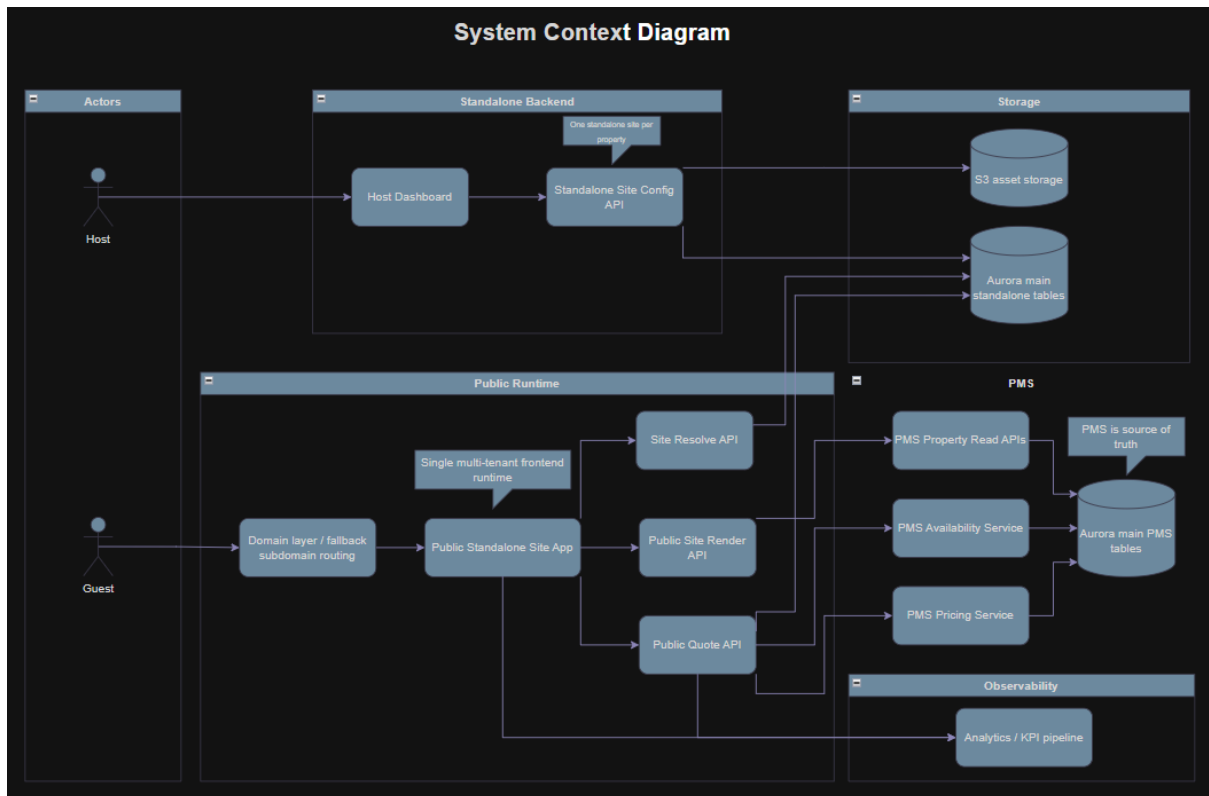
Nadat de analyse is afgerond is het van belang om nu te focussen op het ontwerp, de stappen die ondernomen worden en de toekomstige realisatie van het product zelf.

6.1 Architectuur overzicht en datamodel

Na de analysefase is het belangrijk om de gekozen oplossing te vertalen naar een technisch ontwerp. Dit ontwerp moet duidelijk maken hoe de standalone website feature binnen Domits aansluit op de bestaande omgeving, zonder de huidige PMS functionaliteit te verstoren. De feature moet namelijk gebruik maken van bestaande listingdata, maar tegelijk eigen website informatie kunnen opslaan, zoals het gekozen template, tekstaanpassingen, zichtbaarheid van secties en later ook publicatie- en domein status.

Om dit overzichtelijk te maken is eerst een System Context Diagram opgesteld. Dit diagram laat zien welke onderdelen betrokken zijn bij de standalone website feature en hoe de datastromen tussen host, gast, frontend, backend, opslag en PMS verlopen. Hierbij gebruikt de verhuurder een bestaande listing en selecteert een template om de website draft samen te stellen. Deze configuratie loopt via een Standalone Site Config API, waarna de backend controleert of de gekozen listing geldig is en deze data uit het PMS kan worden opgehaald.

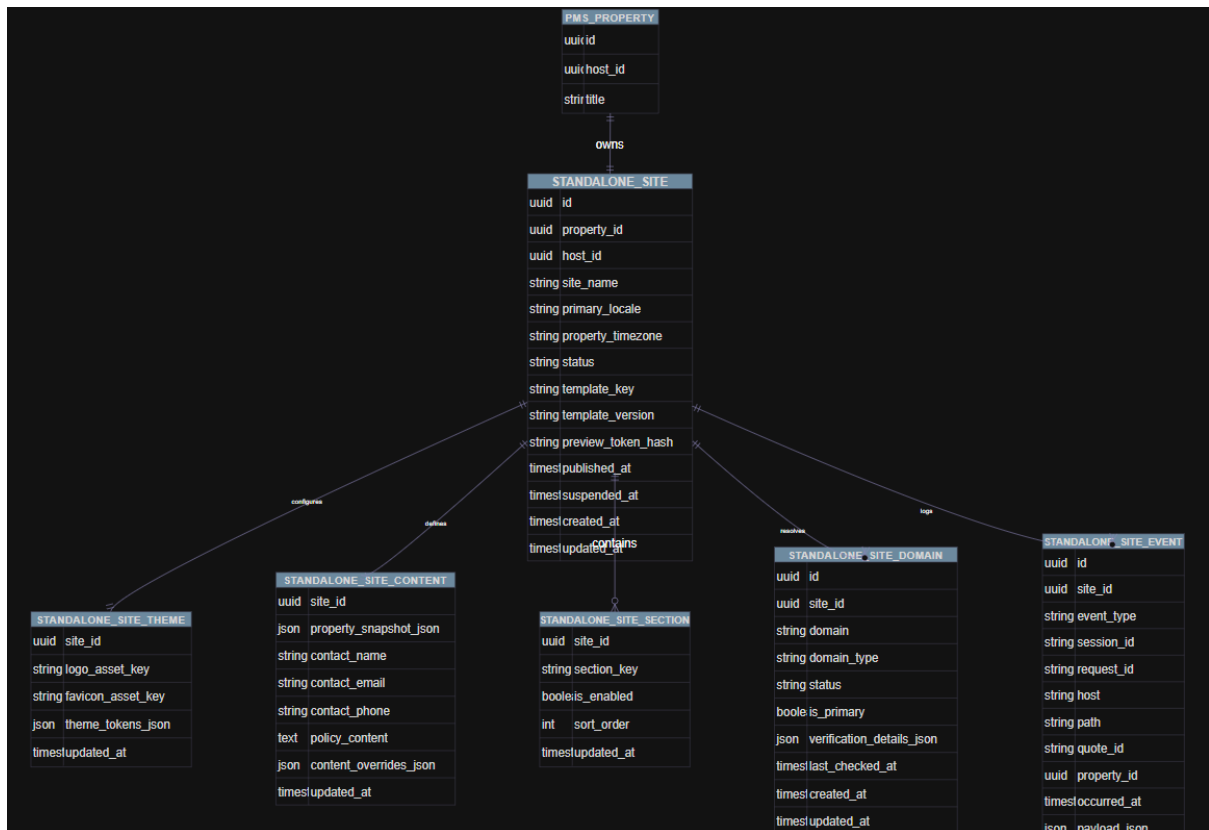
In het onderstaande Figuur 9 is er een duidelijke scheiding zichtbaar tussen host en gast, en wat zij te zien krijgen. Gast ziet onder andere niet het host dashboard, maar een publieke website runtime. Dit wordt getoond op basis van domein, subdomein of voorvertoning link. De publieke rendering komt tot stand door het ophalen van de juiste website configuratie uit de standalone tabellen en gebruikt deze alleen voor website presentatie. Data dat actueel moet blijven, zoals prijs en bezettingsgraad, wordt rechtstreeks uit het PMS als bron gelezen. Dit voorbeeld van datascheiding wordt gehanteerd zodat deze feature geen losstaand systeem wordt dat eigen waarheid gaat bijhouden over prijs en boekingen.



Figuur 9: System Context Diagram

Naast het contextdiagram is een Entity Relationship Diagram opgesteld. Dit figuur verduidelijkt welke data de website feature nodig heeft en hoe deze data gekoppeld wordt aan bestaande PMS data. De bedoeling is dat een samengestelde site gekoppeld is aan één listing. Vanuit de koppeling kan de site de juiste propertydata gebruiken, terwijl de website zelf geen eigen configuratie beheert. Een voorbeeld hiervan is een gekozen template, site-status, thema en sectie configuratie.

Daarnaast moeten we data integreren tussen bestaande tabellen binnen AWS. Het onderstaande ERD (Entity Relationship Diagram) biedt inzicht in hoe AWS geconfigureerd moet worden om de Direct Booking Website data op te kunnen slaan en om de functionaliteiten aan te kunnen, met betrekking tot tabelverbintenis, hun attributen en relaties.



Figuur 10: Entity Relationship Diagram

De waarborging en voorkoming van vervuiling binnen bestaande tabellen is bij dit ontwerp van belang, en dat deze niet onnodig aangepast worden bij toevoeging van nieuwe tabellen voor de nieuwe feature.

De PMS listing blijft verantwoordelijk voor accommodatie informatie en operationele gegevens, terwijl de website feature verantwoordelijk is voor presentatie, template keuze en publicatie status. Hierdoor krijgt de host een simpele manier om een losstaande website samen te stellen op basis van bestaande listing data, terwijl Domits controle houdt over datakwaliteit, gebruikers isolatie en toekomstige uitbreiding.

6.2 Template- en contentmodel

Binnen de configuratie van de standalone website wordt er gebruikgemaakt van een template systeem. De verhuurder heeft niet de mogelijkheid tot een vrije pagina builder, maar maakt een keuze uit een vooraf ontworpen template. Deze keuze sluit aan op de v1 scope, omdat Domits hiermee meerdere direct booking websites kan aanbieden zonder dat elke website een los project wordt.

De keuze uit verschillende templates wordt getoond in een grid layout op het Host Dashboard -> Website tab. Hierdoor kan de host meerdere templates snel vergelijken. Grid grootte past zich aan op de beschikbare resolutie (mobiel / desktop). De templates weergaven geen volledige websites, maar een eenvoudig silhouet preview. Deze preview toont de grove website structuur, waaronder hero, CTA cards, fotogalerij en tekstvelden.

De website inhoud komt niet uit het template zelf. Hiervoor wordt een gedeeld basismodel gebruikt, waarbij data uit het PMS eerst wordt vertaald naar een vaste structuur en daarna aansluiten op beschikbare templates. Denk hierbij aan secties zoals titel, beschrijving, foto's, prijs, bezettingsgraad en regels.

Zoals beschreven in [Data integratie](#) zorgt een basismodel ervoor dat templates alleen bepalen hoe inhoud wordt getoond, niet wat de betekenis van de data is. Een titel blijft hierdoor altijd een titel, ongeacht of deze anders is ingedeeld binnen verschillende templates.

Voor een v1 betekent dit dat de host een listing kan selecteren, een template uitkiest en daarna een preview krijgt te zien waarin de listingdata automatisch op de juiste plekken wordt geplaatst. Daarnaast kan de gebruiker onderdelen beperkt aanpassen, zoals tekst, zichtbaarheid van elementen en toning van afbeeldingen. Een drag-and-drop functionaliteit is bewust buiten de scope gehouden, omdat dit de complexiteit van rendering en validatie significant verhoogt.

Dit template systeem vormt hierdoor een plug-and-play tussen bestaande PMS data en de visuele websiteweergave. De data wordt kortom in één keer vertaald naar een gedeeld tussenmodel, waarna elk template data op zijn eigen manier presenteert. Door deze aanpak blijven templates uitbreidbaar zonder dat toekomstige toevoeging van templates bestaande websites breken.

Content Mapping Tabel			
Bronveld	Eigenaar	Standalone-sectie	Notities
property.title	PMS-bron -> standalone gepubliceerde snapshot	hero	geïmporteerd bij publish/refresh
property.subtitle	PMS-bron -> standalone gepubliceerde snapshot	hero	geïmporteerd bij publish/refresh
property.description	PMS-bron -> standalone gepubliceerde snapshot	hero / intro_highlights	geen niet-bestaande about-sectie gebruiken
images[]	PMS-bron -> standalone gepubliceerde snapshot	gallery	geïmporteerd bij publish/refresh
amenities[]	PMS-bron -> standalone gepubliceerde snapshot	amenities	geïmporteerd bij publish/refresh
location.city / location.country	PMS-bron -> standalone gepubliceerde snapshot	location	volledig adres alleen tonen als beleid dit toestaat
rules[]	PMS-bron -> standalone gepubliceerde snapshot	house_rules	geïmporteerd bij publish/refresh
availability.window	PMS-bron -> standalone weergavesnapshot	availability_quote	alleen voor weergave; quote API valideert live
property.timezone	PMS-bron -> standalone quote-context	availability_quote	opgeslagen op standalone_site; leidend voor quote-gedrag
site.name	Standalone	globaal / hero	door host beheerde sitenaam
logo_asset_key	Standalone	globaal / hero	S3-brand asset
favicon_asset_key	Standalone	globaal	S3-browser asset
primary_locale	Standalone	globaal	één sitetaal in v1 (Engels)
contact.name	Standalone	contact	publieke override
contact.email	Standalone	contact	publieke override
contact.phone	Standalone	contact	publieke override
policy.content	Standalone	policy	publieke override
content_overrides.json	Standalone	meerdere secties	override-laag boven geïmporteerde snapshot waar toegestaan
theme_tokens.json	Standalone	alle secties	runtime theme tokens

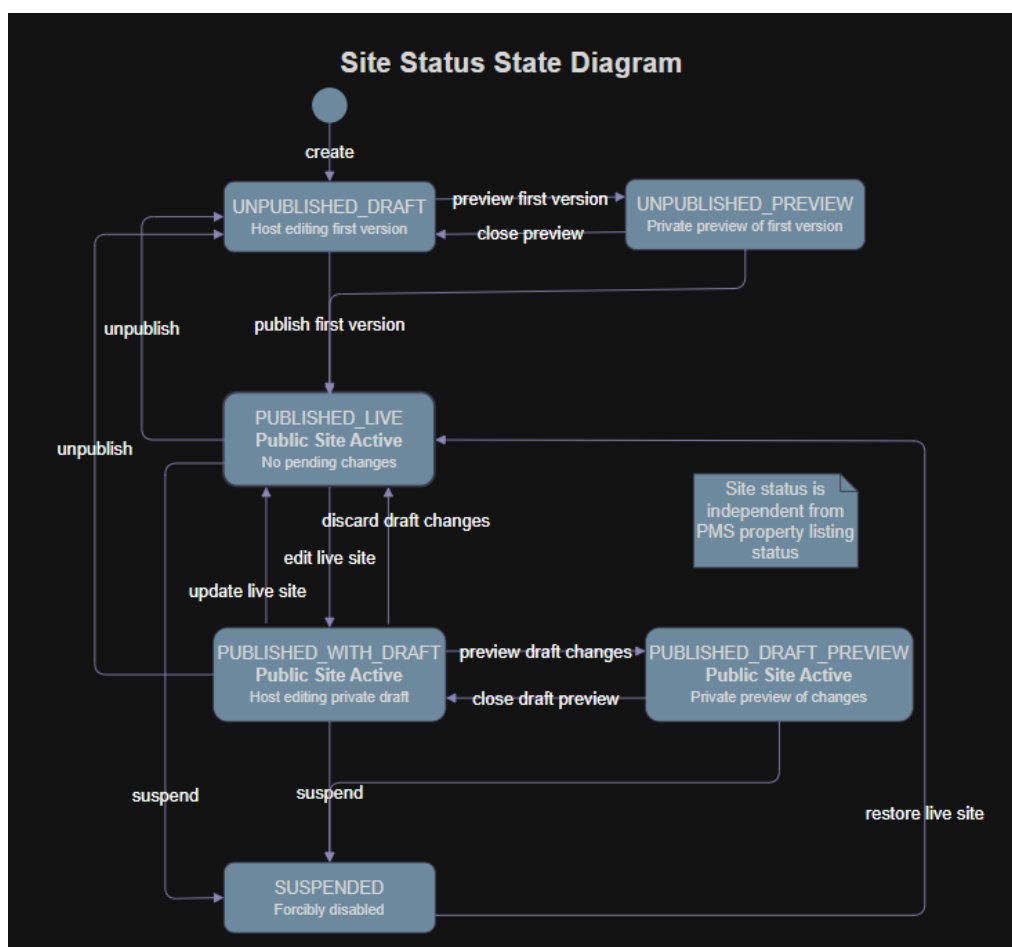
Tabel 4: Content Mapping Tabel

6.3 Website lifecycle en publicatieflow

De Direct Booking Website moet niet alleen opgebouwd kunnen worden, maar ook beheersbaar blijven nadat een verhuurder een site heeft gemaakt binnen Domits. Om overzicht te houden voor de realisatie is er een duidelijke lifecycle nodig. Hiermee wordt het direct tonen van (onafgemaakte) wijzigingen op de gepubliceerde website voorkomen. De lifecycle bepaalt namelijk in welke status de website zich bevindt, wat de gebruiker kan aanpassen en welke versie zichtbaar is voor gasten.

Wanneer een tenant een website bouwt, begint de site als een conceptversie/MVP. Hiervoor selecteert de host een bestaande listing binnen Domits en maakt een keuze uit het template selectiemenu. De backend haalt vervolgens de relevante PMS data op, controleert de accommodatie op validiteit, en stelt een website draft samen. Hierdoor bevat de website de gekozen stijl, zichtbaarheid van diverse elementen en afbeeldingen, rechtstreeks vanuit het PMS. De host kan daarna de website inzien door middel van een voorvertoning, en heeft de mogelijkheid om verdere aanpassingen te verrichten, voordat de site live gepubliceerd wordt.

Het Site Status Diagram hieronder laat zien welke statussen doorlopen worden bij deze feature. Zo is in de eerste fase te zien dat er alleen een ongepubliceerde draft bestaat. Dit geeft de host bewerkingrechten zonder dat er per ongeluk onafgemaakt werk getoond kan worden.



Figuur 11: Site Status State Diagram

Vanuit deze status kan de verhuurder een live preview openen, bedoeld om de website in volledige resolutie te beoordelen.

Na publicatie verandert de status van “draft” naar “published live”. Bij deze status is er een toegankelijke versie beschikbaar voor externen. Bij deze status worden aanpassingen binnen de lokale website weerhouden als vertoning op de live website. Dit geeft de host vrijheid in veranderingen, zonder dat deze automatisch getoond worden op de publiekelijke versie. Hierdoor ontstaat er een nieuw concept bovenop de live site, aangegeven in het Diagram als “published with draft”.

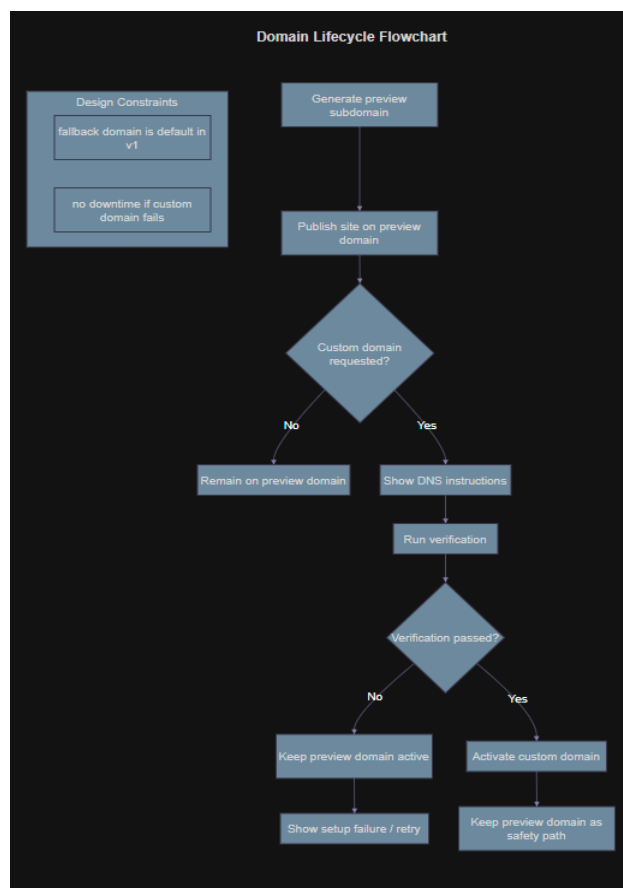
Een scheiding tussen een draft en live versie is belangrijk omtrent kwaliteit en betrouwbaarheid. Zonder deze scheiding is de kans zeer groot op onafgemaakt werk. Bovendien is dit een onwenselijke situatie, vooral omdat Domits zich juist richt op professionele accommodaties, waarbij uitstraling, betrouwbaarheid en vertrouwen rondom boekingen uiterst belangrijk zijn. Daarom moet de host aangebrachte wijzigingen eerst kunnen controleren via een voorvertoning, voordat deze doorgezet worden naar de live versie.

6.4 Routing, preview links en domeinbeheer

Naast publicatiestatus en de inhoud van een website moet ook bepaald worden via welk pad de website beschikbaar is. Hierbij gaat het niet om wat er te zien is, maar of het toegankelijk is. Binnen dit onderdeel onderscheiden we preview links, fallback subdomeinen en custom domains van elkaar.

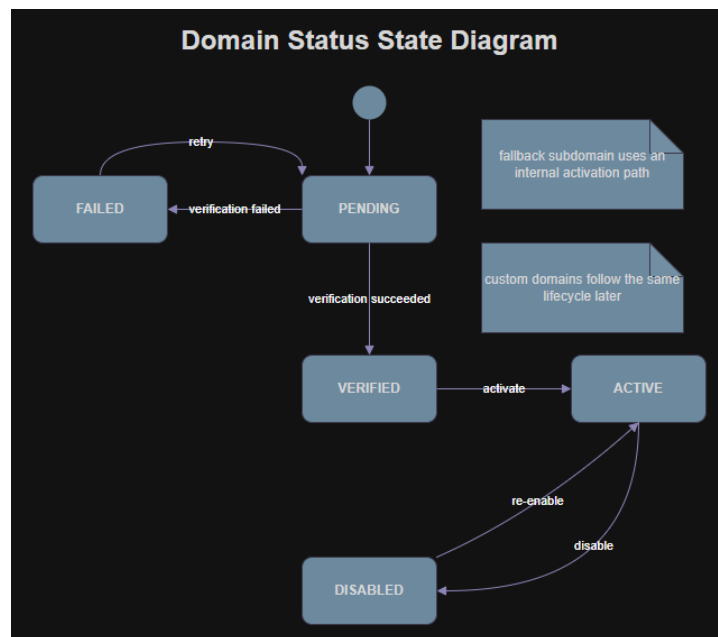
Een preview link is bedoeld om een website in volledige resolutie te bekijken en om te delen met kennissen voor inzicht. Deze versie geldt echter niet als een publieke versie. Een fallback subdomein vormt een navigatie- en standaardpad voor daadwerkelijke publicatie. Dit komt van pas indien het custom domein nog niet is ingesteld of faalt bij gebruikers.

Beginnend is de website eerst toegankelijk via een preview- of fallback route, zoals hieronder te zien in de Domain Lifecycle Flowchart. De host kan vanuit dit punt ervoor kiezen om dit standaardpad te blijven gebruiken of een eigen domein in te stellen. Een validatie controle vindt plaats indien er een custom domein toegevoegd wordt, vervolgens wordt het domein geactiveerd.



Figuur 12: Domain Lifecycle Flowchart

De status van een domein wordt hieronder toegelicht in een Domain Status State Diagram. De status kan bestaan uit pending, verified, active, failed en disabled. Dit geeft aan of een website zich bevindt in draft of gepubliceerd is. En betekent dat een website publiekelijk kan zijn, terwijl een eigen domein nog niet gekoppeld of actief is. Voor een eerste versie ligt de focus bij preview links en een veilige fallback route. Ongeacht dat custom domains behoren tot een latere fase, is het ontwerp al meegenomen zodat toekomstige implementatie de routing structuur niet breekt.

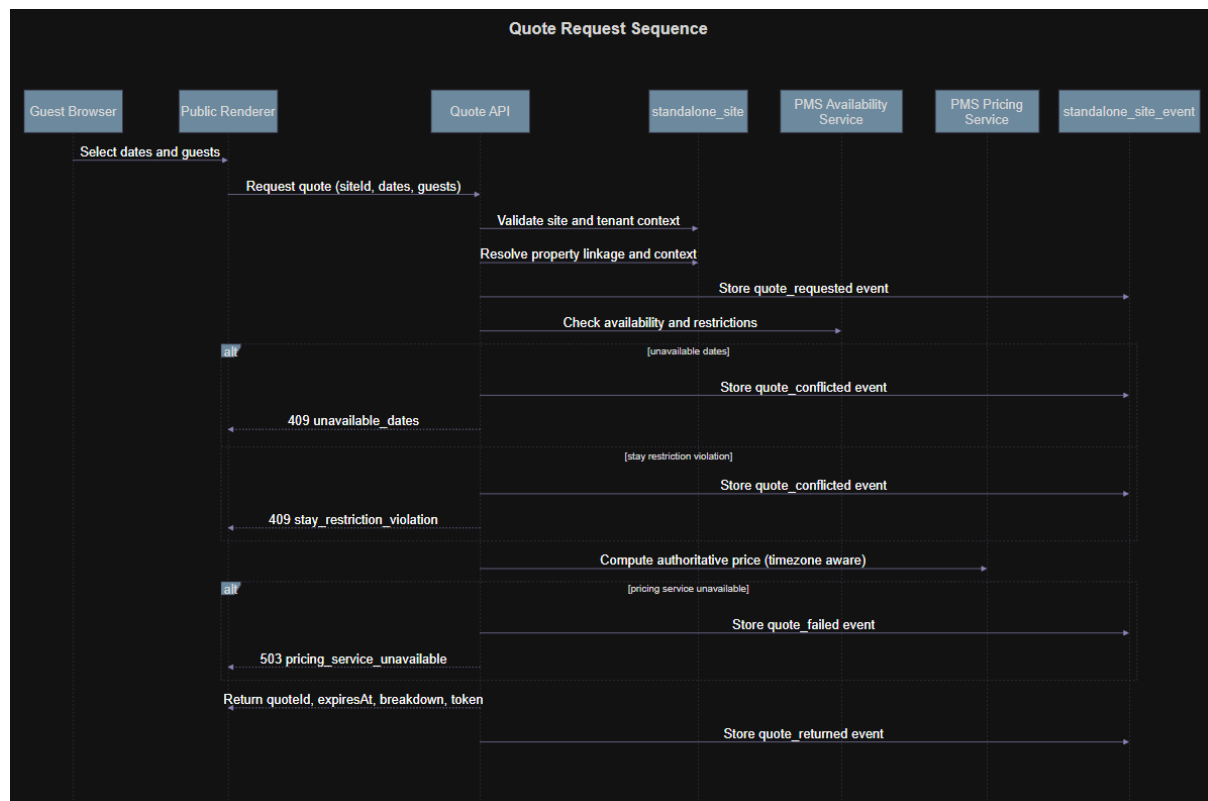


Figuur 13: Domain Status State Diagram

6.5 PMS integratie: bezettingsgraad, prijs en boeking

De Direct Booking Website importeert data vanuit Domits haar PMS, maar niet alle data wordt op dezelfde manier verwerkt. Inhoudelijke content, waaronder titel, beschrijving, voorzieningen en afbeeldingen kunnen worden opgeslagen, terwijl bezettingsgraad en prijs up to date moeten blijven. Dit heeft namelijk directe invloed op de aanvraag en goedkeuring van boekingen. Binnen dit onderdeel maken we een bewuste scheiding tussen statische inhoud en live geïmporteerde data.

Indien een gast de website bezoekt en een verblijfsperiode selecteert, worden de beschikbaarheid en prijzen niet uit het template opgehaald, maar wordt er een aanvraag gestuurd naar Domits haar backend. De stroom wordt hieronder verder toegelicht in de Quote Request Sequence. Allereerst wordt achterhaald bij welke property en website de aanvraag behoort. Vervolgens controleert de Quote API welk PMS gekoppeld is aan de website en wordt de beschikbaarheid vastgesteld door PMS Availability Service en de prijs door PMS Pricing Service.

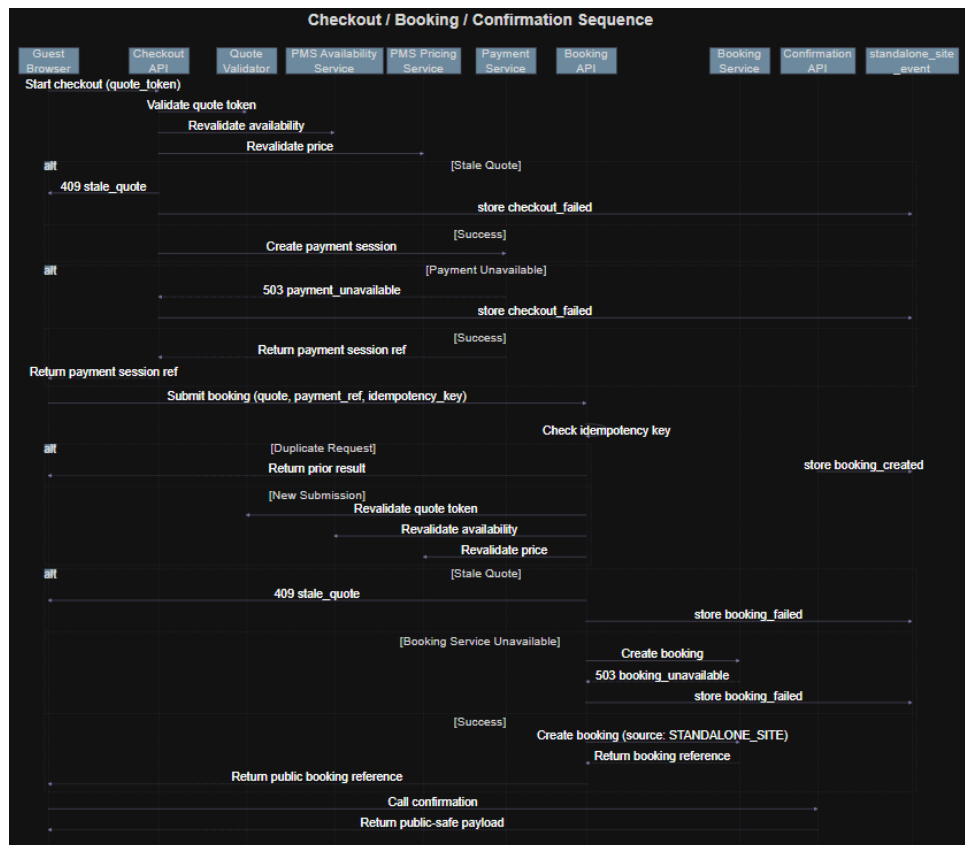


Figuur 14: Quote Request Sequence

Dit ontwerp is van belang omdat specifiek deze data niet betrouwbaar kan worden opgehaald uit eerdere opgeslagen inhoud. Door backend uitlezing worden potentiële foutieve boekingen voorkomen, zoals overlappende boekingen, afwijkend minimum verblijf en asynchrone prijzen.

Boeking, check-out en bevestiging horen in dit ontwerp nog bij een latere fase. De Checkout / Booking / Confirmation Sequence is vooral relevant bij een volgende

implementatiefase. Dit onderstaande diagram toont het verversen van data voordat een boeking definitief verklaard wordt. Voor het huidige ontwerp is een goede basis van belang ter voorbereiding van toekomstige aansluiting van directe boekingen op de gekozen structuur.



Figuur 15: Checkout / Booking / Confirmation Sequence

6.6 Tenant isolatie en security

Waarborging van beveiliging binnen de website is essentieel wanneer boekingen ontvangen kunnen worden. Daarom is het uiterst belangrijk om de structuur en basis goed op te stellen, zodat concrete beveiliging zorgvuldig ingebouwd kan worden.

Voor deze feature is gekozen voor een gedeelte runtime met strikte gebruiker isolatie. Deze keuze sluit aan op de analyse uit [hoofdstuk 5](#), waarin schaalbaarheid en kostenefficiëntie gewaarborgd zijn in plaats van volledige gebruikers scheiding. Dit houdt in dat gebruik altijd server side wordt bepaald en gevalideerd wordt. Hierdoor mag de publieke website niet blindelings vertrouwen op property-id, site-id of andere identiteiten van de cliënt. De juiste website wordt in dit geval gekoppeld op basis van host, domein of preview, die server-side worden uitgelezen, zodat er achterhaald kan worden welke website bij welke host en listing behoort. Hetzelfde geldt voor host-owned data, zoals drafts en gekozen templates.

Ook prijs en bezettingsgraad worden niet cliënt bepaald, maar worden serverside gevalideerd via PMS integratie. Hierdoor voorkomen we browser manipulatie met betrekking tot accommodatie regels en data. Voor dit ontwerp wordt de server aanvraag gecontroleerd via API. Waarbij beveiliging binnen dit ontwerp bestaat uit grenzen tussen publiek getoond data, PMS- en gebruikersdata. Kortom mag een live website nooit eigen waarheid vormen over identiteit, prijs of boeking gerelateerde data.

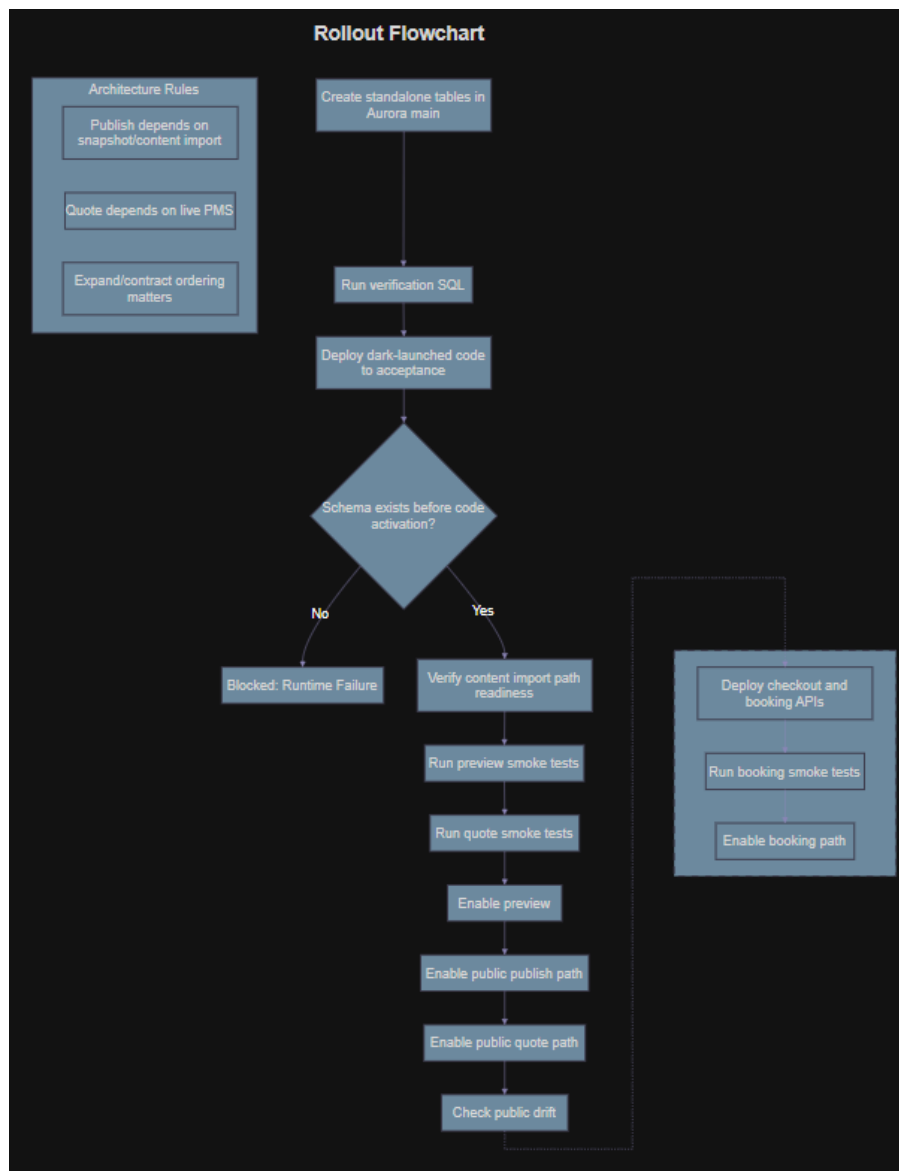
De website gaat veel functionaliteiten hebben, waaronder de host de kans geven om boekingen te ontvangen en de website aan te passen naar wens. Omdat er zoveel bij komt kijken, kunnen er op bepaalde vlakken fouten optreden. Om hierop te kunnen anticiperen is de onderstaande Foutmodus Tabel opgesteld om beveiliging te waarborgen. Deze tabel verduidelijkt de weergave naar de gebruiker, en toelichting voor developers, indien fouten optreden.

Foutmodus Tabel			
Faailmodus	Verwachte fallback	Gebruikersreactie	Logging
Fallback-domein inactief of fout gekoppeld	Tenant niet raden, publieke host blijft onbeschikbaar	Site niet gevonden	Resolve-fout met host en bijbehorend domain
Onbekende host	Tenant niet raden	Site niet gevonden	Resolve-fout met host header
Site geschorst	Publieke render blokkeren	Deze site is tijdelijk niet beschikbaar	Site-status en resolve-log
PMS-import van beschrijvende content mislukt tijdens publiceren of verversen	Vorige gepubliceerde versie actief houden, nieuwe versie niet activeren	Host ziet dat publiceren is mislukt, gasten blijven laatste live versie zien	publish_failed, dependency-fout, propertyId
Gepubliceerde snapshot te oud volgens freshness-drempel	Laatste gepubliceerde snapshot blijven serveren, host waarschuwen	Geen fout voor gast, (optionele) waarschuwing voor host in dashboard	Snapshot-age / freshness-metric
Gepubliceerde site mist content-snapshot	Fail closed op render	Deze site is tijdelijk niet beschikbaar	Render completeness error
Vereist asset ontbreekt of asset-key is ongeldig	Placeholder gebruiken als niet-kritisch; anders fail closed	Visuele degradatie of tijdelijk niet beschikbaar	Asset resolution error met asset-key
Templatecontract komt niet overeen met vastgepinde templateversie	Vorige geldige gepubliceerde versie behouden indien aanwezig; anders fail closed	Deze site is tijdelijk niet beschikbaar	Template resolution / contract validation error
PMS-property is niet langer geschikt of niet meer publiceerbaar terwijl site live is	Pagina zichtbaar houden vanuit laatste snapshot, maar quotes blokkeren	Deze accommodatie is nu niet boekbaar	Eligibility- en quote-dependency-log
Availability service niet beschikbaar	Quote-endpoint geeft 503, pagina blijft renderen	Beschikbaarheid is tijdelijk niet beschikbaar	Dependency error met requestId en siteId
Pricing service niet beschikbaar	Quote-endpoint geeft 503, pagina blijft renderen	Prijsinformatie is tijdelijk niet beschikbaar	Dependency error met requestId en siteId
Read-context ontbreekt of is ongeldig	Read veilig blokkeren	Read is tijdelijk niet beschikbaar	Read context validation error
Verouderde quote	Conflict teruggeven en verversen vereisen	Geselecteerde datums of prijs zijn gewijzigd; vernieuw de quote	quote_conflicted log
Preview-token ongeldig of verlopen	Preview-toegang weigeren, nieuwe preview-link nodig	Preview-link is ongeldig of verlopen	Preview token validation error

Tabel 6: Foutmodus Tabel

6.7 Deployment en rollout

Uit de analyse en het ontwerp kunnen uitmaken dat de Direct Booking Website feature meerdere onderdelen van het platform raakt, waaronder frontend, backend, API Gateway en Aurora SQL. Om breuken in bestaande functionaliteiten te voorkomen is er een specifieke rollout volgorde opgezet die verduidelijkt wordt in het onderstaande Rollout Flowchart. In dit Figuur wordt de configuratie stroom visueel gerepresenteerd.



Figuur 16: Rollout Flowchart

De eerste stap is het creëren van nieuwe tabellen of kolommen binnen Aurora SQL voordat nieuwe code overgedragen wordt naar de development tak. Dit is van belang omdat de backend code afhankelijk is van de configuratie. Mocht de code eerst vervoerd worden voordat configuratie plaatsvindt, kan dit runtime risico's met zich meebrengen, en lopen huidige functionaliteiten gevaar. Hierdoor stellen we schema wijzigingen centraal, waarna pas backend en frontend code volgen.

Na database configuratie verschepen we de backend, waarbij lambda routes, repository logica en validatie op server side niveau aangepast worden. Nadat dit correct functioneert, kan de frontend gebruikmaken van de nieuwe functionaliteiten.

Na deployment volgen er smoke tests, waarbij gecontroleerd wordt of de nieuwe data stromingen werken, zoals draft opslaan, preview openen, editor aanpassen, live preview bijwerken en eerder bestaande gebruiksmogelijkheden die dezelfde backend delen. Vanwege het gebruikmaken van gedeelde infrastructuur is dit een essentieel onderdeel binnen de rollout. Zo kan een simpele fout andere onderdelen van de website raken. Wanneer de tests slagen kan de website preview worden geactiveerd en vervolgens de toevoeging van publishflow. Uitlezing van prijs en bezettingsgraad wordt geconfigureerd in een latere fase, omdat dit afhankelijk is van live PMS validatie. Deze boeking afhandelingen worden bewust buiten de eerste rollout gehouden en behoren tot een toekomstige implementatie fase. Op deze manier blijft onze initiële rollout beheersbaar en wordt voorkomen dat te veel risicovolle functionaliteiten tegelijk live gaan.

7. Feature implementatie

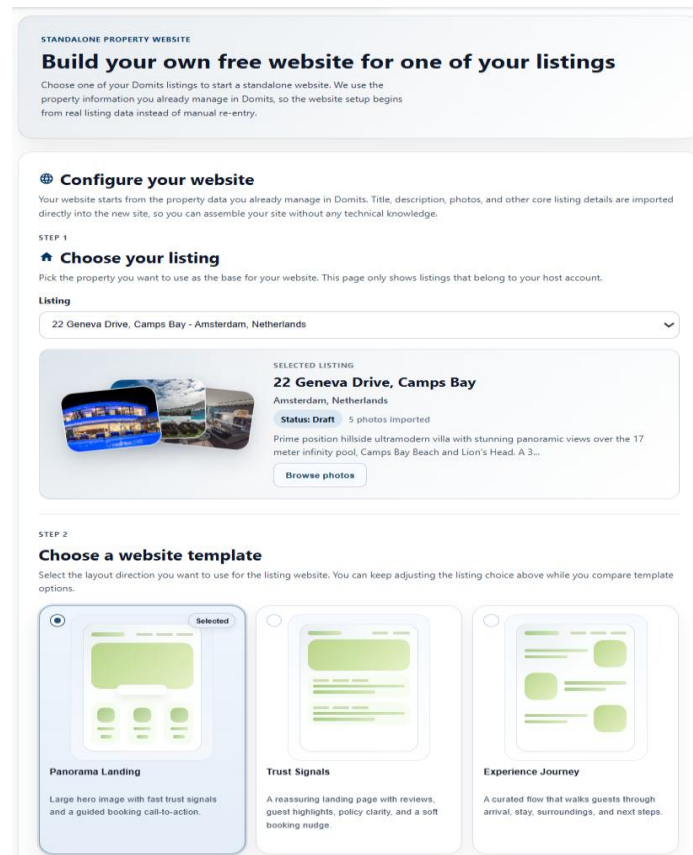
Aan de hand van de verrichtte analyse en het opgesteld ontwerp is de Direct Booking Website iteratief gerealiseerd binnen de bestaande Domits structuur. De eerste implementatie begon bij een noodzakelijk werkend fundament, omdat de feature, onder andere, afhankelijk is van listing data, host-dashboard data, server routes via AWS en opslag in Aurora SQL. Op deze basis zijn meerdere uitbreidingen aangebracht, zodat risico's beperkt bleven.

Wat is gerealiseerd binnen een V1?

Voor de v1 is er een basis gerealiseerd waardoor een host een bestaande Domits listing kan selecteren en hiermee een website kan bouwen, bekijken, opslaan en aanpassingen kan verrichten. Hier waarborgen we dat de Direct Booking Website geen losstaand data systeem vormt, maar voortbouwt op bestaande data uit Domits haar PMS.

Tijdens het bouwen van de website vindt er een uitgebreide detail-fetch plaats. Waarna deze opgehaalde data vertaald wordt naar één gedeelte template structuur, om ervoor te zorgen dat alle data over de listing in het PMS correct overgedragen wordt. En zodat elke template consistent met PMS brondata werkt, zonder dat elke template zijn eigen data interpretatie vanuit het PMS hoeft te implementeren.

Met deze basislaag als fundament is een website builder gerealiseerd binnen het Host Dashboard. De verhuurder kiest binnen deze pagina eerst een van de aangemaakte listings en maakt daarna een keuze uit het template selectiemenu. Momenteel kan de verhuurder kiezen uit drie templates: Panorama Landing, Trust Signals en Experience Journey.



Feature afbeelding 1: Website constructie

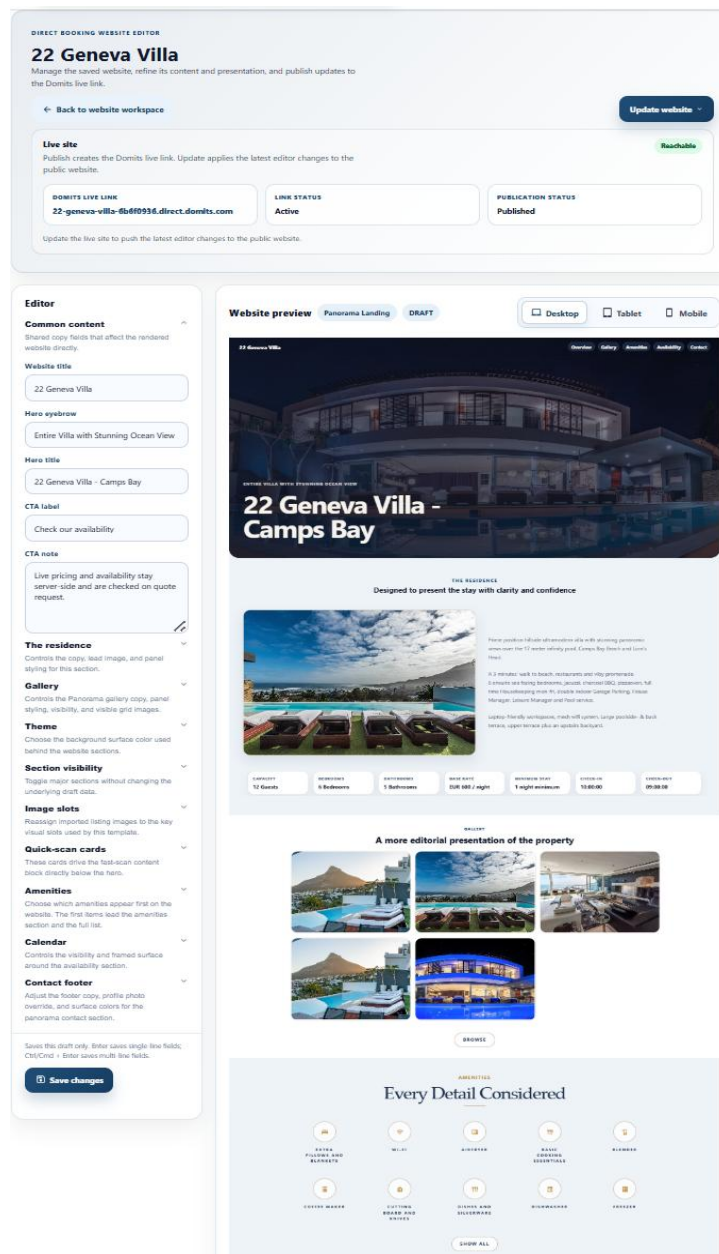
In totaal worden er negen templates gepresenteerd in dit selectie menu, waarbij we de data koppeling centraal hebben gesteld en daarom verder geïtereerd hebben op de koppeling zelf, ten opzichte van uitbreiding van de hoeveelheid templates. Dat betekent dat de overig getoonde opties silhouet-only zijn en als voorbereiding bieden voor toekomstige implementaties. Reden hiervoor is dat de plaatsing van elementen voor visueel professionalisme in deze implementatie tijd minder zwaar weegt dan het goed verlopen en koppelen van PMS data.



Feature afbeelding 2: Website template keuze

Naast website constructie via de builder wordt er een draft opgeslagen, zodat de host gemakkelijk naar de samengestelde website kan navigeren zonder deze opnieuw aan te hoeven maken.

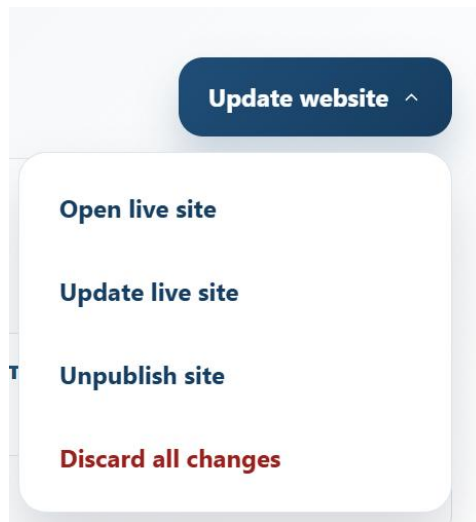
Daarbij kan de website draft geopend worden in een aparte website editor pagina, waar de verhuurder aanpassingen kan verrichten, zoals afbeeldingen, teksten, voorzieningen, secties kan verbergen of tonen, iconen selectie en achtergrondkleur.



Feature afbeelding 3: Website editor

In deze versie is de vrijheid van de host bewust beperkt gehouden, vanwege overmatige complexiteit voor een v1. Er is geen free page builder en geen drag-and-drop systeem toegevoegd. In plaats daarvan biedt de editor een override laag, die zich boven op de vaste template structuur bevindt. Dit betekent dat de host element wijzigingen kan aanbrengen, maar niet de pagina architectuur kan aanpassen.

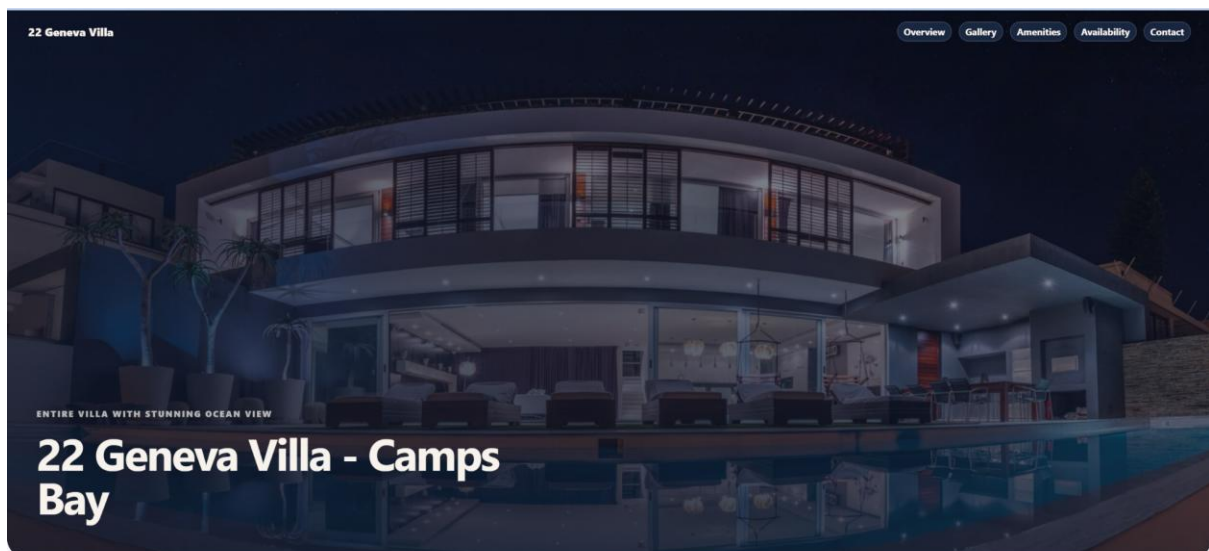
Binnen deze editor is er een scheiding tussen draft- en preview-status gemaakt. Hierdoor blijven wijzigingen in de draft opgeslagen en hebben geen directe impact op de live preview.



Feature afbeelding 4: Website versie beheer

Pas nadat de host kiest voor het updaten van de live preview, wordt deze aangepast. Wijzigen blijven op deze manier geïsoleerd en schaden geen publieke websites.

De live preview is gerealiseerd om de gebouwde website te kunnen tonen buiten het host dashboard. Een publiekelijke versie van de creatie kan hierdoor naar kennissen worden doorgestuurd.



Feature afbeelding 5: Panorama template

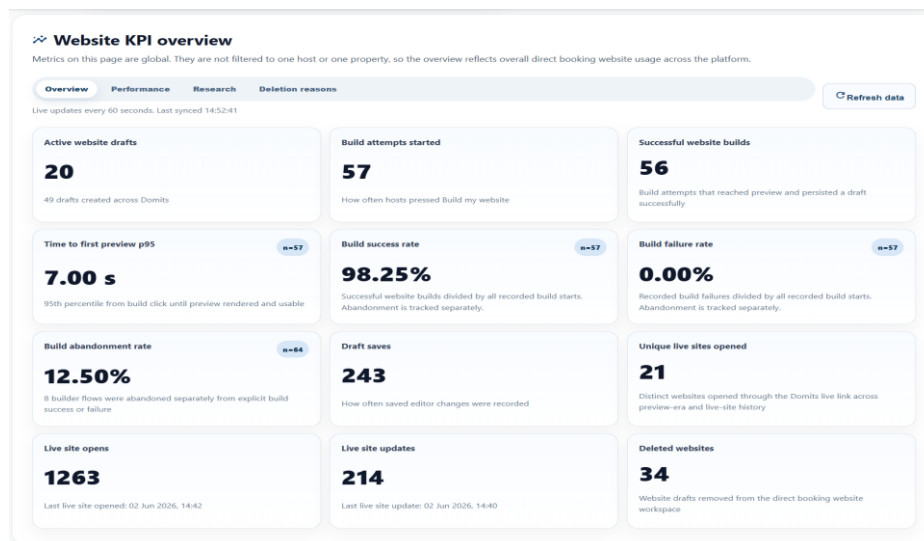
De inhoud van de website beschikt onder andere over de bezettingsgraad van de accommodatie. Deze wordt getoond aan de hand van een iCal-synchronisatie kalender die geblokeerde, geboekte data en externe boekingen via andere

platforms correct weergeeft. Een bezoeker kan nog geen boekingen aanvragen via deze Direct Booking Website versie. Hierbij is het binnen een v2 belangrijk om de kalender slechts informatief te houden en de juistheid van kalenderdata altijd via de server uit te lezen.

Qua kosten is de v1 bewust schaalbaar gehouden. De feature draait op een gedeelde runtime, gebruikt een gedeeld template systeem en hergebruikt Domits haar bestaande infrastructuur. Hierdoor blijven de kosten per extra website laag. Mochten honderden of duizenden websites gemaakt zijn, stijgen zeker de kosten, maar komt dit vooral door gastenverkeer. Dit is logisch, omdat er data opgehaald en opgeslagen wordt via API's, lambda functies en Aurora read/write. Niet vanwege dat websites aparte deployments of frontend nodig hebben. Deze keuze omtrent multi-tenant runtime en gedeeld domein, verdedigt de financiering op grote schaal en op lange termijn.

KPI Validatie

Om inzicht te krijgen over hoe de feature functioneert, zijn er gebruikershandelingen gekoppeld aan database registraties. Hierdoor kunnen we data onderverdelen in grafieken, en meten hoe de geïmplementeerde functionaliteiten worden gebruikt. De huidige metrics geven vooral inzicht over gebruik en richten zich op het proces. Voorbeelden hiervan zijn gemaakte websites, build success, build rate, opgeslagen wijzigingen, aantal geopende preview links, aantal verwijderde websites en de reden waarom hosts een website verwijderen.



KPI's 1: KPI overview

Door data op te vangen en inzichtelijk te maken is de v1 niet alleen functioneel opgesteld, maar ook meetbaar.

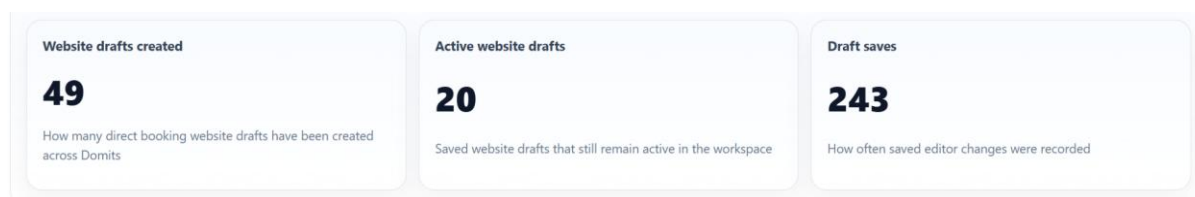
8. Validatie & Evaluatie

In dit hoofdstuk wordt de gerealiseerde Direct Booking Website gevalideerd aan de hand van KPI-data die binnen Domits verzameld is. Het doel van deze validatie is om te beoordelen in hoeverre het gerealiseerde product voldoet aan de eerder vastgestelde eisen op het gebied van gebruikservaring, betrouwbaarheid, schaalbaarheid, kostenefficiënt en technische correctheid. De gebruikte data is afkomstig uit het gerealiseerde KPI dashboard binnen Domits en geven inzicht in zowel het website bouwproces als gebruik van de editor pagina, publicatieflow en live website informatie.

Belangrijk om te weten is dat niet iedere KPI uit het onderzoek al volledig meetbaar is. De huidige gerealiseerde versie bevat nog geen volledige end-to-end bookingflow met externe boeking aanvragen en afhandeling hiervan. Daardoor zijn KPI's die gekoppeld zijn aan de boeking API en completion funnel nog niet meetbaar. Dit betekent dat validatie in deze fase zich vooral richt op kwaliteit omtrent website-builder, editor, publicatieflow en live website runtime.

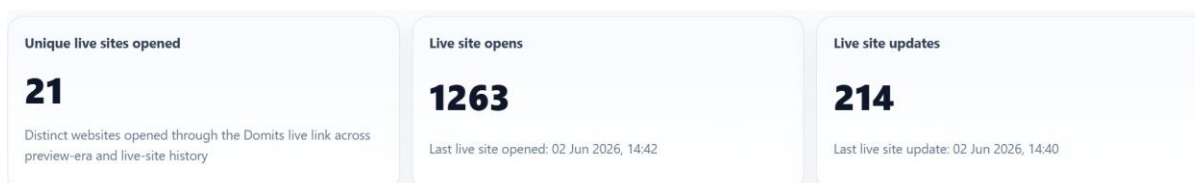
8.1 KPI validatie

De eerste groep KPI's richt zich op de vraag of de feature in de praktijk daadwerkelijk gebruikt wordt. Binnen het dashboard zijn 49 website drafts aangemaakt, waarvan op het meetmoment 20 actieve drafts overbleven. Daarnaast zijn 243 draft opgeslagen wijzigingen geregistreerd. Dit laat zien dat de feature niet alleen geopend wordt, maar ook actief gebruikt en aangepast wordt. De editor wordt dus niet alleen als een eenmalige builder gebruikt, maar ook als iteratie omgeving.



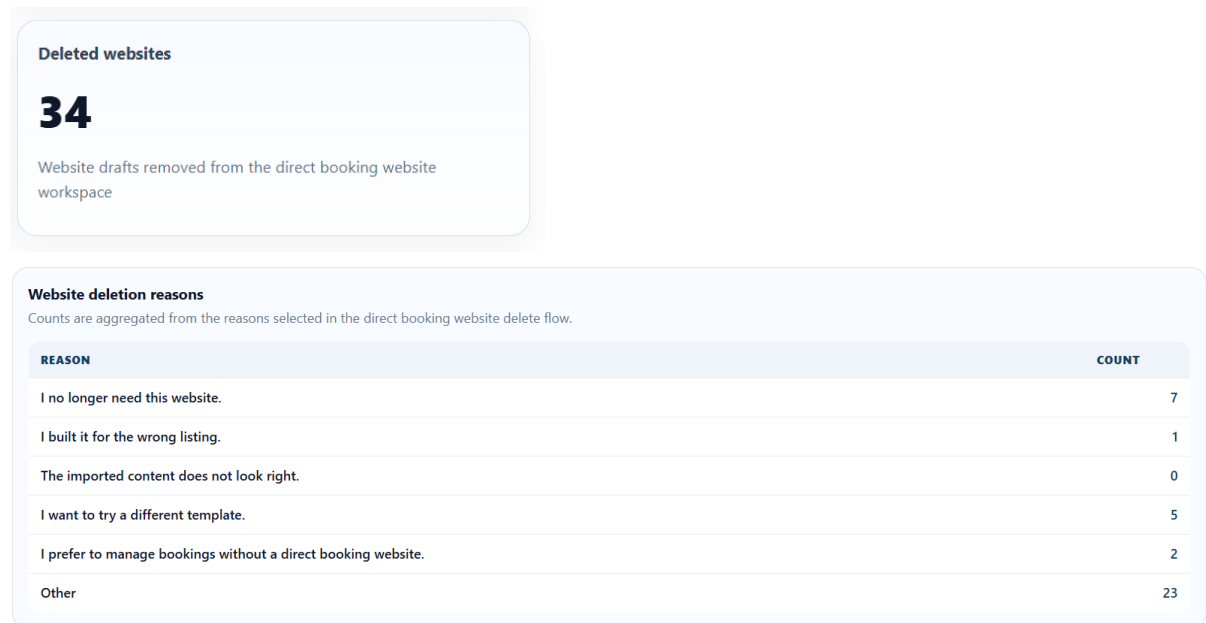
KPI's 2: Praktijk metingen

De live website toont 21 unieke live geopende websites, met totaal 1263 website bezoeken en 214 live website synchronisaties. Hiermee wordt aangetoond dat de feature niet alleen intern gebruikt wordt, maar ook dat er externe bezoeken, aanpassingen en publicaties plaatsvinden.



KPI's 3: Live website activiteit

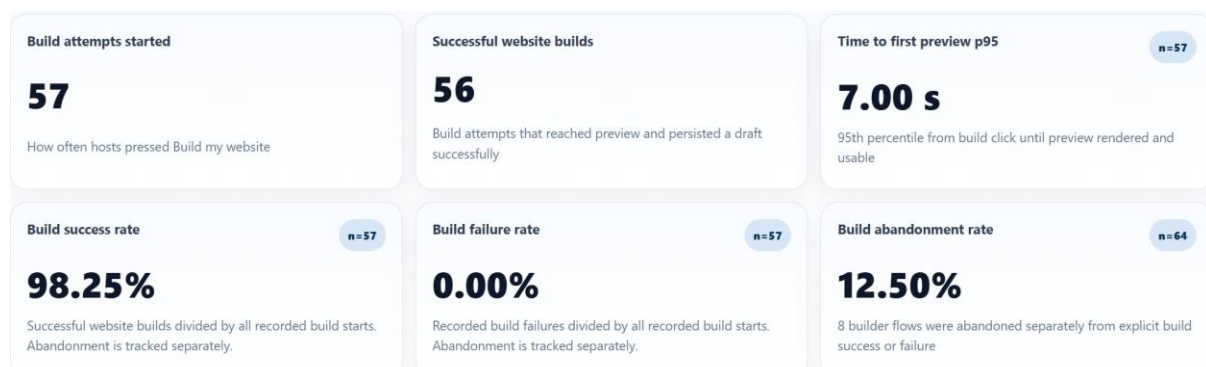
De meeting van 34 verwijderde websites komt grotendeels voort uit test scenario's, en toont aan dat het de gehele feature loop werkt. In de toekomst kan dit extra inzicht geven op gebied van gedrag en mogelijke afhaakmomenten.



KPI's 4: Website verwijdering gerelateerde data

Een tweede belangrijke categorie focust op technische haalbaarheid en betrouwbaarheid van het bouwproces. binnen de gemeten periode zijn 57 bouw pogingen geregistreerd, waarvan 56 succesvol gebouwd zijn. Dit resulteert in een succesratio van 98,25%. De gefaalde bouw ratio staat op 0,00%, wat betekent dat er geen technische builds mislukt zijn.

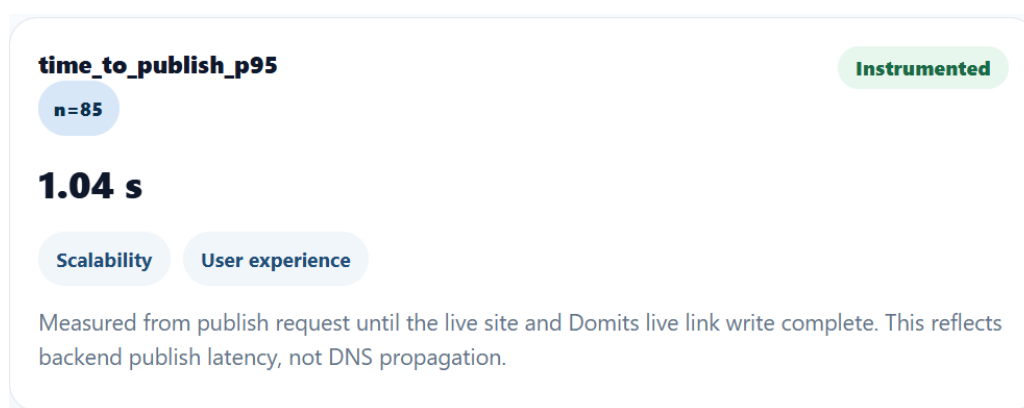
Het verschil tussen een succes ratio en de gemeten 98,25% wordt verklaard door waargenomen verlating van de website binnen een gestarte poging om een website te bouwen. Deze abandonment ratio bedraagt 12,50%, waarbij 8 builder bouw pogingen niet zijn afgemaakt, grotendeels door testscenario's. Dit duidt op technische laagdrempeligheid, maar ook dat de UX- of oriëntatie-gedrag nog niet helemaal frictieloos is.



KPI's 5: Technische haalbaarheid

Voor de snelheid van de builder is gekeken naar de `time_to_first_preview_p95` KPI. Deze toont een snelheid van 7,00 seconden over 57 metingen. Dit betekent dat in 95% van de gemeten gevallen vanaf het punt van website bouw tot een beschikbare en bruikbare preview onder de 7 seconden bleef. Voor een proces waarin listing data eerst wordt opgehaald, vertaald wordt naar een gedeeld template model en vervolgens als preview wordt opgebouwd en opgeslagen, is dit een bruikbare uitkomst. Het ondersteunt de aanname dat de one-click builder binnen de gestelde scope snel genoeg is om aan te bieden aan platform gebruikers.

Ook het publicatieproces is gemeten. De KPI `time_to_publish` meet 1,04 seconden bij een aantal van 85. Deze meting heeft te maken met de publicatie vertraging vanaf verzoek tot publicatie tot moment waarop de live website en live link zijn bijgewerkt.



KPI's 6: Publicatieduur

Deze waarneming ondersteunt de technische haalbaarheid van de gekozen publicatiestrategie: de website wordt niet als losse deployment behandeld, maar als een publicatie van een reeds bestaand multi-tenant model. Dit resultaat sluit goed aan op de ontwerpkeuze om kosten en complexiteit laag te houden.

Voor prestatie van de publieke website is gekeken naar de Largest Contentful Paint (LCP). De gemeten waarden zijn:

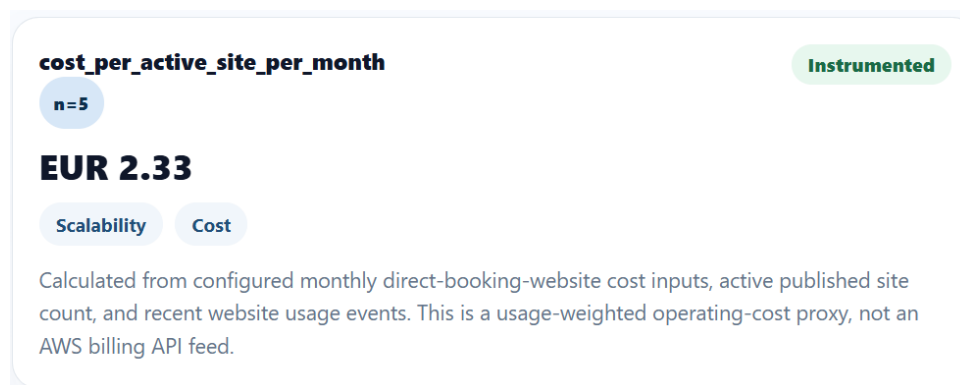
Desktop n=737	Tablet n=12	Mobile n=40
3,01 seconden	3,04 seconden	2,59 seconden

De metingen laten zien dat live website runtime bijspijking nodig heeft volgens Google's Core Web Vitals guidelines [53].

Good	Needs improvement	Poor
under 2.5 seconds	2,5 - 4.0 seconds	above 4.0 seconds

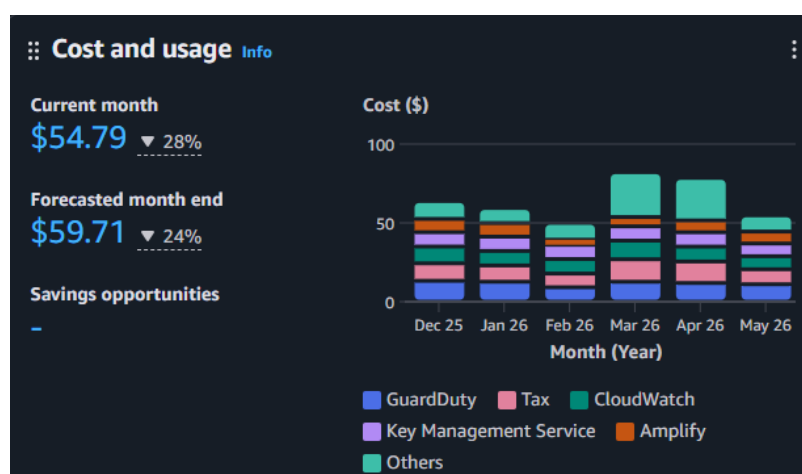
Dit toont dat deze feature aanpak een solide basis heeft, maar dat verdere optimalisatie waardevol is. Voorbeelden hiervan zijn verdere beeldoptimalisatie, render prioriteit van de hero sectie en het nog verder beperken van niet-kritische frontend bij eerste render.

Om kosten en schaalbaarheid te meten is de `cost_per_active_site_per_month` KPI gerealiseerd. De huidige meting bedraagt EUR 2,33 bij 5 aantallen. Deze waarde is berekend op basis van geconfigureerde maandelijks Direct Booking Website kosten, het aantal actieve gepubliceerde websites en recente usage events. Deze KPI betreft dus een usage gebaseerde kostenberekening en geen directe AWS data export. Toch is de KPI relevant, omdat deze laat zien dat de gekozen multi-tenant structuur in de huidige fase zorgt voor lage operationele kosten per actieve website. Indien meer data is verzameld binnen AWS over tijd omtrent de Direct Booking Website, kan de KPI nauwkeuriger worden geconfigureerd.



KPI's 7: Kosten per active website

Daarnaast is binnen AWS (cost and usage) nauwelijks verandering te zien na het introduceren van deze feature, wat een zeer goed teken is.



KPI's 8: Geregistreerde kosten binnen AWS

Ten slotte is voor de bereikbaarheid van de live websites gekeken naar de `fallback_subdomain_availability` KPI. Deze meting staat momenteel op 100.00% bij een aantal van 5. Deze KPI is gebaseerd op de gepubliceerde live site plus een active routing status naar een sub link. Dit toont aan dat alle websites een actief domein en een terugval domein hebben.

KPI's zoals `booking_api_error_rate` en `booking_funnel_completion_rate` staan bewust op pending, omdat de bijbehorende boeking flows tot een volgende implementatiefase behoort.

Op basis van deze KPI's kan worden geconcludeerd dat de gerealiseerde feature in deze fase valideert op daadwerkelijk gebouwde onderdelen. De builderflow is technisch stabiel, de publicatieflow is snel, de live runtime is bereikbaar en de kosten per actieve website zijn laag. Naast dat de booking KPI's geconfigureerd moeten worden, is het ook verstandig om de performance van de publieke website te verbeteren, zodat deze sneller te zien is voor bezoekers. Daarbij laten de abandonment rate en de nog niet gekoppelde boeking KPI's zien dat validatie nog niet volledig is afgerond voor een uiteindelijke v2 of v3, waarin deze flow end-to-end actief hoort te zijn.

Om meer inzicht te kunnen krijgen over de Direct Booking Website Feature en om nog steviger te kunnen valideren, is het van belang om over een bepaalde tijd weer een blik te werpen naar de KPI's, vanwege het hebben van een grotere hoeveelheid data naarmate meer personen de feature gebruiken.

8.2 Validatie van scenario's en interpretatie

Naast de losse KPI waarden is ook gekeken naar onderliggende scenario's die de cijfers vertegenwoordigen. Het eerste scenario is gericht op het bouwen van de website vanuit een bestaande listing binnen Domits. De combinatie van 57 bouw pogingen, 56 succesvol gebouwde sites en een `time_to_first_preview_p95` van 7,00 seconden toont aan dat het kernproces in de praktijk werkt. Hosts kunnen dus daadwerkelijk vanuit bestaande PMS data een website genereren en tot een bruikbare versie kunnen komen zonder te complexe stappen.

Het tweede scenario gaat over het beheren en itereren van aangemaakte websites. De 243 draft opgeslagen wijzigingen en 214 live site updates laten zien dat wijzigingen niet alleen mogelijk zijn, maar ook gebruikt worden. Hierdoor kunnen we constateren dat de editor niet alleen als demonstratie gebruikt wordt, maar ook dient als een iteratieve omgeving en werkbare beheerlaag.

Het derde scenario betreft het publiceren en live beschikbaar stellen van de website. De gemeten `time_to_publish_p95` van 1,04 seconden en de `fallback_subdomein_availability` van 100,00% tonen aan dat dit scenario technisch

betrouwbaar verloopt binnen de huidige v1 scope. Het aantal bezoeken van de live site bevestigt bovendien dat gepubliceerde websites niet alleen gepubliceerd worden, maar ook bezocht worden.

Het vierde scenario duidt op performance van de website op verschillende apparaten. De Largest Contentful Paint metingen van 3,01 op desktop, 3,04 op tablet en 2,59 op mobiel tonen aan dat de gerealiseerde website in de goede richting zit, maar extra inzicht kan gebruiken op gebied van laadtijd. Wel is dit al een sterk resultaat voor een feature die zich eigenlijk nog steeds bevindt in een startende fase.

Het vijfde scenario gaat over de correctheid van data uit het PMS. De huidige gerealiseerde versie ondersteunt al de koppeling met beschikbaarheid en prijs data, maar de validatie is op dit punt nog gedeeltelijk. De bezettingsgraad is informatief en gekoppeld aan PMS- en externe kalenderdata, terwijl boeking afhandeling nog niet actief is. Daardoor zijn metriecken van correctheid op boekingen nog niet volledig te meten.

8.3 Conclusie van de validatie

De KPI data laat zien dat de Direct Booking Website binnen Domits in de huidige fase succesvol gevalideerd is als technisch fundament. De feature wordt daadwerkelijk gebruikt, websites worden gebouwd, opgeslagen, aangepast, gepubliceerd en geopend. De builder- en publicatiefow presteren snel en stabiel, de live runtime is betrouwbaar bereikbaar en de kosten per activee website blijven laag. Ook de performance op desktop, tablet en mobiel laat zien dat de gekozen architectuur geschikt is voor publieke weergave. Wel is het verstandig om de laadsnelheid van de website te verbeteren, zodat dit binnen de Google's Core Web Vitals guidelines [53].

De validatie laat tegelijkertijd duidelijk zien waar de grens van de huidige versie ligt. KPI's met betrekking tot de volledige bookingflow zijn nog niet beschikbaar, omdat deze stroming nog niet live is. Daarbij ondersteunt dit hoofdstuk de conclusie dat de huidige realisatie een geslaagde en meetbare v1/vroege v2 basis vormt, maar dat een volgende iteratie zeker nodig is om ook prijs, boekingcorrectheid en funnel completion volledig te onderbouwen.

9. Conclusie

Dit onderzoek heeft als doel te bepalen hoe Domits een template gebaseerde, on-click Direct Booking Website kan ontwerpen en realiseren die schaalbaar, veilig en kostenefficiënt is, en tegelijkertijd accuraat blijft aansluiten op data uit het huidige Property Management System. Op basis van de uitgevoerde analyse, de vergelijking van aanpak mogelijkheden, het opgestelde ontwerp en de gerealiseerde implementatie kan geconcludeerd worden dat dit voor Domits haalbaar is, mits er gekozen wordt voor een beheerbare architectuur en een iteratief product aanpak.

Uit het onderzoek blijkt dat een eerste versie niet hoeft te bestaan uit een volledig vrij in te delen websitebouwer of een complete end-to-end boeking mogelijkheid om al waardevol te zijn. Voor een v1 is juist een beperkte maar bruikbare scope het meest passend. De grootste waarde voor hosts ligt in het snel kunnen publiceren van een professionele website die gekoppeld is aan de bestaande listing data, visueel aanpasbaar is, en waarin essentiële informatie, zoals titel, beschrijving, afbeeldingen, voorzieningen, contactinformatie en bezettingsgraad correct worden getoond. Daarmee ontstaat voor verhuurders een directe boeking mogelijkheid die onafhankelijk is van externe platformen, zonder dat de verhuurder zelf technisch beheer of implementaties met code hoeft uit te voeren.

De meest geschikte architectuur voor Domits is daarbij een vastgestelde template aanpak met beperkte configuratie door middel van secties en design tokens, in plaats van een vrije websitebouwer. Deze keuze maakt het mogelijk om meerdere templates binnen een gedeelde code omgeving/base te beheren, terwijl technische complexiteit, validatie en backwards compatibility beheersbaar blijven. Dit sluit aan op de behoefte van Domits om snel waarde te kunnen leveren en tegelijkertijd ruimte te houden voor toekomstige uitbreiding. Voor hosting blijkt een hybride aanpak het meest geschikt: beschrijvende en statische informatie kan via snapshots of vastgestelde structuren snel en efficiënt worden geleverd, terwijl veranderlijke data, zoals beschikbaarheid, prijs en boekingsinformatie server-side gevalideerd uitgelezen moeten worden.

Voor multi tenancy en domeinbeheer is geconcludeerd dat een gedeelde runtime met een strenge server-side tenant isolatie voor Domits de beste balans biedt tussen schaalbaarheid, kostenefficiëntie en veiligheid. Een volledig geïsoleerde infrastructuur per host zou ongetwijfeld meer scheiding bieden, maar past niet bij de huidige doelstelling van Domits om deze feature snel en betaalbaar op te kunnen schalen. In combinatie met herkenning op basis van host of domeinnaam, automatisch gegenereerd terugval subdomein als standaard publieke route en centrale evaluatie via de server, ontstaat een model dat goed beheersbaar blijft en voldoende bescherming biedt tegen datalekken en website misbruik.

De gerealiseerde feature bevestigt dat de ontwerpkeuzes in de praktijk bruikbaar zijn. Binnen Domits is een werkende Direct Booking Website feature gecreëerd,

waarbij hosts een bestaande listing kunnen selecteren, een website layout kunnen kiezen, automatisch hun accommodatie data kunnen laten vertalen naar een gedeeld template model, wijzigingen binnen de editor kunnen opslaan, een voorvertoning kunnen zien en de website kunnen publiceren. Hiermee is een technisch fundament gerealiseerd dat bruikbaar is voor hosts en logisch door te ontwikkelen is naar een volgende feature versie.

De validatie met KPI's en gebruiksscenario's laat zien dat de huidige oplossing in deze fase overtuigend functioneert als v1/vroege v2 basis. Het bouw- en publicatieproces is snel en stabiel, websites worden daadwerkelijk aangemaakt, aangepast en geopend, terwijl kosten per actieve website laag blijven dankzij de gedeelde runtime en het gedeelde template systeem. Tegelijkertijd laat de evaluatie ook zien waar de grens ligt. De feature ondersteunt nog geen volledige end-to-end booking stroom met quote, checkout, betaalverzoek en voltooiing. Daardoor zijn sommige KPI's nog gedeeltelijk meetbaar. Ook blijft verdere optimalisatie op het gebied van performance, boeking correctheid en custom domein koppeling een wens.

Op basis hiervan kan de hoofdvraag worden beantwoord: Domits kan een template gebaseerde on-click Direct Booking Website realiseren die schaalbaar, veilig en kostenefficiënt is en correct aansluit op het bestaande PMS, mits gekozen wordt voor een beheerbare template architectuur, hybride datakoppeling, een gedeelde runtime met strenge tenant isolatie en een geïntegreerde ontwikkeling waarin eerst de kernwaarde voor hosts wordt gerealiseerd en complexere onderdelen, zoals de volledige boeking mogelijkheid en custom domains, in een latere implementatiefase worden toegevoegd. Daarmee vormt dit onderzoek niet alleen een onderbouwing van de gemaakte ontwerpkeuzes, maar ook een bruikbaar fundament voor toekomstige realisering van de Direct Booking Website binnen Domits.

Bronnenlijst (IEEE)

Bronnen

- [1] Nielsen Norman Group, "NN/g Lean UX & Agile Glossary," 2024. [Online]. Available: https://media.nngroup.com/media/articles/attachments/NNg_Lean_UX_%26_Agile_Glossary.pdf
- [2] L. Wang, R. Law, B. D. Guillet, K. Hung, and D. K. C. Fong, "Impact of hotel website quality on online booking intentions: eTrust as a mediator," *International Journal of Hospitality Management*, vol. 47, pp. 108-115, 2015. doi: 10.1016/j.ijhm.2015.03.012. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0278431915000493>
- [3] S. Y. Kim, J. U. Kim, and S. C. Park, "The Effects of Perceived Value, Website Trust and Hotel Trust on Online Hotel Booking Intention," *Sustainability*, vol. 9, no. 12, p. 2262, 2017. doi: 10.3390/su9122262. Available: <https://www.mdpi.com/2071-1050/9/12/2262>
- [4] M. Amin, K. Ryu, C. Cobanoglu, and A. Nizam, "Determinants of online hotel booking intentions: website quality, social presence, affective commitment, and e-trust," *Journal of Hospitality Marketing & Management*, vol. 30, no. 7, pp. 845-870, 2021. doi: 10.1080/19368623.2021.1899095. Available: <https://www.tandfonline.com/doi/full/10.1080/19368623.2021.1899095>
- [5] Ghost Foundation, "Custom Settings," Ghost Developer Docs. [Online]. Available: <https://docs.ghost.org/themes/custom-settings>
- [6] Ghost Foundation, "Custom Settings," Ghost Developer Docs, sectie over breaking changes bij setting keys. [Online]. Available: <https://docs.ghost.org/themes/custom-settings>
- [7] Shopify, "JSON templates," Shopify Developer Documentation. [Online]. Available: <https://shopify.dev/docs/storefronts/themes/architecture/templates/json-templates>
- [8] Shopify, "Blocks," Shopify Developer Documentation. [Online]. Available: <https://shopify.dev/docs/storefronts/themes/architecture/blocks>
- [9] WordPress.org, "Block themes," WordPress Documentation. [Online]. Available: <https://wordpress.org/documentation/article/block-themes/>
- [10] WordPress.org, "Site Editor Pages," WordPress Documentation. [Online]. Available: <https://wordpress.org/documentation/article/site-editor-pages/>

- [11] Webflow, "Components overview," Webflow Help Center. [Online]. Available: <https://help.webflow.com/hc/en-us/articles/33961303934611-Components-overview>
- [12] Wix, "Wix Editor: Generating Page Sections with AI," Wix Help Center. [Online]. Available: <https://support.wix.com/en/article/wix-editor-generating-page-sections-with-ai>
- [13] Lodgify website builder: <https://www.lodgify.com/vacation-rental-website-builder/>
- [14] Lodgify showcase/blog: <https://www.lodgify.com/blog/beautiful-lodgify-vacation-rental-websites/>
- [15] Guesty direct reservations: <https://www.guesty.com/features/direct-reservations/>
- [16] Hostaway direct booking: <https://get.hostaway.com/direct-booking/>
- [17] Smoobu website builder: <https://www.smoobu.com/en/website-builder-vacation-rentals/>
- [18] Wix, "Wix Editor: Customizing Your Sections," Wix Help Center. [Online]. Available: <https://support.wix.com/en/article/wix-editor-customizing-your-sections>
- [19] Wix, "Studio Editor: Using Global Sections," Wix Help Center. [Online]. Available: <https://support.wix.com/en/article/studio-editor-using-global-sections>
- [20] WordPress.org, "Block themes," WordPress Documentation. [Online]. Available: <https://wordpress.org/documentation/article/block-themes/>
- [21] WordPress.org, "Site Editor Pages," WordPress Documentation. [Online]. Available: <https://wordpress.org/documentation/article/site-editor-pages/>
- [22] Webflow, "Components overview," Webflow Help Center. [Online]. Available: <https://help.webflow.com/hc/en-us/articles/33961303934611-Components-overview>
- [23] Wix, "Wix Editor: Generating Page Sections with AI," Wix Help Center. [Online]. Available: <https://support.wix.com/en/article/wix-editor-generating-page-sections-with-ai>
- [24] Amazon Web Services, "Hosting a static website using Amazon S3," Amazon S3 Documentation. [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/dev/WebsiteHosting.html>
- [25] Amazon Web Services, "Host a Static Website on AWS," AWS Getting Started. [Online]. Available: <https://aws.amazon.com/getting-started/hands-on/host-static-website/>
- [26] Vercel, "Automatic Static Optimization," Next.js Documentation. [Online]. Available: <https://nextjs.org/docs/pages/building-your-application/rendering/automatic-static-optimization>

- [27] Cloudflare, "Cloudflare Pages," Cloudflare Documentation. [Online]. Available: <https://developers.cloudflare.com/pages/>
- [28] Cloudflare, "Functions," Cloudflare Pages Documentation. [Online]. Available: <https://developers.cloudflare.com/pages/functions/>
- [29] Amazon Web Services, "Pool isolation," AWS Well-Architected SaaS Lens. [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/pool-isolation.html>
- [30] Amazon Web Services, "Full stack isolation," AWS Well-Architected SaaS Lens. [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/full-stack-isolation.html>
- [31] Amazon Web Services, "Tier-based isolation," AWS Well-Architected SaaS Lens. [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/tier-based-isolation.html>
- [32] Amazon Web Services, "Enforcing tenant isolation," AWS Prescriptive Guidance. [Online]. Available: <https://docs.aws.amazon.com/prescriptive-guidance/latest/agent-ai-multitenant/enforcing-tenant-isolation.html>
- [33] Amazon Web Services, "Creating a public hosted zone," Amazon Route 53 Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/CreatingHostedZone.html>
- [34] Amazon Web Services, "Creating a subdomain that uses Amazon Route 53 as the DNS service without migrating the parent domain," Amazon Route 53 Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/CreatingNewSubdomain.html>
- [35] Amazon Web Services, "Use custom URLs by adding alternate domain names (CNAMEs)," Amazon CloudFront Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/CNAMEs.html>
- [36] Amazon Web Services, "Use wildcards in alternate domain names," Amazon CloudFront Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/alternate-domain-names-wildcard.html>
- [37] Amazon Web Services, "Choosing between alias and non-alias records," Amazon Route 53 Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resource-record-sets-choosing-alias-non-alias.html>

- [38] Vercel, "Caching and Revalidating," Next.js Documentation. [Online]. Available: <https://nextjs.org/docs/app/getting-started/caching-and-revalidating>
- [39] Amazon Web Services, "Manage how long content stays in the cache (expiration)," Amazon CloudFront Developer Guide. [Online]. Available: <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Expiration.html>
- [40] Amazon Web Services, "What Is Amazon EventBridge?," Amazon EventBridge User Guide. [Online]. Available: <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>
- [41] Amazon Web Services, "Amazon EventBridge," AWS Prescriptive Guidance. [Online]. Available: <https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-integrating-microservices/eventbridge.html>
- [42] Amazon Web Services, "Tenant isolation - SaaS Architecture Fundamentals," AWS Whitepapers. [Online]. Available: <https://docs.aws.amazon.com/whitepapers/latest/saas-architecture-fundamentals/tenant-isolation.html>
- [43] Amazon Web Services, "Multi-tenant SaaS authorization and API access control: Implementation options and best practices," AWS Prescriptive Guidance. [Online]. Available: <https://docs.aws.amazon.com/prescriptive-guidance/latest/saas-multitenant-api-access-authorization/introduction.html>
- [44] OWASP, "API1:2023 Broken Object Level Authorization," OWASP API Security Top 10. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>
- [45] OWASP, "API4:2023 Unrestricted Resource Consumption," OWASP API Security Top 10. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/>
- [46] OWASP, "Cross Site Scripting Prevention Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- [47] OWASP, "Content Security Policy Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
- [48] European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation), Article 5," EUR-Lex. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2016/679/2016-05-04/eng>

[49] N. Verhoeven, “De ‘juiste’ vragen stellen,” *Nel Verhoeven*. [Online]. Available: https://www.nelverhoeven.nl/actueel/209-385_De-juiste-vragen-stellen

[50] P. Walton, “Best practices for measuring Web Vitals in the field,” *web.dev*, Apr. 19, 2021. Available: <https://web.dev/articles/vitals-field-measurement-best-practices>

[51] P. Walton, “Web Vitals,” *web.dev*, May 4, 2020. Available: <https://web.dev/articles/vitals>

[52] Amazon Web Services, “CloudWatch statistics definitions,” *Amazon CloudWatch Documentation*. <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Statistics-definitions.html>

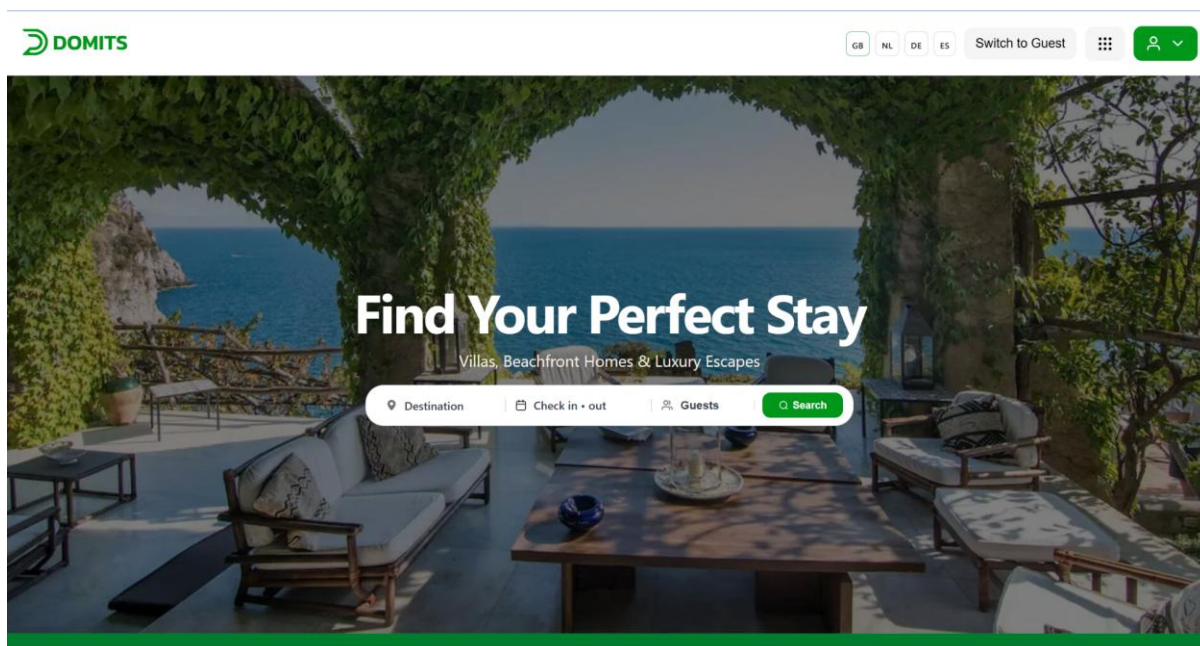
[53] Google, “Core Web Vitals report,” *Search Console Help*. [Online]. Available: <https://support.google.com/webmasters/answer/9205520>

Bijlagen

In de bijlagen zijn ondersteunende materialen geplaatst die verdieping geven op het onderzoek en de gerealiseerde feature.

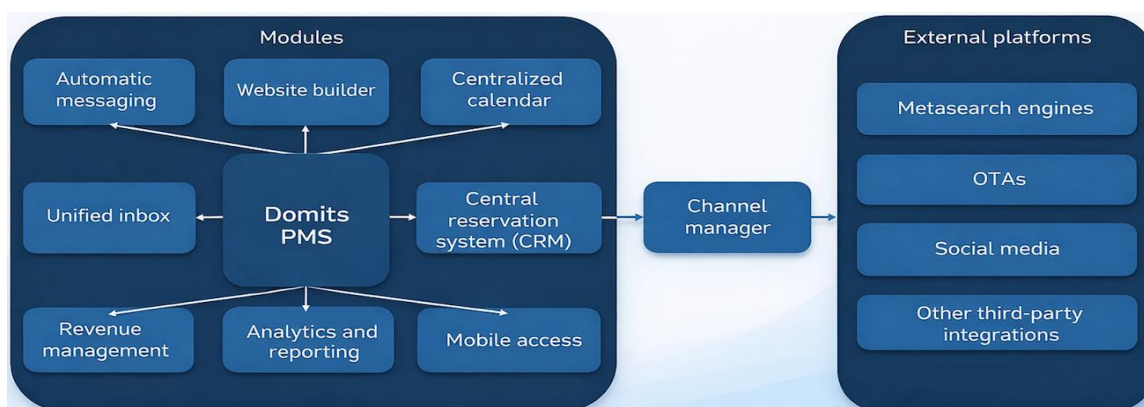
Figuren

Een visuele representatie van het Domits platform, waar gebruikers accommodaties kunnen boeken, zoals een Airbnb of Booking.com.



Figuur 1: Domits product

Diagram die de verschillende modules binnen Domits visueel representeert en hoe de Direct Booking Website in verbinding staat met het PMS.



Figuur 2: Positie van de Website Builder binnen Domits, als extra direct boekingskanaal bovenop bestaande PMS data.

[illegible]

Figuur 5 hieronder toont een voorbeeld van de gestructureerde context binnen Obsidian, waarin zichtbaar is hoe notities over business context, technische afspraken, voortgang en werkwijzen met elkaar in verbinding staan.



88

Figuur 6 hieronder toont een voorbeeld van steeds meer toegevoegde context binnen Obsidian, waarin notitie samenhang zichtbaar is.



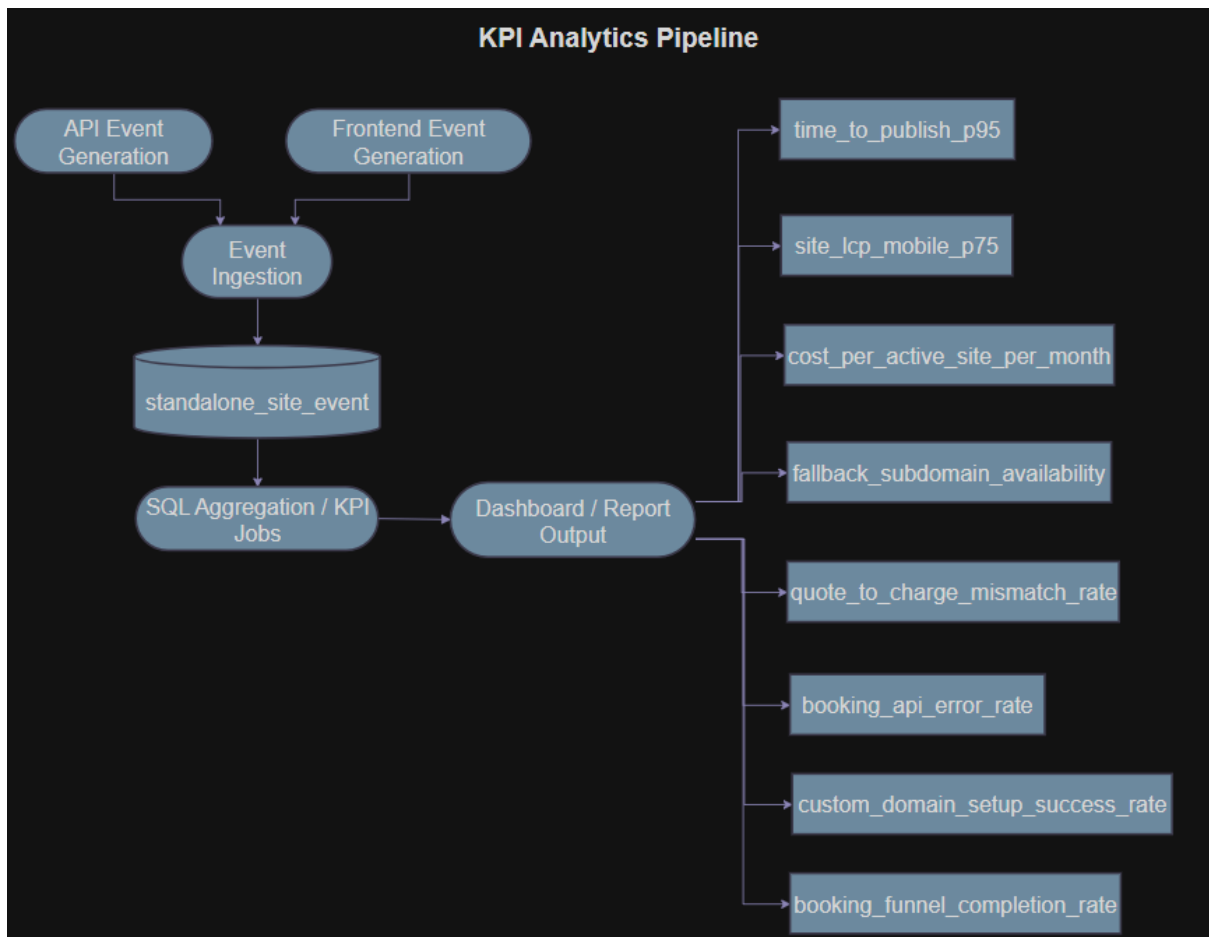
Figuur 6: Weergeeft de verbetering van meer verzamelde notities en context binnen het Obsidian tweede brein.

Dit diagram is opgesteld om de verschillen tussen de drie strategieën te verduidelijken.



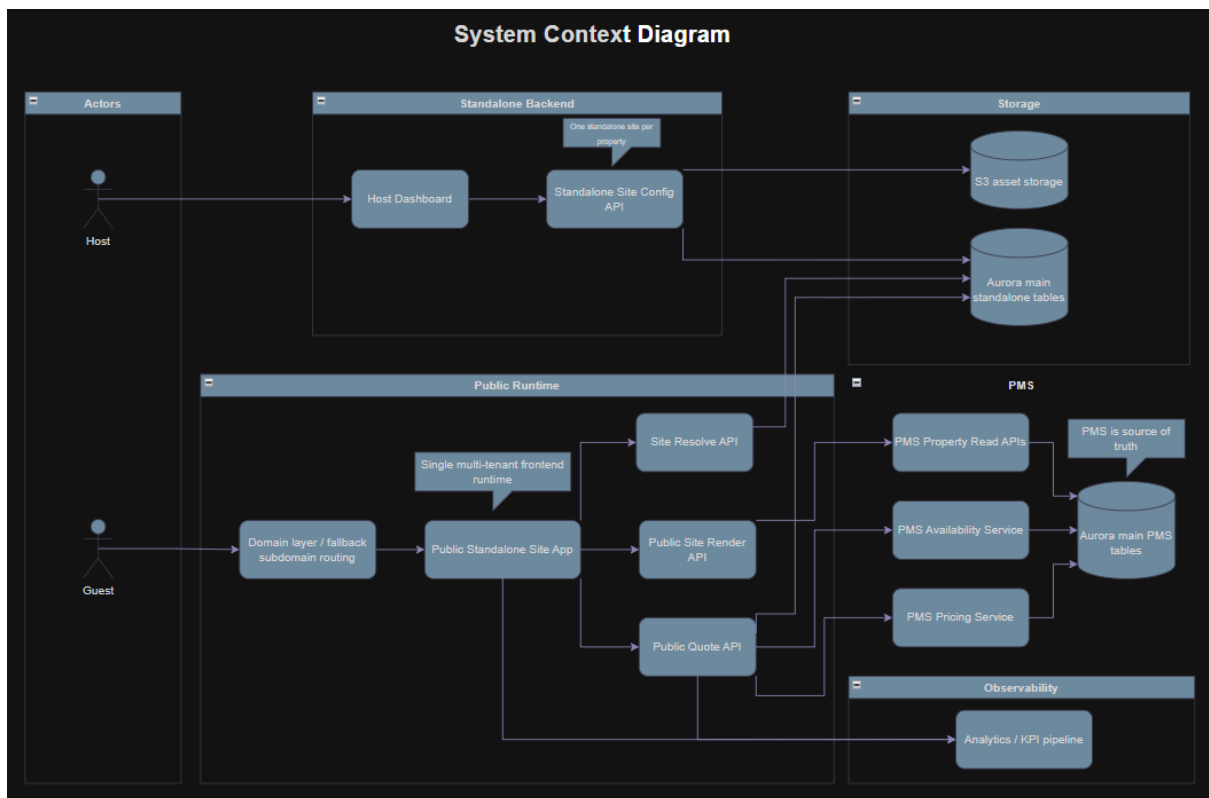
Figuur 7: Vergelijking tussen statische hosting, server-side rendering en hybride hosting

KPI Analytics Pipeline om de verzameling van data in een visuele stroming weer te geven. Zo is te zien waar data van afkomstig is en hoe we deze verzamelen.



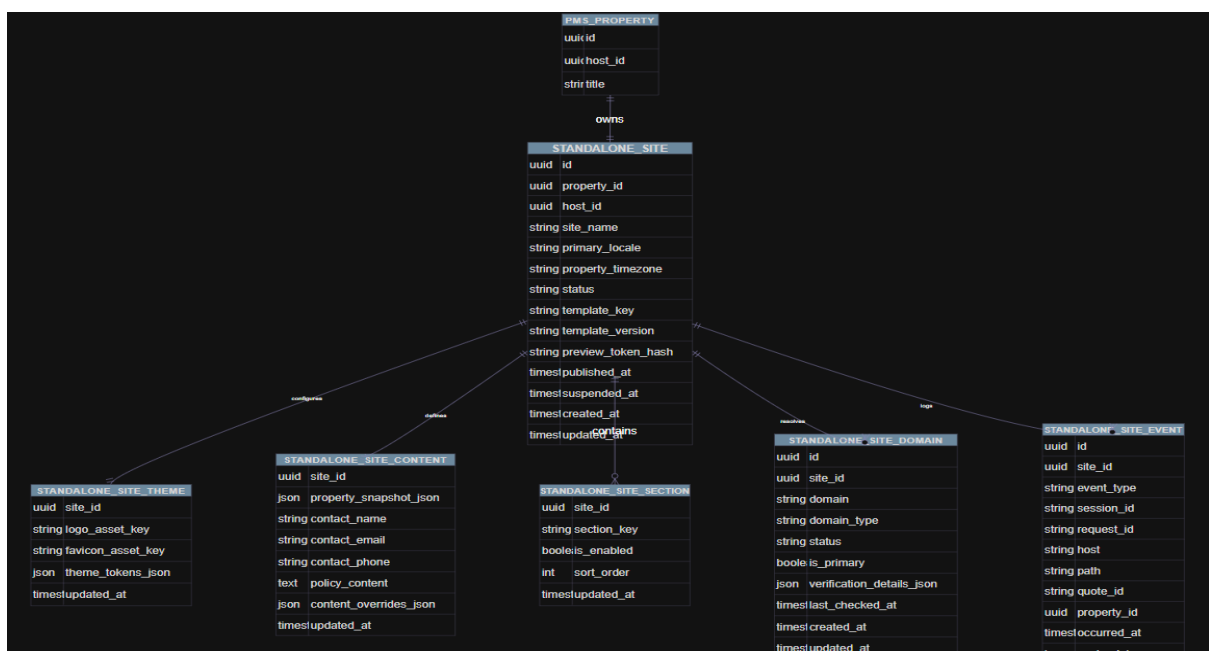
Figuur 8: KPI Analytics Pipeline

System Context Diagram is opgesteld om een duidelijke scheiding zichtbaar te maken tussen host en gast, en wat zij te zien krijgen. Dit voorbeeld van datascheiding wordt gehanteerd, zodat deze feature geen losstaand systeem wordt dat eigen waarheid gaat bijhouden over prijs en boekingen.



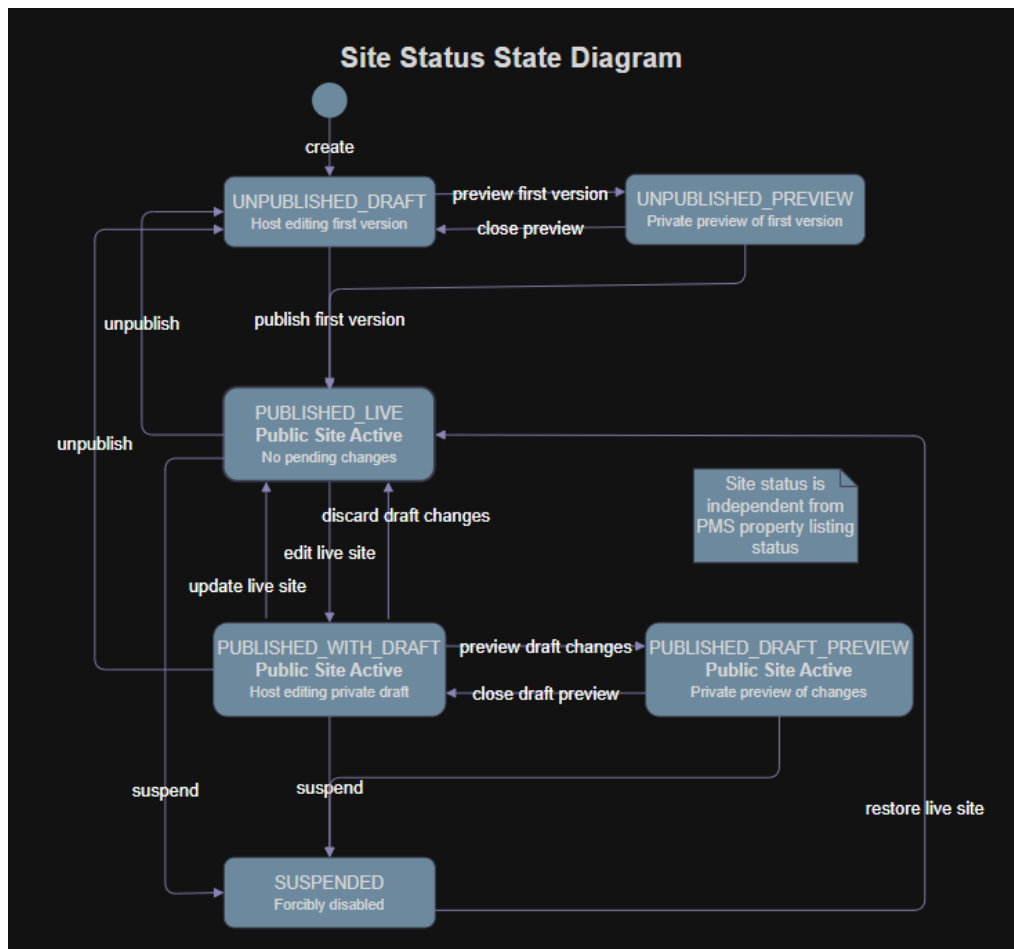
Figuur 9: System Context Diagram

Het onderstaande ERD (Entity Relationship Diagram) biedt inzicht over hoe AWS geconfigureerd moet worden om de Direct Booking Website data op te kunnen slaan, met betrekking tot tabelverbintenis, hun attributen en relaties.



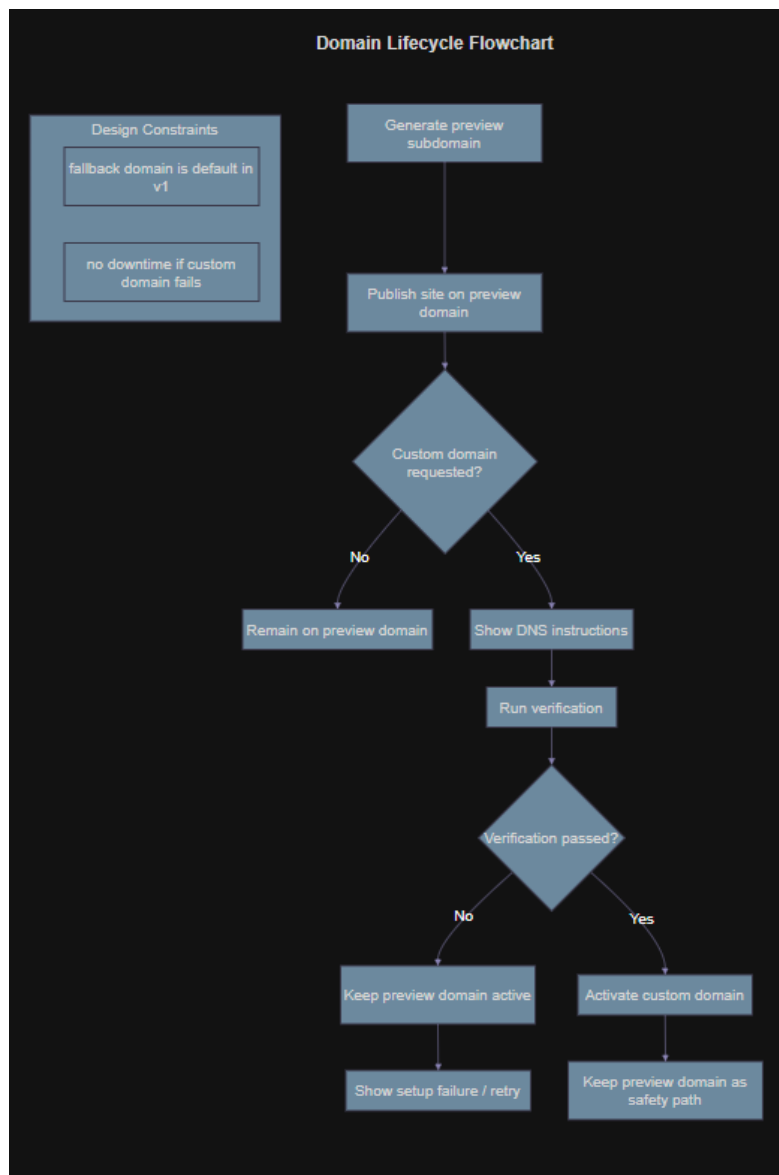
Figuur 10: Entity Relationship Diagram

Het Site Status Diagram is opgesteld om te laten zien welke statuses doorlopen worden bij deze feature, zoals creatie, update en publicatie.



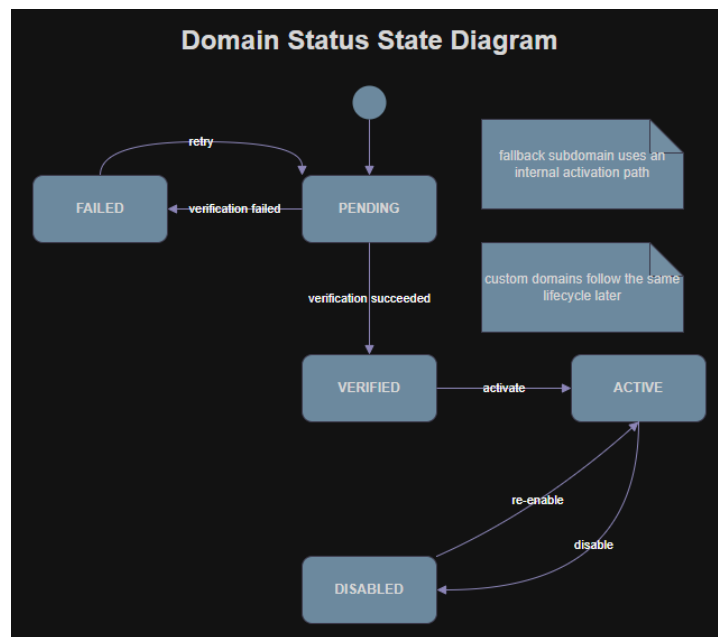
Figuur 11: Site Status State Diagram

De Domain Lifecycle Flowchart is opgesteld om de stroom van publieke websites te visualiseren. De flowchart laat zien dat er een mogelijkheid is tot custom domain, maar dat de live domein site de standaard route is.



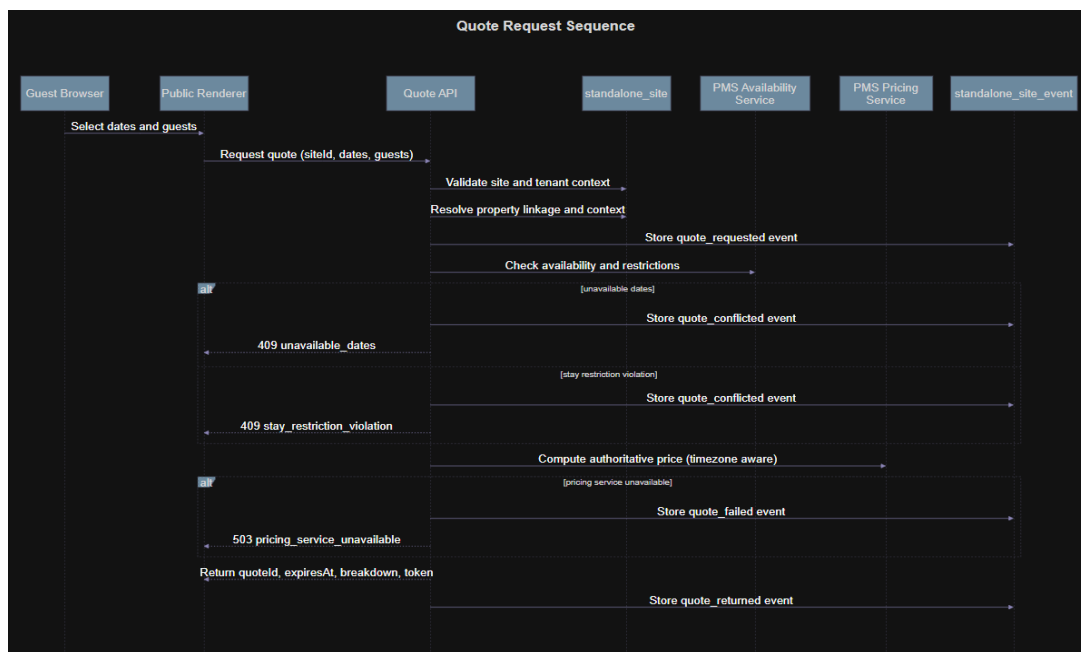
Figuur 12: Domain Lifecycle Flowchart

De status van een domein wordt hieronder toegelicht in dit Domain Status State Diagram. De status kan bestaan uit pending, verified, active, failed en disabled wat aangeeft of een website zich bevindt in draft of gepubliceerd is.



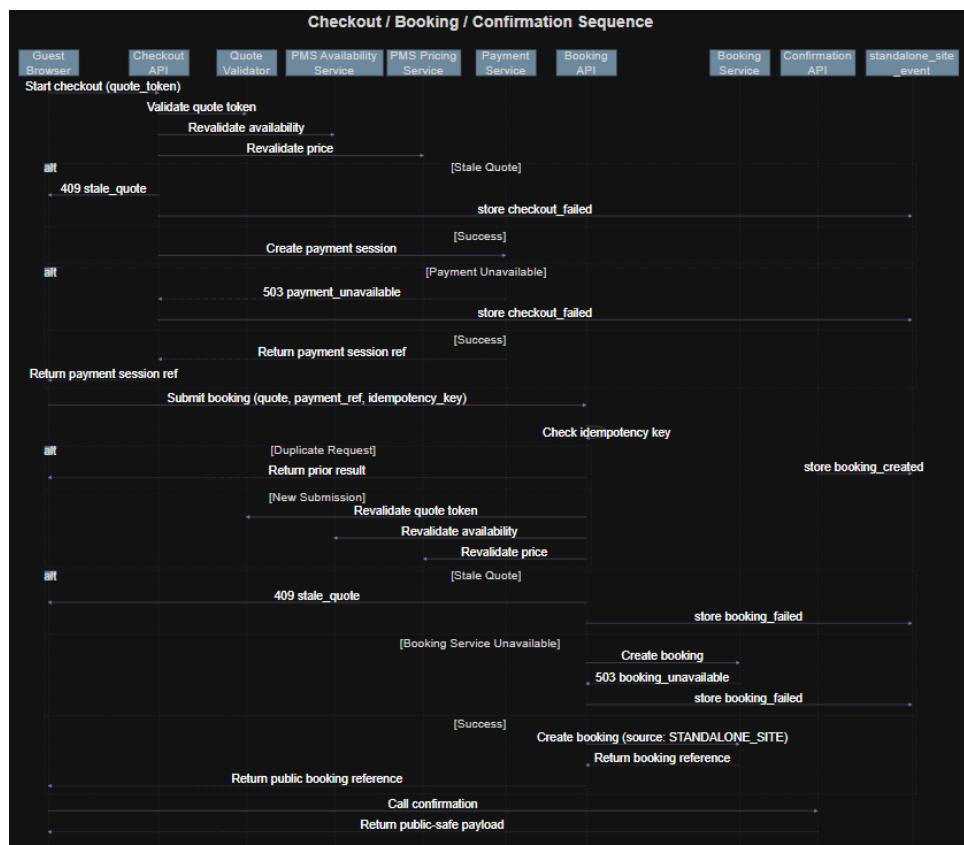
Figuur 13: Domain Status State Diagram

Het diagram hieronder licht de Quote Request Sequence toe. Allereerst wordt achterhaald bij welke property en website de aanvraag behoort. Vervolgens controleert de Quote API welk PMS gekoppeld is aan de website en wordt de beschikbaarheid vastgesteld door PMS Availability Service en de prijs door PMS Pricing Service.



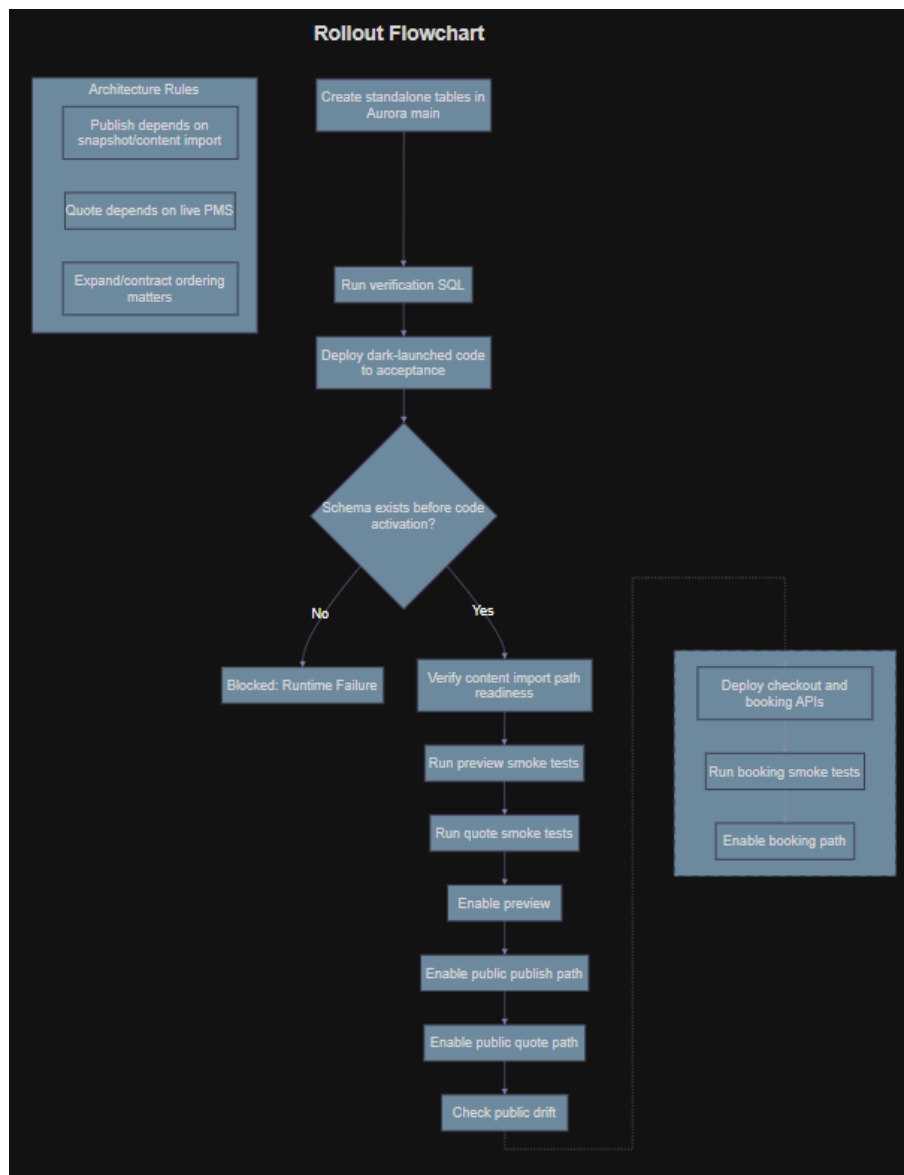
Figuur 14: Quote Request Sequence

Dit onderstaande diagram toont het verversen van data voordat een boeking definitief verklaard wordt. Voor het huidige ontwerp is een goede basis van belang ter voorbereiding van toekomstige aansluiting van directe boekingen op de gekozen structuur.



Figuur 15: Checkout / Booking / Confirmation Sequence

Om breuken in bestaande functionaliteiten te voorkomen is er een specifieke rollout volgorde opgezet die verduidelijkt wordt in het onderstaande Rollout Flowchart. In dit Figuur wordt de configuratie stroom visueel gerepresenteerd.



Figuur 16: Rollout Flowchart

Tabellen

Deze tabel is opgesteld om verduidelijking te bieden omtrent KPI's op gebied van betrekking, meting en betekenis.

Criterion	KPI / meting	Betekenis
Schaalbaarheid	time_to_publish_p95	Meet hoe snel een website in 95% van de gevallen gepubliceerd kan worden.
Kosten	cost_per_active_site_per_month	Geeft een indicatie van de gemiddelde operationele kosten per actieve website per maand.
Performance	site_lcp_p75 (mobile, tablet & desktop)	Meet hoe snel de belangrijkste zichtbare content geladen wordt.
Betrouwbaarheid	fallback_subdomain_availability	Laat zien in hoeverre terugval domeinen beschikbaar en bereikbaar blijven.
Correctheid	quote_to_charge_mismatch_rate	Geeft aan hoe vaak er verschil ontstaat tussen verwachte prijsinformatie en uiteindelijke prijsafhandeling.
Toekomstige betrouwbaarheid (v2)	booking_api_error_rate	Meet hoe vaak fouten optreden tijdens boekingen via api, zodra boeken een mogelijkheid is
Gebruikerservaring (v2)	booking_funnel_completion_rate	Laat zien welk deel van gebruikers de boeking stroom afrondt
Domeinbeheer (v2)	custom_domain_setup_succes_rate	Meet hoe vaak het koppelen van eigen domein succesvol verloopt.
Praktische validatie	end-to-end testing	Test of de functionaliteit van

		bouwen, publiceren en uitlezen correct werkt in praktijk scenario's.
--	--	--

Tabel 1: KPI's en betekenis binnen validatie van de Direct Booking Website

Verduidelijking van het stappenplan om eerst de verschillende mogelijkheden te vergelijken en pas daarna te realiseren, waardoor een betere basis ontstaat voor de uiteindelijke implementatie.

Methode	Uitleg
Vereisten analyse	In kaart brengen van functionele- en technische eisen, incl stakeholder behoeften en v1-scope afbakening
Oplossingsrichtingen	Afweging van architectuur/strategie: template-architecturen, hostingmodellen, domeinbeheer
Vergelijking m.b.t. basiscriteria	Beoordelen van alternatieven op criteria zoals kosten, schaalbaarheid, performance, onderhoudbaarheid, complexiteit en risico's
Ontwerp en realisatie	Uitwerking van keuzes in een concreet ontwerp en het ontwikkelen van een v1 MVP
Validatie	Testen/validatie van keuze d.m.v. KPI's, testscenario's en gebruikssituaties, waaronder site publicatie en correcte weergave van beschikbaarheid en prijs

Tabel 2: Onderzoeksstappen

Deze tabel is opgesteld om de verschillen tussen de onderzochte opties overzichtelijk te maken en bevat de belangrijkste eigenschappen.

Aspect	Vastgestelde templates	Block-based templates	Vrije builder
Vrijheid voor host	Laag	Gemiddeld	Hoog
Technische complexiteit	Laag	Gemiddeld	Hoog
Beheersbaarheid	Hoog	Gemiddeld	Laag
Testbaarheid	Hoog	Gemiddeld	Laag
Achterwaartse compatibiliteit	Goed beheersbaar	Complexer	Moeilijk
Implementatie snelheid	hoog	Gemiddeld	Laag
geschiktheid voor v1	Hoog	Middelmatig	laag
Toekomstige uitbreidbaarheid	Goed, mits gedeeld contract	Goed	Hoog, maar risicovoller

Tabel 3: Vergelijking van template architecturen

Tabel 4 toont de scheiding van data met betrekking tot het PMS.

Content Mapping Tabel			
Bronveld	Eigenaar	Standalone-sectie	Notities
property.title	PMS-bron -> standalone gepubliceerde snapshot	hero	geïmporteerd bij publish/refresh
property.subtitle	PMS-bron -> standalone gepubliceerde snapshot	hero	geïmporteerd bij publish/refresh
property.description	PMS-bron -> standalone gepubliceerde snapshot	hero / intro_highlights	geen niet-bestaande about-sectie gebruiken
images[]	PMS-bron -> standalone gepubliceerde snapshot	gallery	geïmporteerd bij publish/refresh
amenities[]	PMS-bron -> standalone gepubliceerde snapshot	amenities	geïmporteerd bij publish/refresh
location.city / location.country	PMS-bron -> standalone gepubliceerde snapshot	location	volledig adres alleen tonen als beleid dit toestaat
rules[]	PMS-bron -> standalone gepubliceerde snapshot	house_rules	geïmporteerd bij publish/refresh
availability.window	PMS-bron -> standalone weergavesnapshot	availability_quote	alleen voor weergave; quote API valideert live
property.timezone	PMS-bron -> standalone quote-context	availability_quote	opgeslagen op standalone_site; leidend voor quote-gedrag
site.name	Standalone	globaal / hero	door host beheerde sitenaam
logo.asset_key	Standalone	globaal / hero	S3-brand asset
favicon.asset_key	Standalone	globaal	S3-browser asset
primary.locale	Standalone	globaal	één sitetaal in v1 (Engels)
contact.name	Standalone	contact	publieke override
contact.email	Standalone	contact	publieke override
contact.phone	Standalone	contact	publieke override
policy.content	Standalone	policy	publieke override
content.overrides.json	Standalone	meerdere secties	override-laag boven geïmporteerde snapshot waar toegestaan
theme.tokens.json	Standalone	alle secties	runtime theme tokens

Tabel 4: Content Mapping Tabel

Risico matrix opgesteld om foutscenario's uit te stippen, zodat er minder verrassingen ontstaan tijdens de ontwikkeling van de feature.

Risico Matrix				
Risico	Oorzaak	Invloed	Voorzorg	Detectie
Verouderde content op live site	PMS-content verandert na publish of refresh gebeurt te laat	Verkeerde titel, foto's, beschrijving of voorzieningen zichtbaar	Snapshot refresh + republish flow met zichtbare versiebeheer	Monitoring op snapshot-leefijd + refresh audit logs
Prijs of beschikbaarheid klopt niet meer	Data verandert na genereren van quote	Gast ziet verkeerde prijs	Korte expiry + hervalidatie vóór boeken	Quote conflict ratio + stale-quote metrics
Site niet bereikbaar of verkeerde site geladen	Foutle DNS / mapping	Geen site of verkeerde tenant zichtbaar	Unieke domain index + validatie + fallback domein	Resolve logs + synthetic host checks
Data van ene tenant zichtbaar bij andere	Host resolution bug / geen tenant validatie	Datalek (groot incident)	Resolve op host + valideer site + noot client propertyId vertrouwen	Cross-tenant mismatch alarmen + request tracing
Verkeerde database schema wordt gebruikt	Queries raken public i.p.v. main	Foutle data / missende records	Expliciet main.table_name + verificatie SQL + tests	Informatie checks + rollout verificatie
Verkeerde site wordt geserveerd (v2)	Cache key niet gebaseerd op host	Cross-tenant content lek	Host-aware cache keys + geen path-only caching	Synthetische oplossende checks + cache anomaly monitoring
Fake host request manipuleert routing	Onverwachte / vervalste host headers	Tenant kaping / verkeerde content	Host toelatinglijst + strenge validatie	vreemde of verdachte host logs + mismatch alerts
Manipulatie van prijs via client (v2)	Client past quote aan	Verkeerde prijs / inconsistentie	Alleen Server-side prijsbepaling	Quote vs booking mismatch checks + logs
Overbelasting (v2)	Massa aanvragen	Prestatievermindering	Rate limiting + throttling + WAF	Meldingen over aanvraagpercentages + dashboards voor misbruik
Oude quote opnieuw gebruikt (v2)	Hergebruik token	Ongeldige boeking	Vervalling + revalidatie	Invalid token metrics + replay detectie
Injectie of script exploit (v2)	Onveilige content of escaping	Sessie kaping / beschadiging	Strikte CSP + escaping + vertrouwde assets	Browser reports + frontend error monitoring

Tabel 5: Risico Matrix

De website gaat veel functionaliteiten hebben, waaronder de host de kans geven om boekingen te ontvangen en de website aan te passen naar wens. Omdat er zoveel Om te kunnen anticiperen op fouten is de onderstaande Foutmodus Tabel opgesteld. Deze tabel verduidelijkt de weergave naar de gebruiker, en toelichting voor developers, indien fouten optreden.

Foutmodus Tabel			
Faalmodus	Verwachte fallback	Gebruikersreactie	Logging
Fallback-domein inactief of fout gekoppeld	Tenant niet raden, publieke host blijft onbeschikbaar	Site niet gevonden	Resolve-fout met host en bijbehorend domein
Onbekende host	Tenant niet raden	Site niet gevonden	Resolve-fout met host header
Site geschorst	Publieke render blokkeren	Deze site is tijdelijk niet beschikbaar	Site-status en resolve-log
PMS-import van beschrijvende content mislukt tijdens publiceren of verversen	Vorige gepubliceerde versie actief houden; nieuwe versie niet activeren	Host ziet dat publiceren is mislukt, gasten blijven laatste live versie zien	publish_failed, dependency-fout, propertyId
Gepubliceerde snapshot te oud volgens freshness-drempel	Laatste gepubliceerde snapshot blijven serveren, host waarschuwen	Geen fout voor gast; (optionele) waarschuwing voor host in dashboard	Snapshot-age / freshness-metric
Gepubliceerde site mist content-snapshot	Fail closed op render	Deze site is tijdelijk niet beschikbaar	Render completeness error
Vereist asset ontbreekt of asset-key is ongeldig	Placeholder gebruiken als niet-kritisch; anders fail closed	Visuele degradatie of tijdelijk niet beschikbaar	Asset resolution error met asset-key
Templatecontract komt niet overeen met vastgepinde templateversie	Vorige geldige gepubliceerde versie behouden indien aanwezig; anders fail closed	Deze site is tijdelijk niet beschikbaar	Template resolution / contract validation error
PMS-property is niet langer geschikt of niet meer publiceerbaar terwijl site live is	Pagina zichtbaar houden vanuit laatste snapshot, maar quotes blokkeren	Deze accommodatie is nu niet boekbaar	Eligibility- en quote-dependency-log
Availability service niet beschikbaar	Quote-endpoint geeft 503, pagina blijft renderen	Beschikbaarheid is tijdelijk niet beschikbaar	Dependency error met requestId en siteld
Pricing service niet beschikbaar	Quote-endpoint geeft 503, pagina blijft renderen	Prijsinformatie is tijdelijk niet beschikbaar	Dependency error met requestId en siteld
Read-context ontbreekt of is ongeldig	Read veilig blokkeren	Read is tijdelijk niet beschikbaar	Read context validation error
Verouderde quote	Conflict teruggeven en verversen vereisen	Geselecteerde datums of prijs zijn gewijzigd; vernieuw de quote	quote_conflicted log
Preview-token ongeldig of verlopen	Preview-toegang weigeren, nieuwe preview-link nodig	Preview-link is ongeldig of verlopen	Preview token validation error

Tabel 6: Foutmodus Tabel